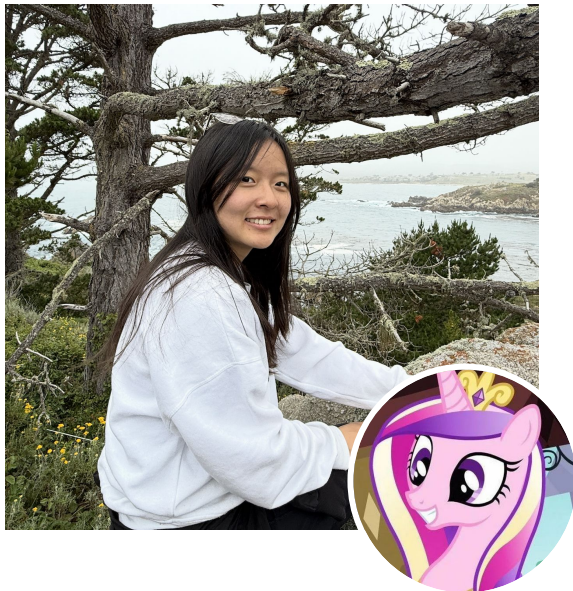


Introductions

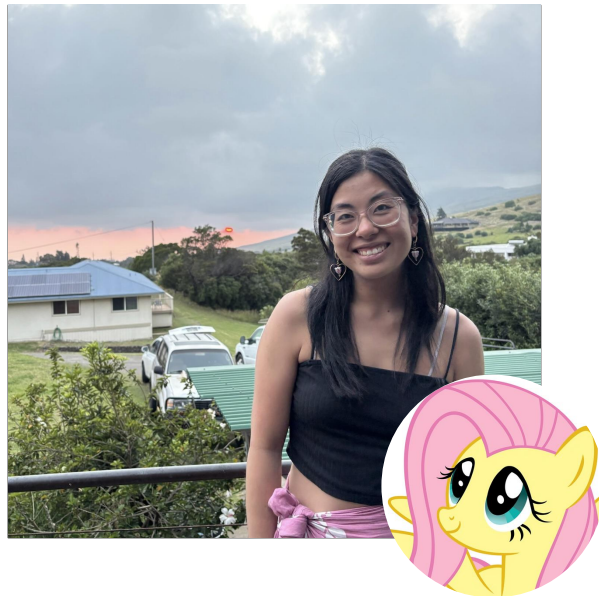


Stacey Lei

4th year, CS Major

Favorite non-CS class:

LD Arch 1



Michelle Koga

3rd year, EECS major

Favorite non-CS class:

THAI 1A



Lecture 24 (Graphs 4)

Minimum Spanning Trees

CS61B, Spring 2026 @ UC Berkeley

Stacey Lei and Michelle Koga

Slides credit: Josh Hug

Graph Problem Warmup

Lecture 25, CS61B, Fall 2025

Graph Problem Warmup

Minimum Spanning Trees

- Intro
- The Cut Property

Prim's Algorithm

- Basic Prim's (Demo)
- Optimized Prim's (Demo)
- Prim's Algorithm Code and Runtime

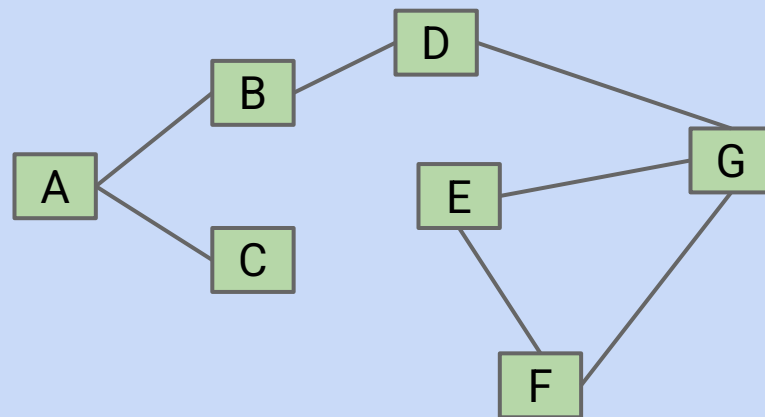
Kruskal's Algorithm:

- Basic Kruskal's (Demo)
- Optimized Kruskal's (Demo)
- Kruskal's vs. Prim's
- Kruskal's Algorithm Code and Runtime

Warm-up Problem

Given an undirected graph, determine if it contains any cycles.

- May use any data structure or algorithm from the course so far.



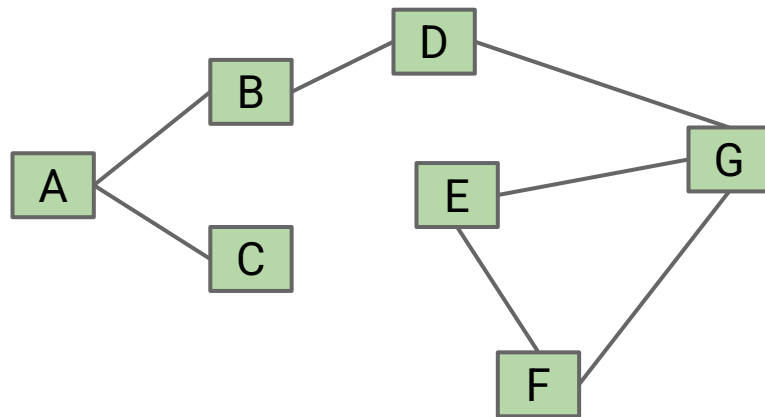
Warm-up Problem

Given an undirected graph, determine if it contains any cycles.

- May use any data structure or algorithm from the course so far.

Approach 1: Do DFS from 0 (arbitrary vertex).

- Keep going until you see a marked vertex.
- Potential danger:
 - 1 looks back at 0 and sees marked.
 - Solution: Just don't count the node you came from.

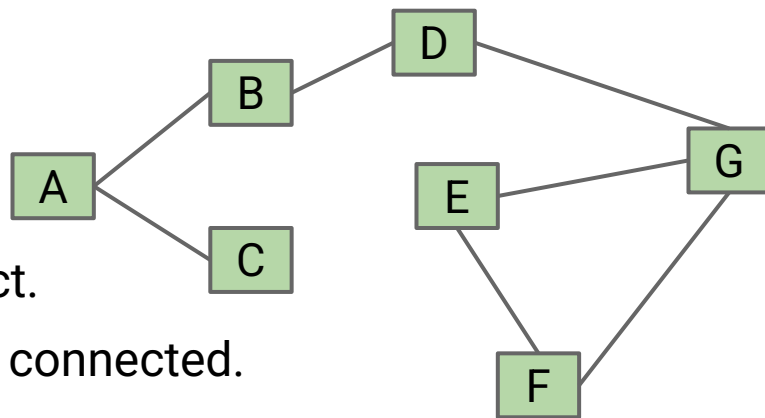


Worst case runtime: $O(V + E)$ -- do study guide problems to reinforce this.

Warm-up Problem

Given an undirected graph, determine if it contains any cycles.

- May use any data structure or algorithm from the course so far.



Approach 2: Use a WeightedQuickUnionUF object.

- For each edge, check if the two vertices are connected.
 - If not, union them.
 - If so, there is a cycle.

Worst case runtime: $O(V + E \alpha(V))$ if we have path compression.

- Here $\alpha(V)$ is the [inverse Ackermann function](#) from Disjoint Sets.
- With similar reasoning from before, we can reduce to $O(V \alpha(V))$

Minimum Spanning Trees

Intro

Lecture 24, CS61B, Spring 2026

Minimum Spanning Trees

- **Intro**
- The Cut Property

Prim's Algorithm

- Basic Prim's (Demo)
- Optimized Prim's (Demo)
- Prim's Algorithm Code and Runtime

Kruskal's Algorithm:

- Basic Kruskal's (Demo)
- Optimized Kruskal's (Demo)
- Kruskal's vs. Prim's
- Kruskal's Algorithm Code and Runtime

Spanning Trees

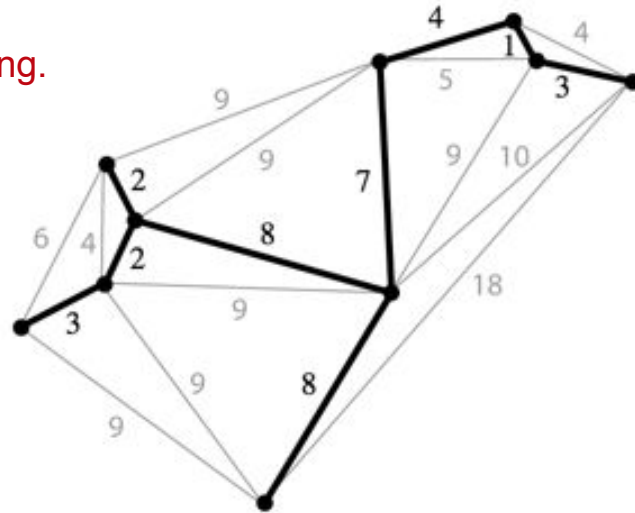
Given an **undirected** graph, a **spanning tree** T is a subgraph of G , where T :

- Is connected.
 - Is acyclic.
 - Includes all of the vertices.
- These two properties make it a tree.
- This makes it spanning.

Has $V-1$ edges (also property of a tree)

Example:

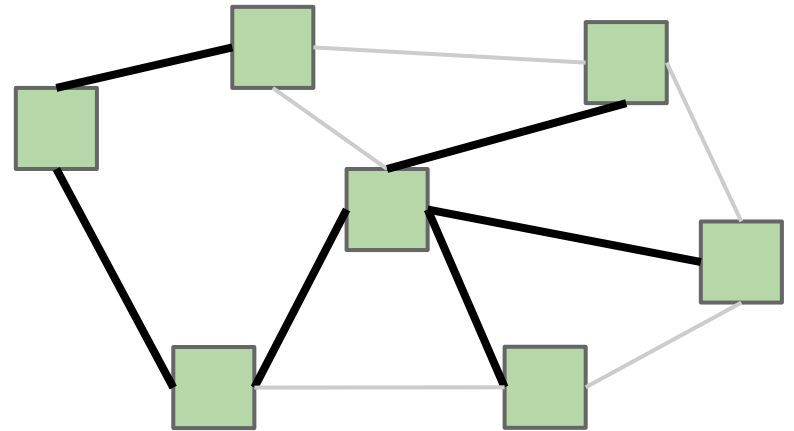
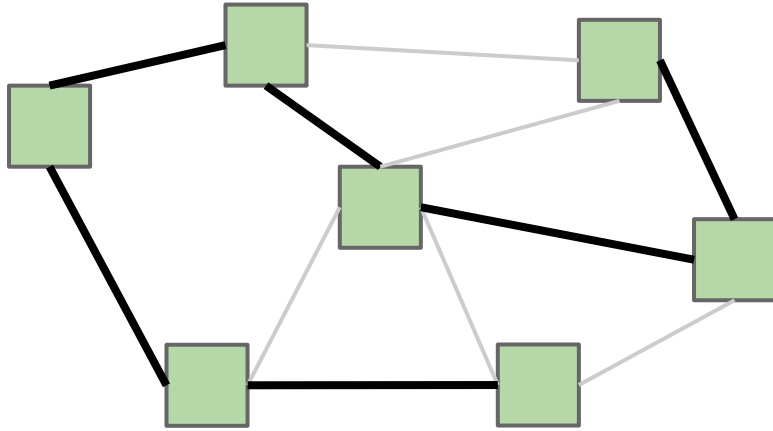
- Spanning tree is the black edges and vertices.



A **minimum spanning tree** is a spanning tree of minimum total weight.

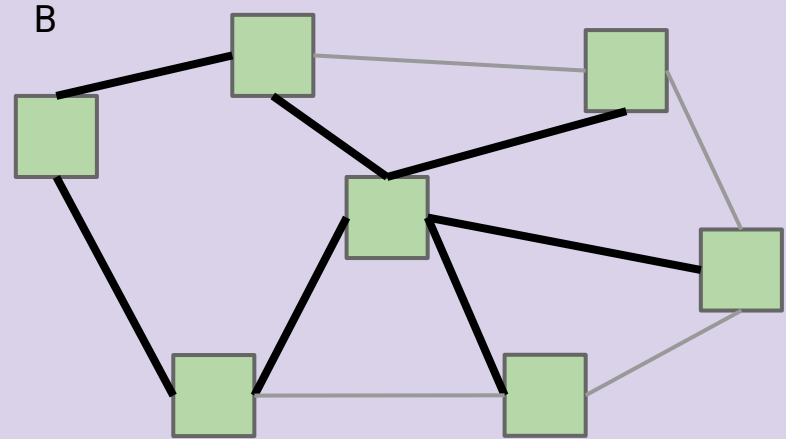
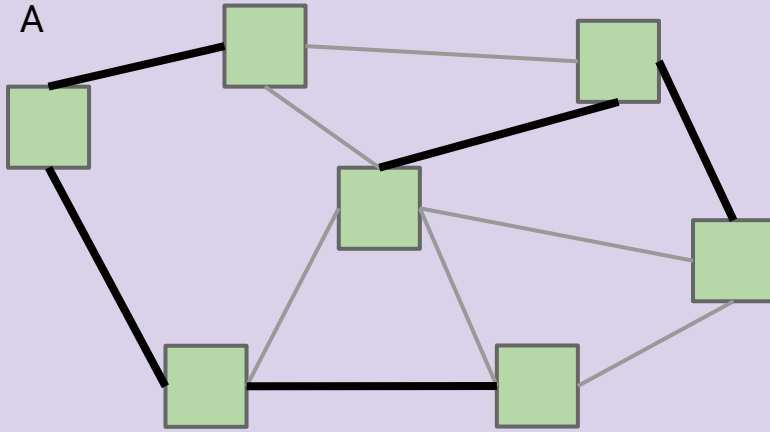
- Example: Network of power lines that connect a bunch of buildings.

Spanning Trees



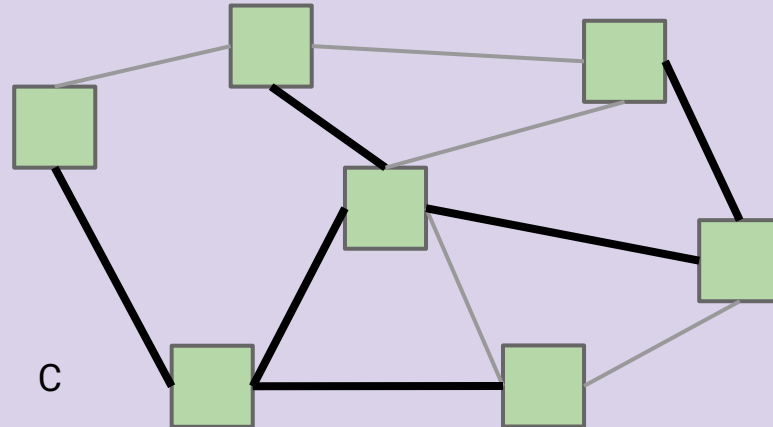
Spanning Tree:

- Is connected.
- Is acyclic.
- Includes all of the vertices.



Spanning Tree:

- Is connected.
- Is acyclic.
- Includes all of the vertices.

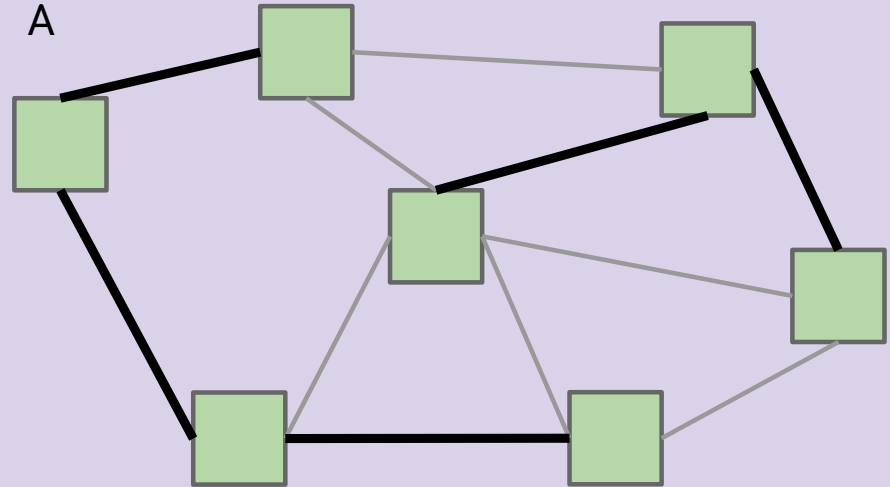


Is this a valid spanning tree?

If not, why (which property does it break)?

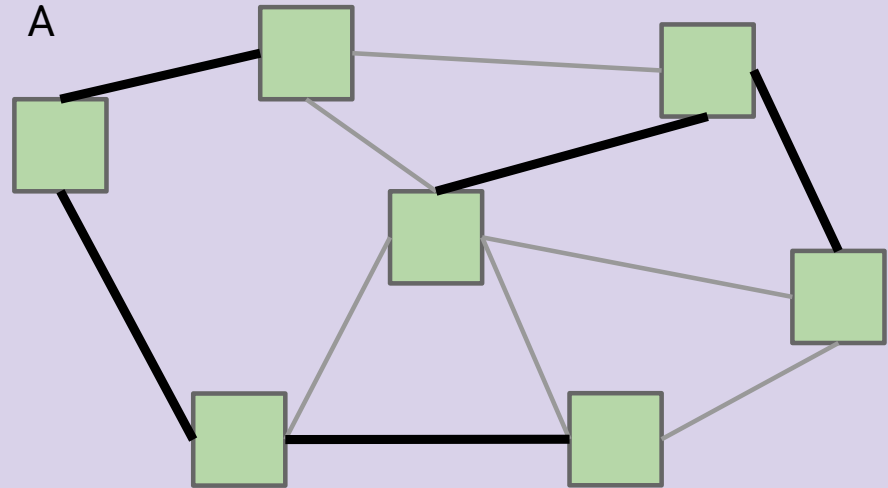
Spanning Tree:

- Is connected.
- Is acyclic.
- Includes all of the vertices.



Is this a valid spanning tree?

If not, why (which property does it break)?



Spanning Tree:

- Is connected.
- Is acyclic.
- Includes all of the vertices.

Not valid!

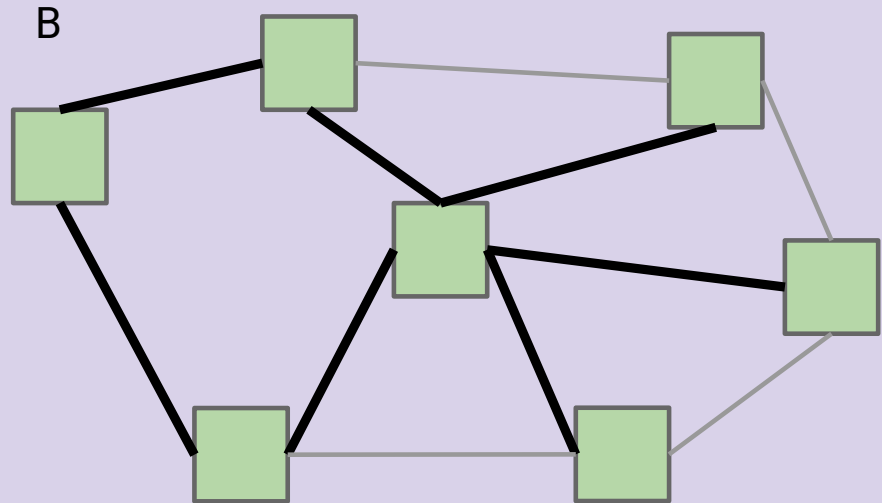
Breaks the “is connected” property

Is this a valid spanning tree?

If not, why (which property does it break)?

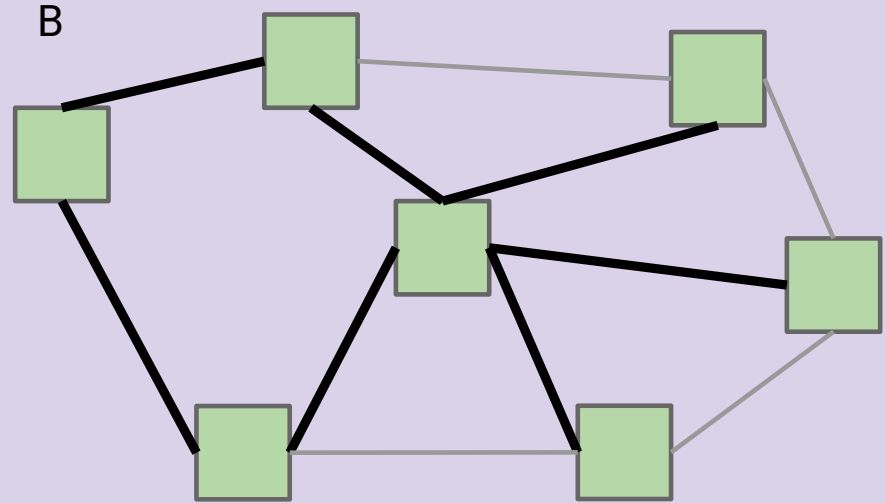
Spanning Tree:

- Is connected.
- Is acyclic.
- Includes all of the vertices.



Is this a valid spanning tree?

If not, why (which property does it break)?



Spanning Tree:

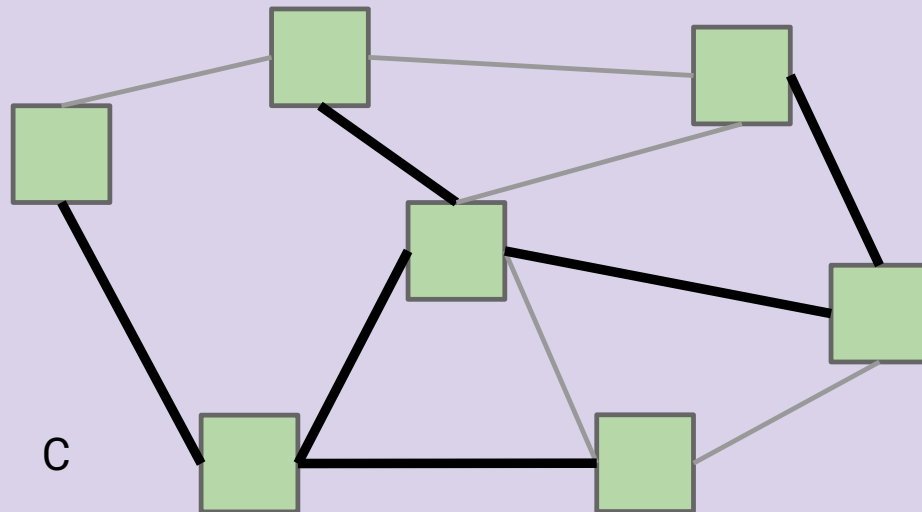
- Is connected.
- Is acyclic.
- Includes all of the vertices.

Not valid!

Breaks the “is acyclic” property

Is this a valid spanning tree?

If not, why (which property does it break)?

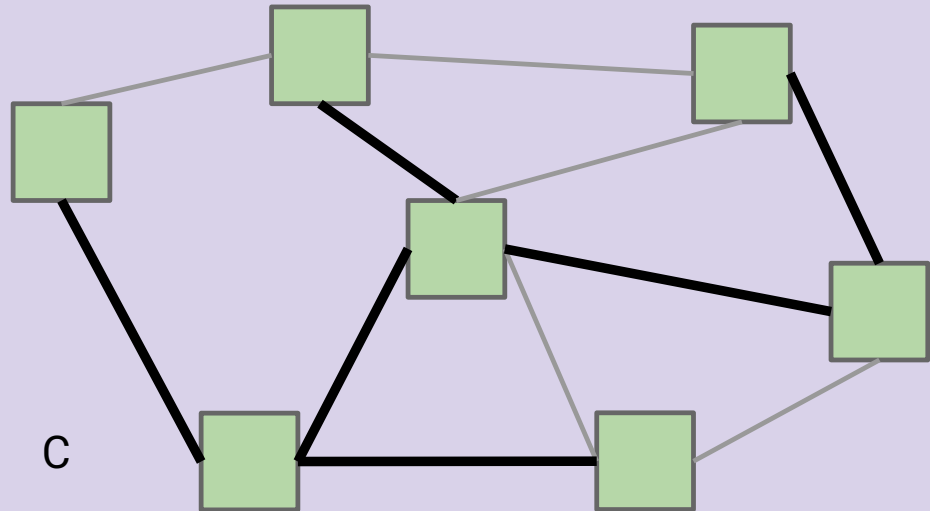


Spanning Tree:

- Is connected.
- Is acyclic.
- Includes all of the vertices.

Is this a valid spanning tree?

If not, why (which property does it break)?



Spanning Tree:

- Is connected.
- Is acyclic.
- Includes all of the vertices.

Valid!

Left: Old school handwriting recognition ([link](#))

Right: Medical imaging (e.g. arrangement of nuclei in cancer cells)

For more, see: <http://www.ics.uci.edu/~eppstein/gina/mst.html>

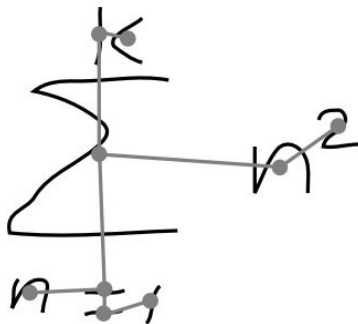
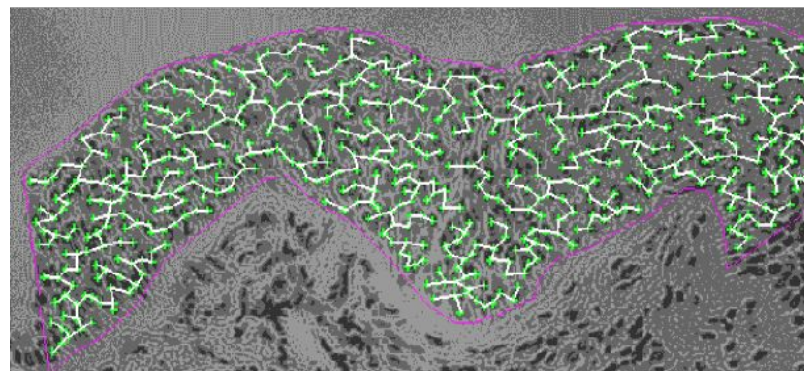
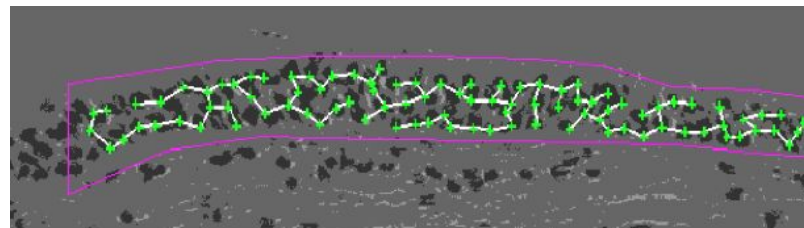


Figure 4-3: A typical minimum spanning tree



These slides are covered in the [web videos](#), but we won't cover them live.

Extra: Minimum Spanning Trees vs. Shortest Paths Trees

Lecture 24, CS61B, Spring 2026

Minimum Spanning Trees

- Intro
- The Cut Property

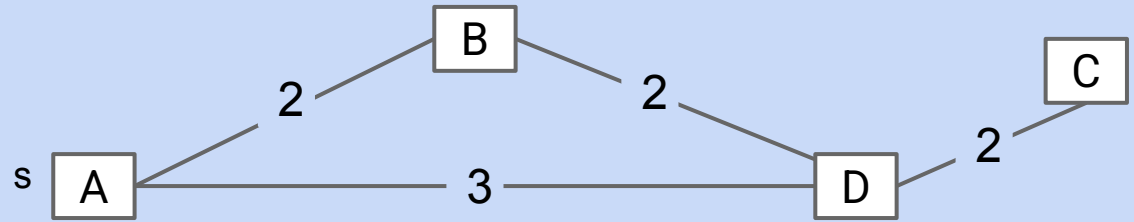
Prim's Algorithm

- Basic Prim's (Demo)
- Optimized Prim's (Demo)
- Prim's Algorithm Code and Runtime

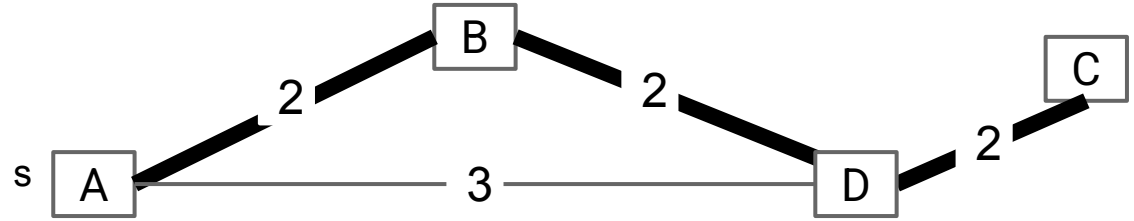
Kruskal's Algorithm:

- Basic Kruskal's (Demo)
- Optimized Kruskal's (Demo)
- Kruskal's vs. Prim's
- Kruskal's Algorithm Code and Runtime

Find the MST for the graph.

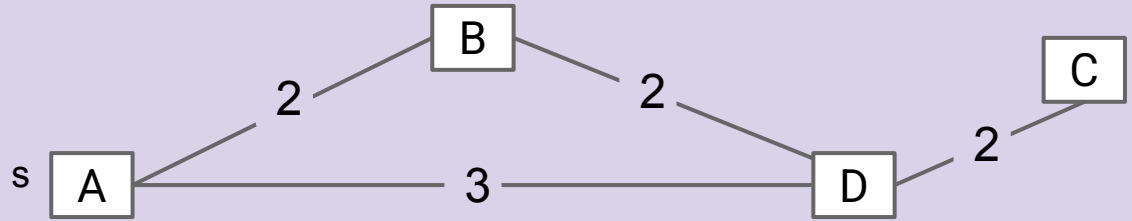


Find the MST for the graph.



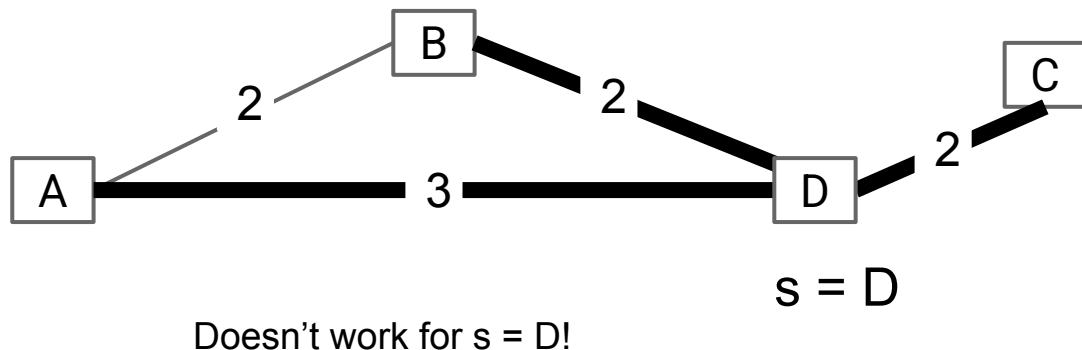
Is the MST for this graph also a shortest paths tree? If so, using which node as the starting node for this SPT?

- A. A
- B. B
- C. C
- D. D
- E. No SPT is an MST.



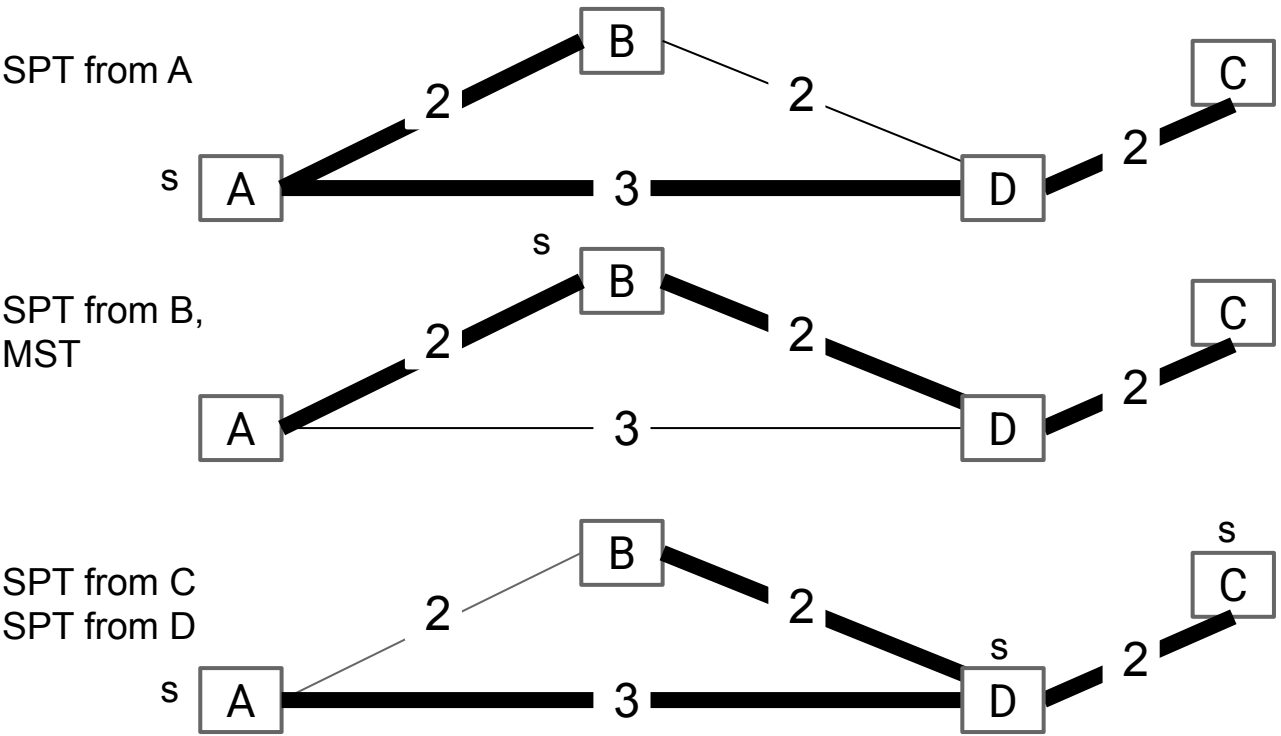
Is the MST for this graph also a shortest paths tree? If so, using which node as the starting node for this SPT?

- A. A
- B. B
- C. C
- D. D
- E. No SPT is an MST.



Is the MST for this graph also a shortest paths tree? If so, using which node as the starting node for this SPT?

- A. A
- B. B**
- C. C
- D. D
- E. No SPT is an MST.



A shortest paths tree depends on the start vertex:

- Because it tells you how to get from a source to EVERYTHING.

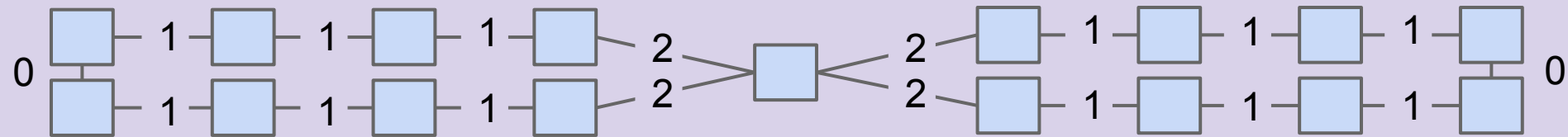
There is no source for a MST.

Nonetheless, the MST sometimes happens to be an SPT for a specific vertex.

Spanning Tree

Give a valid MST for the graph below.

- Hard B level question: Is there a node whose SPT is also the MST?

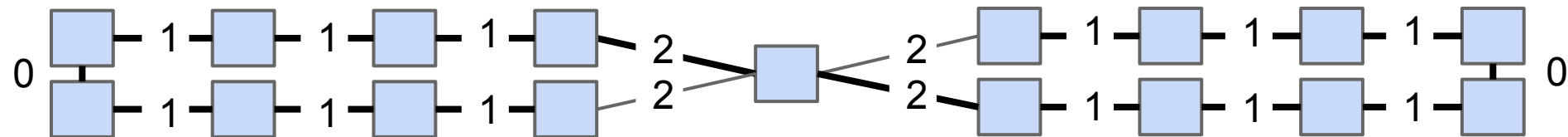


- A. Yes
- B. No

Spanning Tree

Give a valid MST for the graph below.

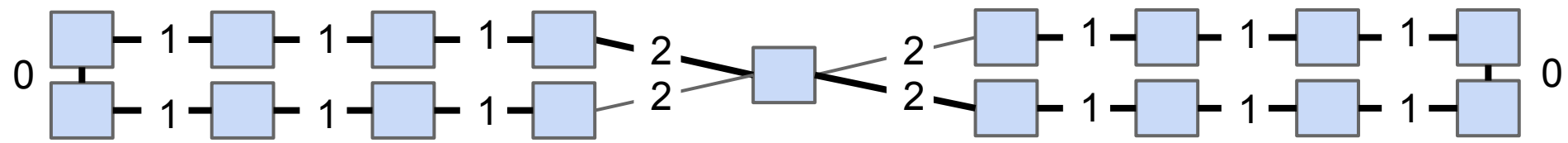
- Is there a node whose SPT is also the MST? [see next slide]



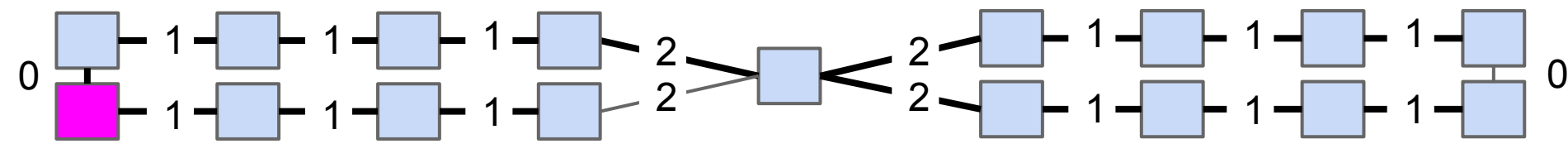
Spanning Tree

Give a valid MST for the graph below.

- Is there a node whose SPT is also the MST?
- **No!** Minimum spanning tree must include only 2 of the 2 weight edges, but the SPT always includes at least 3 of the 2 weight edges.



Example SPT from bottom left vertex:



The Cut Property

Lecture 24, CS61B, Spring 2026

Minimum Spanning Trees

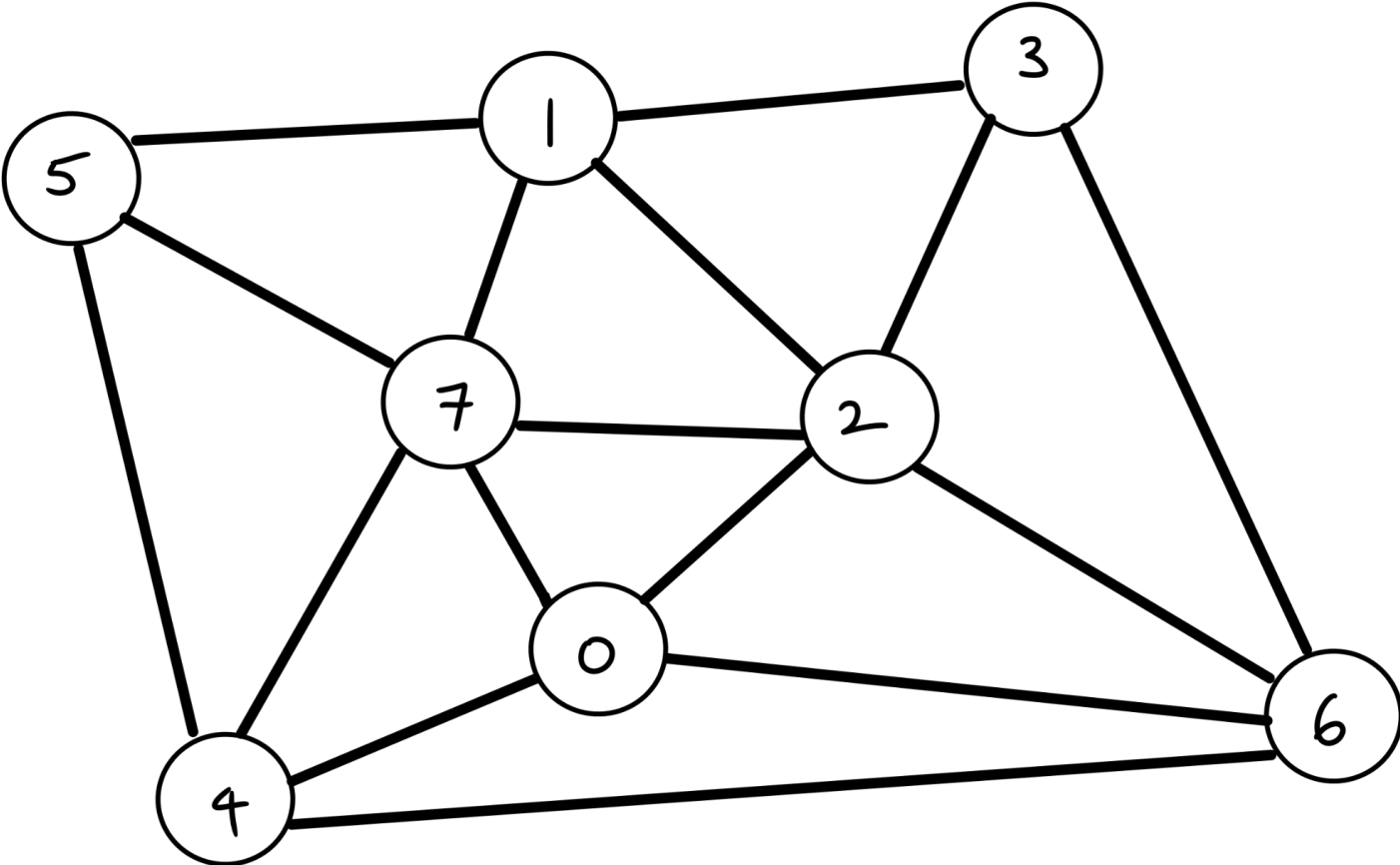
- Intro
- **The Cut Property**

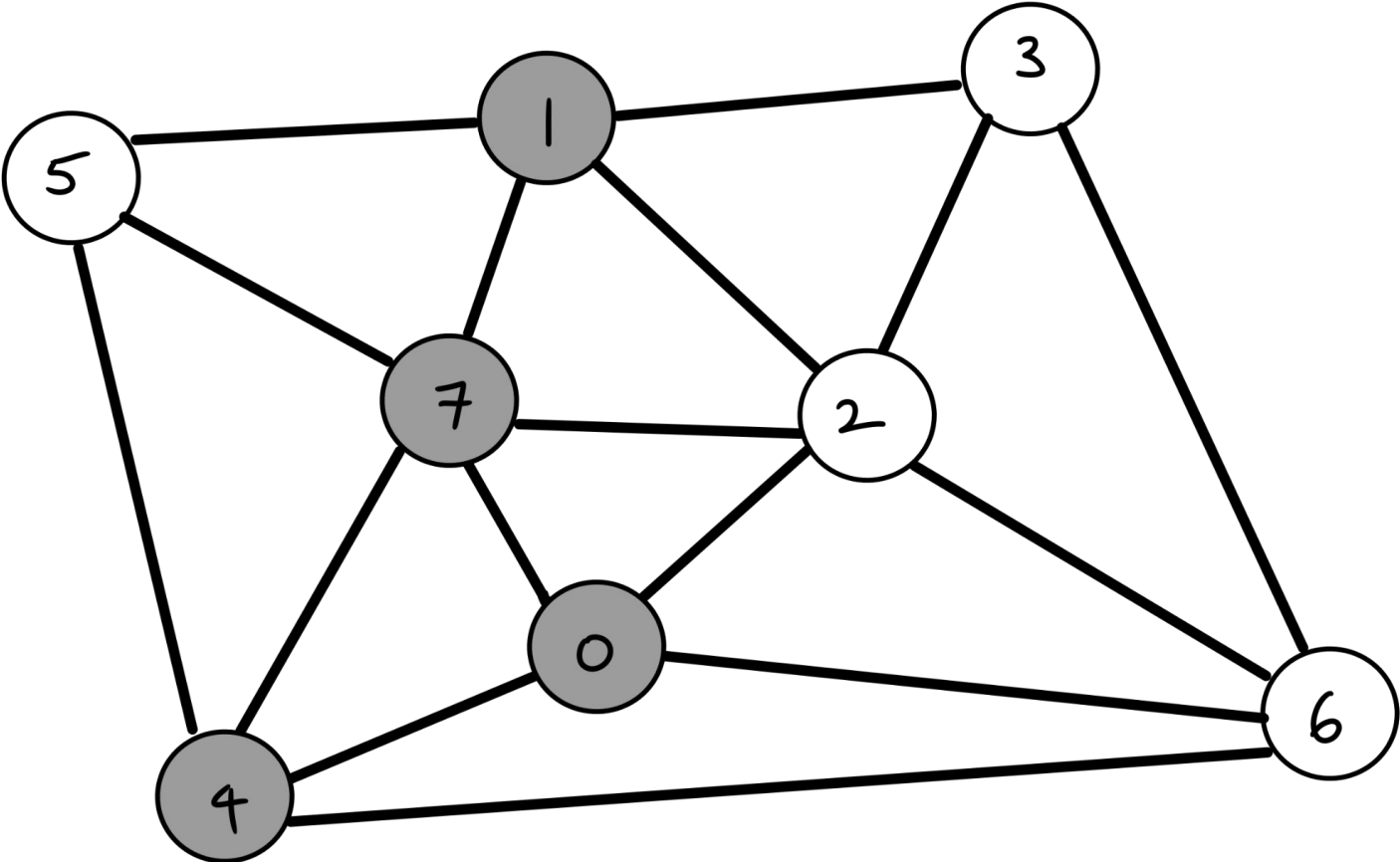
Prim's Algorithm

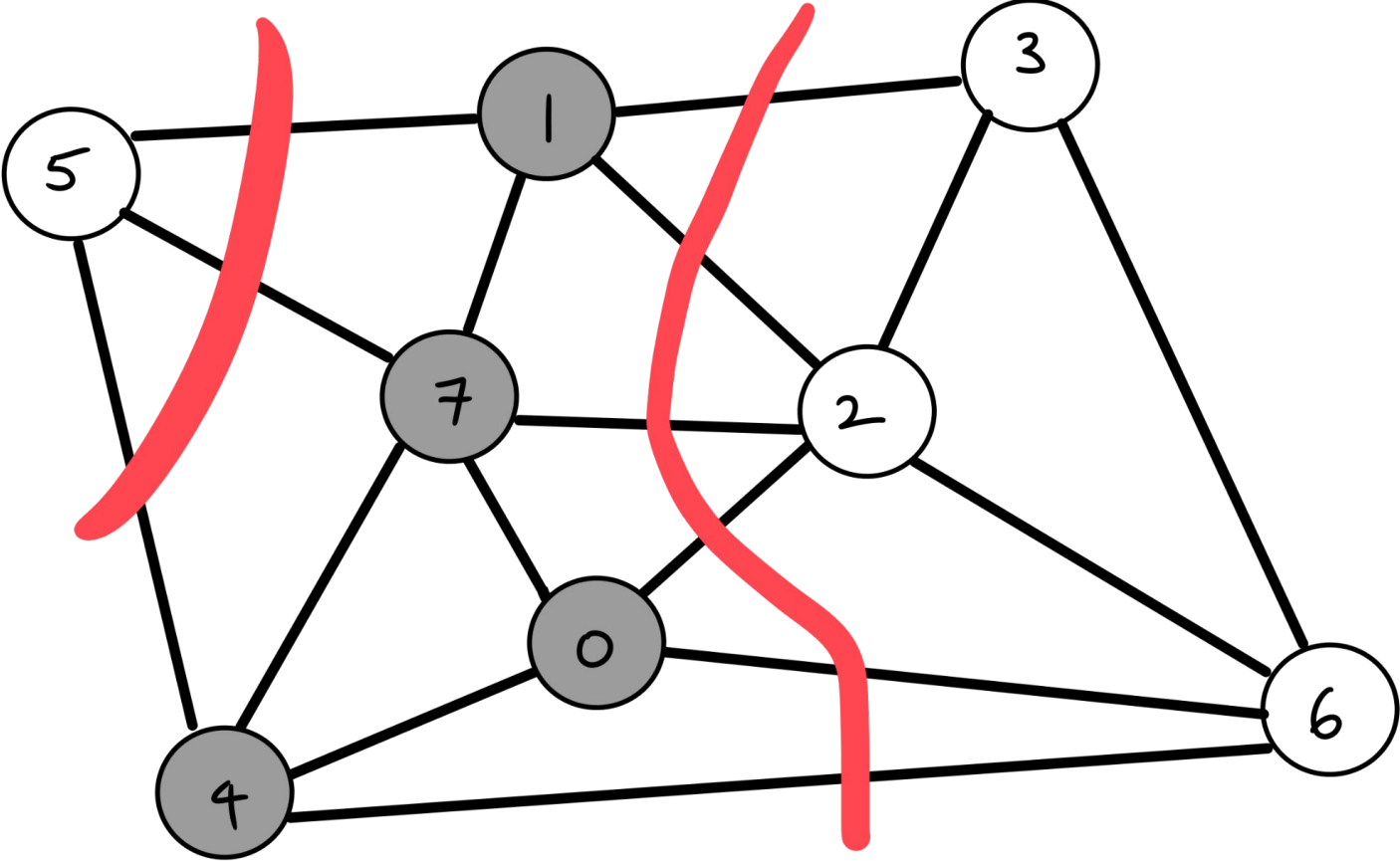
- Basic Prim's (Demo)
- Optimized Prim's (Demo)
- Prim's Algorithm Code and Runtime

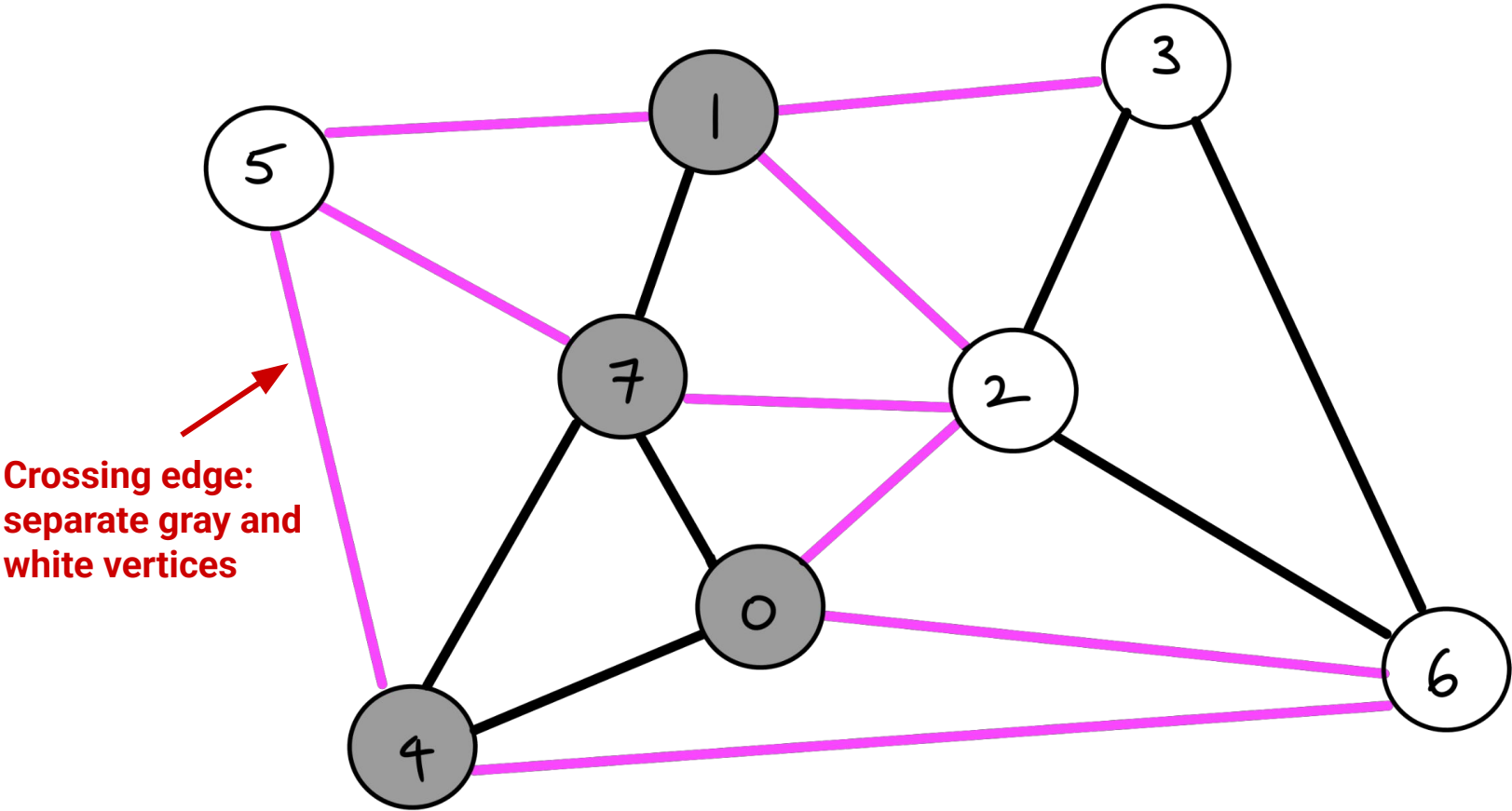
Kruskal's Algorithm:

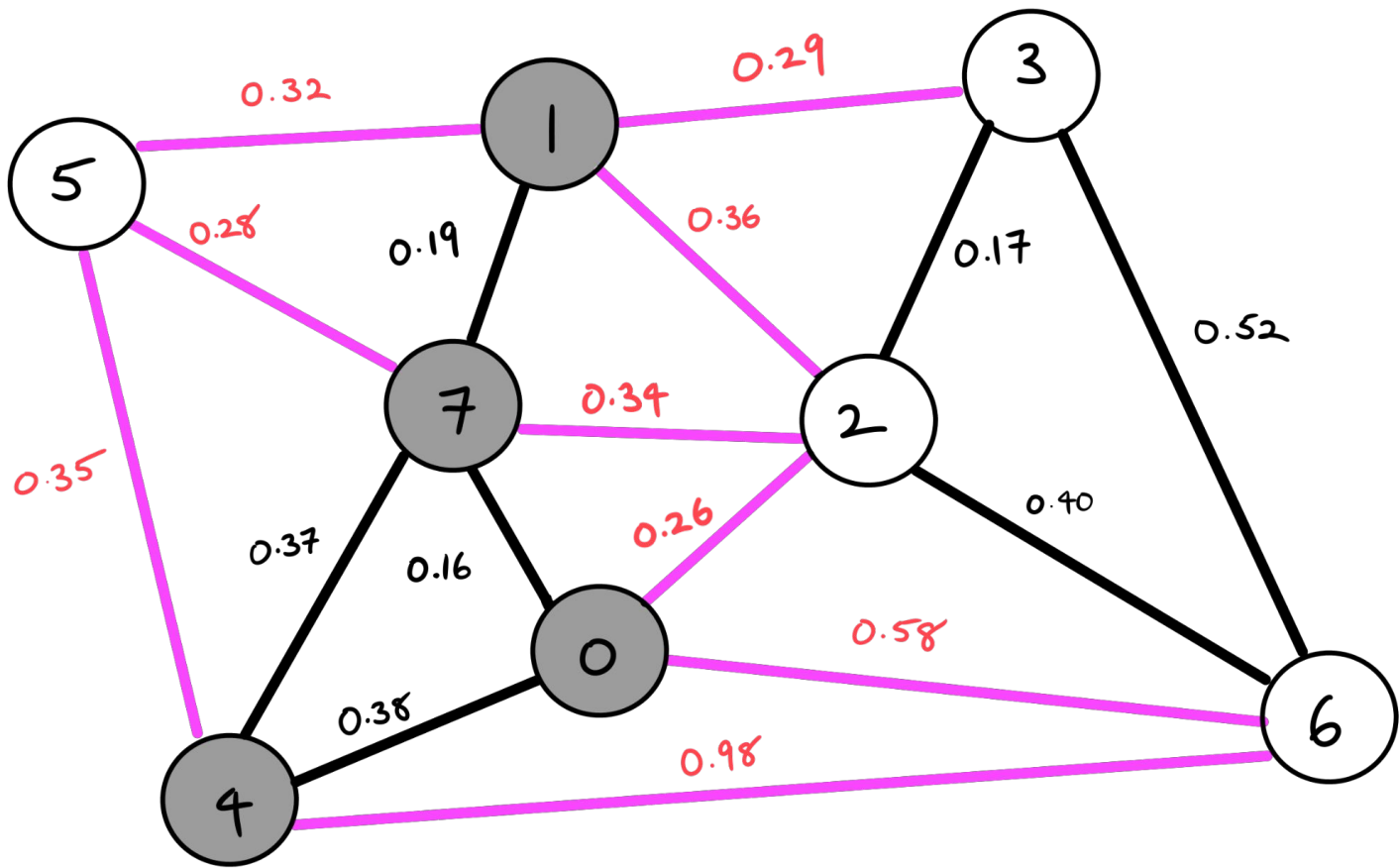
- Basic Kruskal's (Demo)
- Optimized Kruskal's (Demo)
- Kruskal's vs. Prim's
- Kruskal's Algorithm Code and Runtime



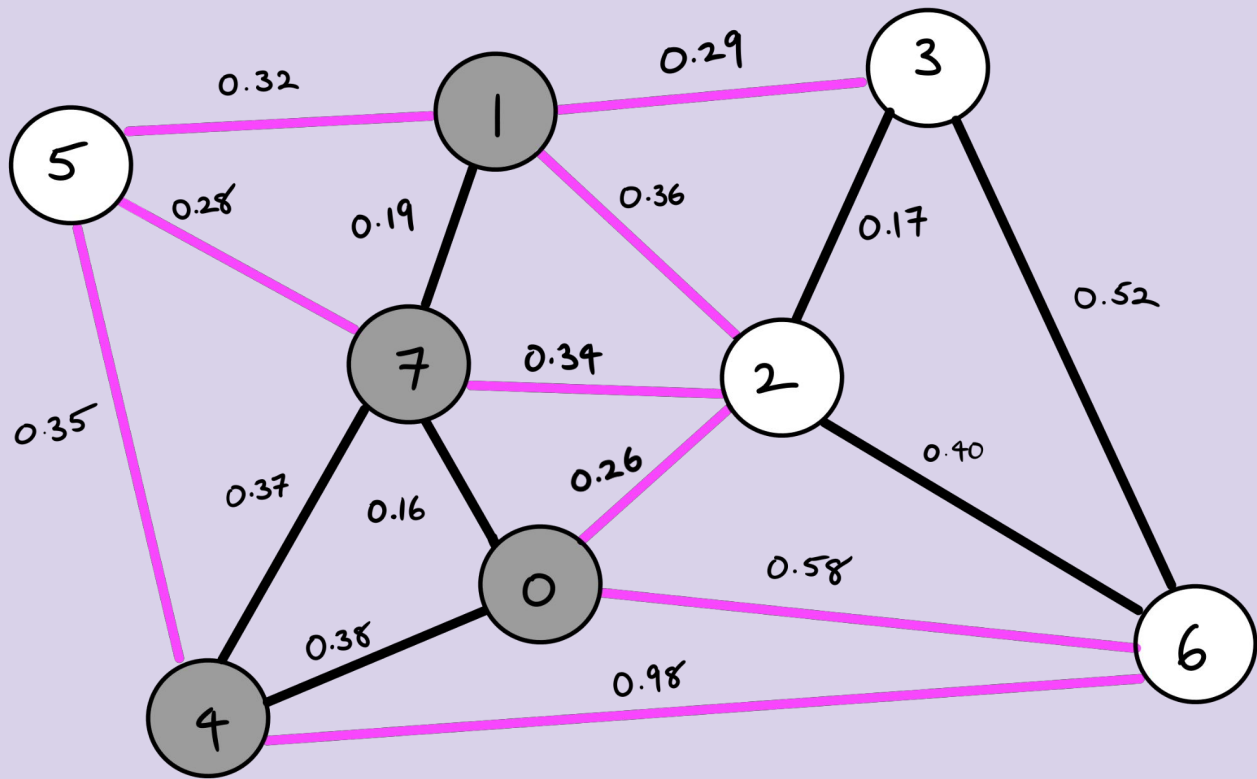








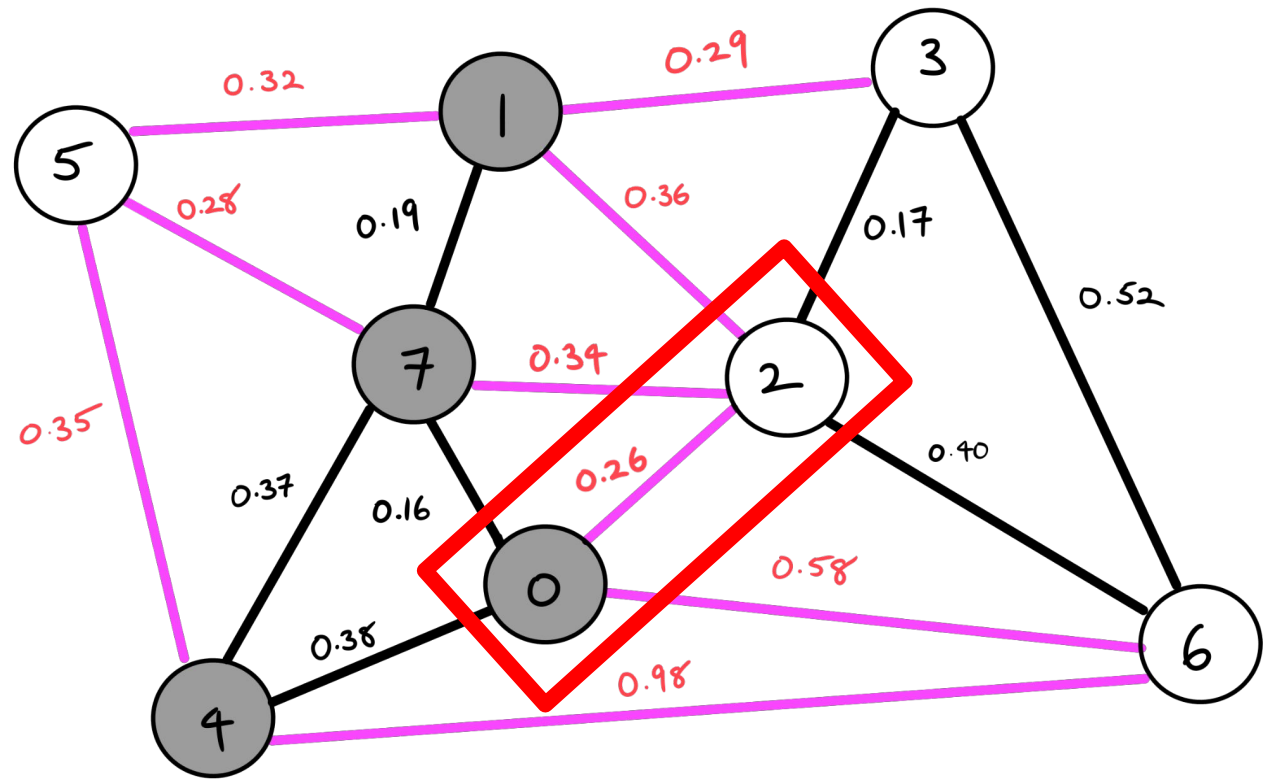
Which edge is the **minimum weight edge** that connects gray and white vertices?



0-7	0.16
2-3	0.17
1-7	0.19
0-2	0.26
5-7	0.28
1-3	0.29
1-5	0.32
2-7	0.34
4-5	0.35
1-2	0.36
4-7	0.37
0-4	0.38
6-2	0.40
3-6	0.52
6-0	0.58
6-4	0.93
4-6	0.98v

Which edge is the **minimum weight edge** that connects gray and white vertices?

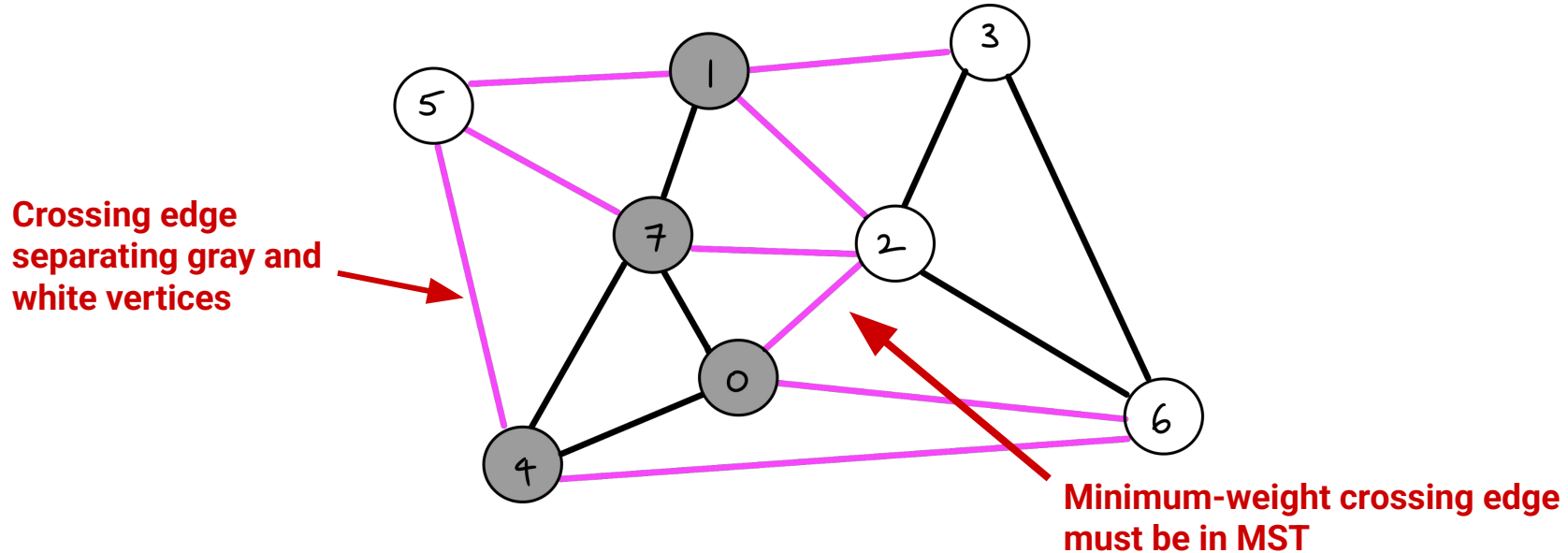
- 0-2



0-7	0.16
2-3	0.17
1-7	0.19
0-2	0.26
5-7	0.28
1-3	0.29
1-5	0.32
2-7	0.34
4-5	0.35
1-2	0.36
4-7	0.37
0-4	0.38
6-2	0.40
3-6	0.52
6-0	0.58
6-4	0.93
4-6	0.98

A Useful Tool for Finding the MST: Cut Property

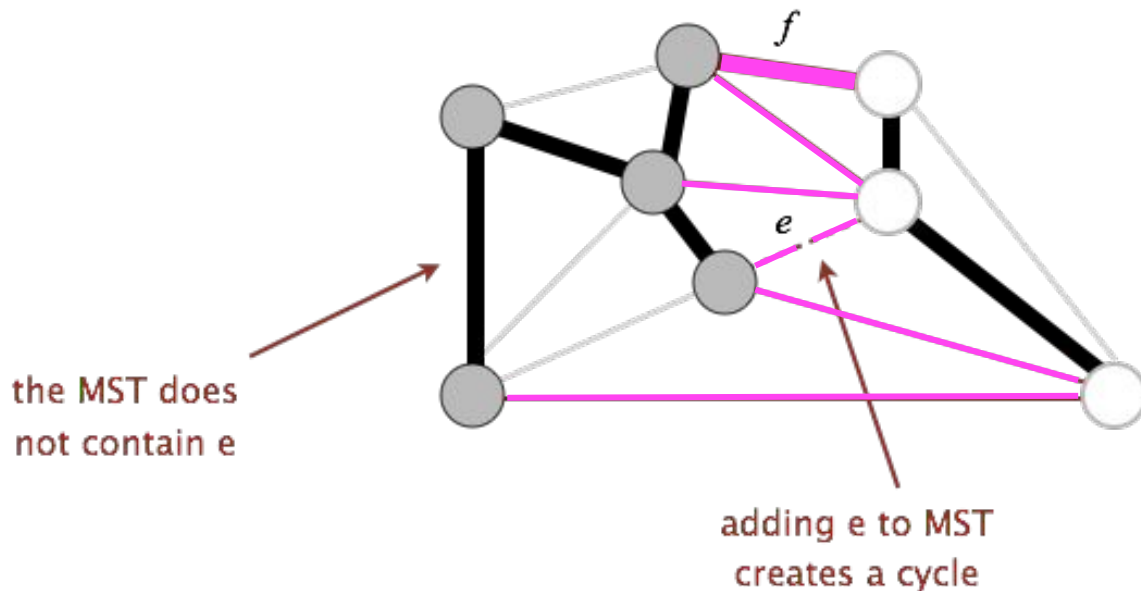
- A **cut** is an assignment of a graph's nodes to two non-empty sets.
- A **crossing edge** is an edge which connects a node from one set to a node from the other set.



Cut property: Given any cut, minimum weight crossing edge is in the MST.

Suppose that the minimum crossing edge e were not in the MST.

- Adding e to the MST creates a cycle.
- Some other edge f must also be a crossing edge.
- Removing f and adding e is a lower weight spanning tree.
- Contradiction!

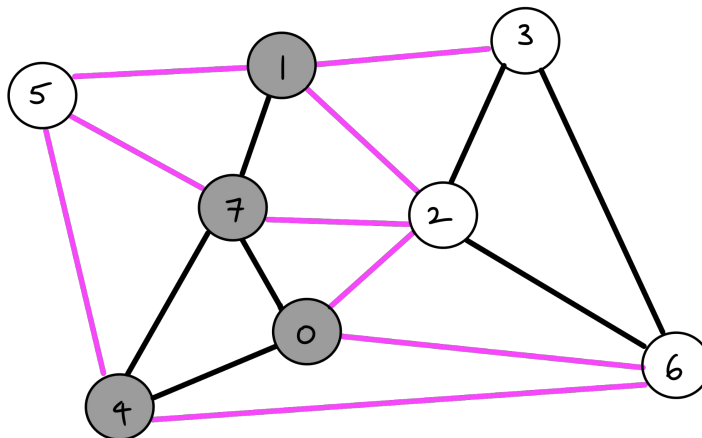


Start with no edges in the MST.

- Find a cut that has no crossing edges in the MST.
- Add smallest crossing edge to the MST.
- Repeat until $V-1$ edges.

This should work, but we need some way of finding a cut with no crossing edges!

- Random isn't a very good idea.



Basic Prim's (Demo)

Lecture 24, CS61B, Spring 2026

Minimum Spanning Trees

- Intro
- The Cut Property

Prim's Algorithm

- **Basic Prim's (Demo)**
- Optimized Prim's (Demo)
- Prim's Algorithm Code and Runtime

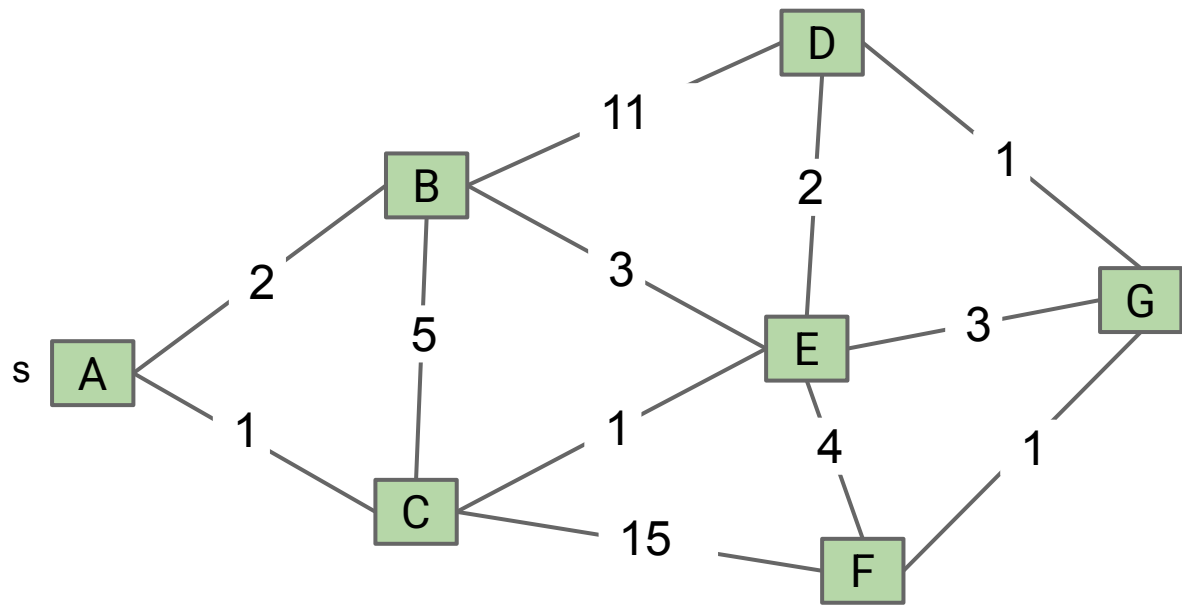
Kruskal's Algorithm:

- Basic Kruskal's (Demo)
- Optimized Kruskal's (Demo)
- Kruskal's vs. Prim's
- Kruskal's Algorithm Code and Runtime

Start from some arbitrary start node.

- Add shortest edge that has one node inside the MST under construction. Repeat until $V-1$ edges.

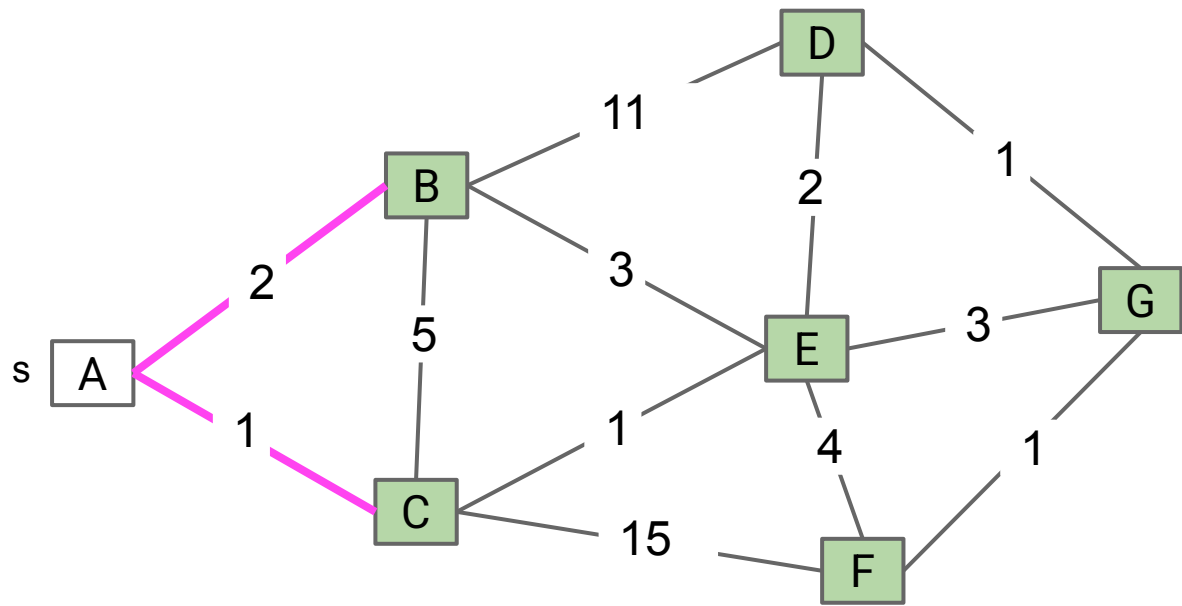
Node	edgeTo
A	-
B	-
C	-
D	-
E	-
F	-
G	-



Start from some arbitrary start node.

- Add shortest edge that has one node inside the MST under construction. Repeat until $V-1$ edges.

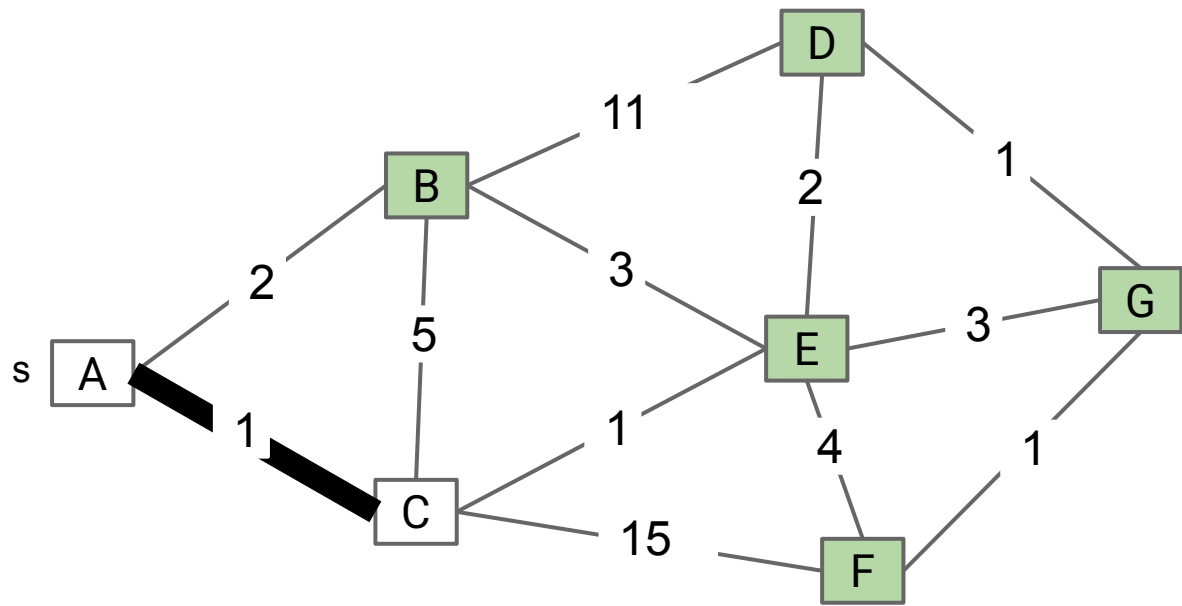
Node	edgeTo
A	-
B	-
C	-
D	-
E	-
F	-
G	-



Start from some arbitrary start node.

- Add shortest edge that has one node inside the MST under construction. Repeat until $V-1$ edges.

Node	edgeTo
A	-
B	-
C	A
D	-
E	-
F	-
G	-

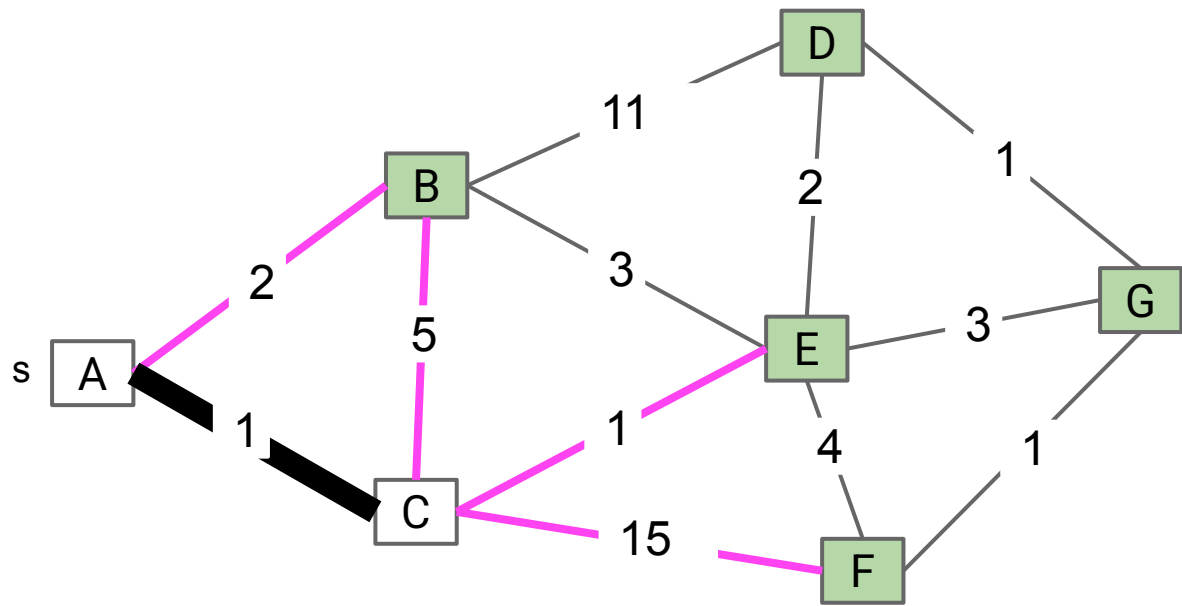


Prim's Demo (Conceptual)

Start from some arbitrary start node.

- Add shortest edge that has one node inside the MST under construction. Repeat until $V-1$ edges.

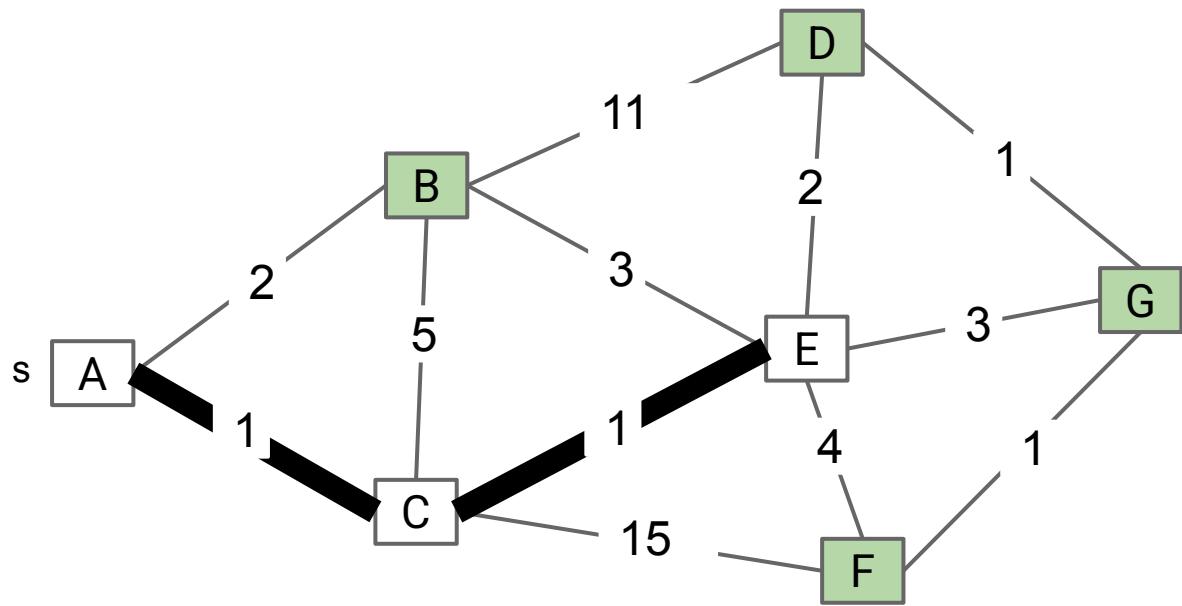
Node	edgeTo
A	-
B	-
C	A
D	-
E	-
F	-
G	-



Start from some arbitrary start node.

- Add shortest edge that has one node inside the MST under construction. Repeat until $V-1$ edges.

Node	edgeTo
A	-
B	-
C	A
D	-
E	C
F	-
G	-

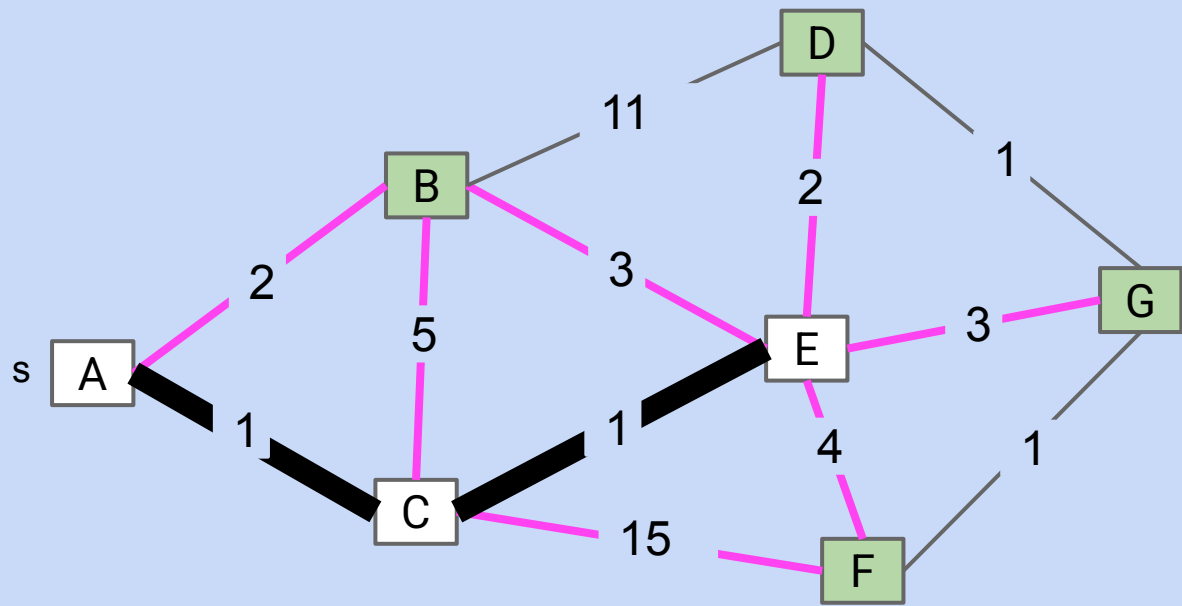


Prim's Demo (Conceptual)

Start from some arbitrary start node.

- Add shortest edge that has one node inside the MST under construction. Repeat until $V-1$ edges.

Node	edgeTo
A	-
B	-
C	A
D	-
E	C
F	-
G	-



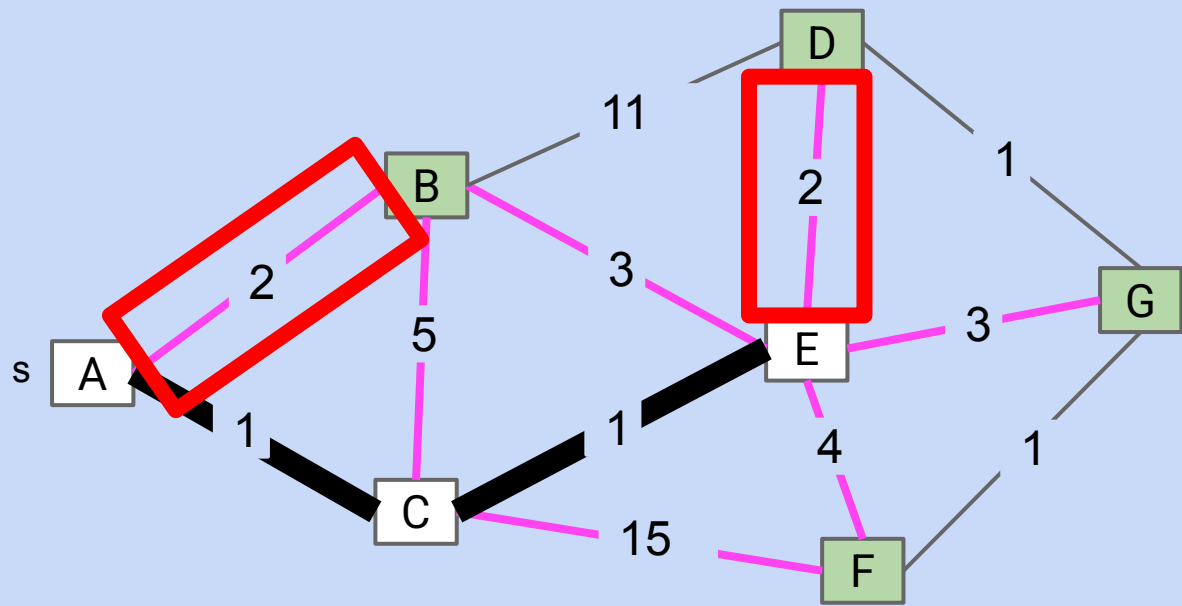
Which edge is added next?

Prim's Demo (Conceptual)

Start from some arbitrary start node.

- Add shortest edge that has one node inside the MST under construction. Repeat until $V-1$ edges.

Node	edgeTo
A	-
B	-
C	A
D	-
E	C
F	-
G	-

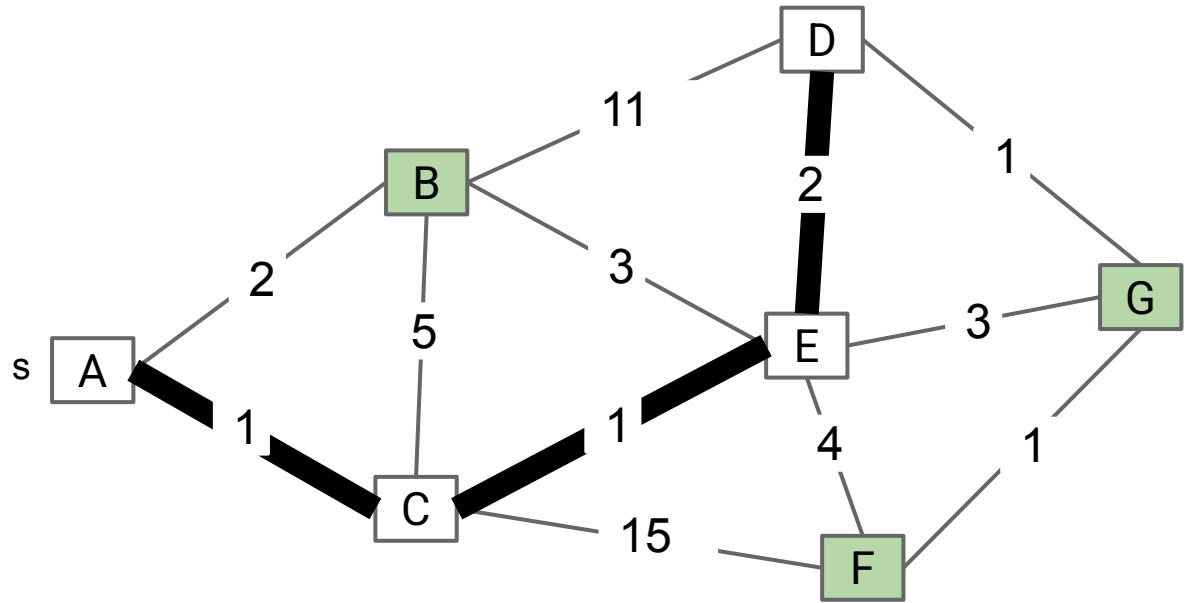


Both of **A-B** and **D-E** work

Start from some arbitrary start node.

- Add shortest edge that has one node inside the MST under construction. Repeat until $V-1$ edges.

Node	edgeTo
A	-
B	-
C	A
D	E
E	C
F	-
G	-



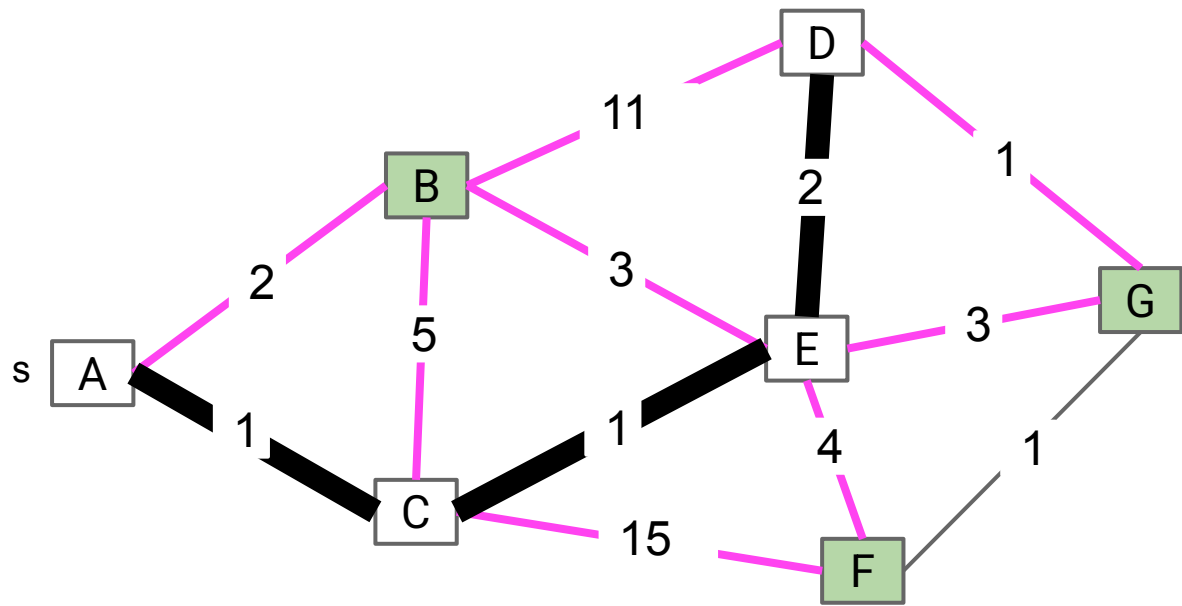
Which edge is added next?

- Either A-B or D-E are guaranteed to work (see exercises for proof)!
- Note: They are not both guaranteed to be in the MST.

Start from some arbitrary start node.

- Add shortest edge that has one node inside the MST under construction. Repeat until $V-1$ edges.

Node	edgeTo
A	-
B	-
C	A
D	E
E	C
F	-
G	-

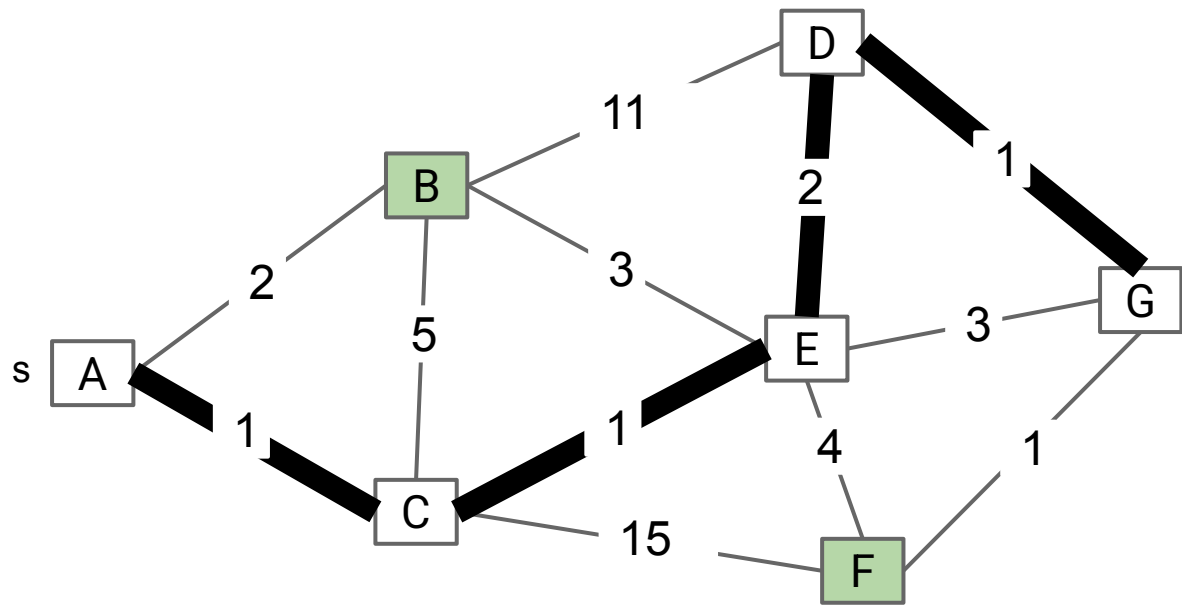


Prim's Demo (Conceptual)

Start from some arbitrary start node.

- Add shortest edge that has one node inside the MST under construction. Repeat until $V-1$ edges.

Node	edgeTo
A	-
B	-
C	A
D	E
E	C
F	-
G	D

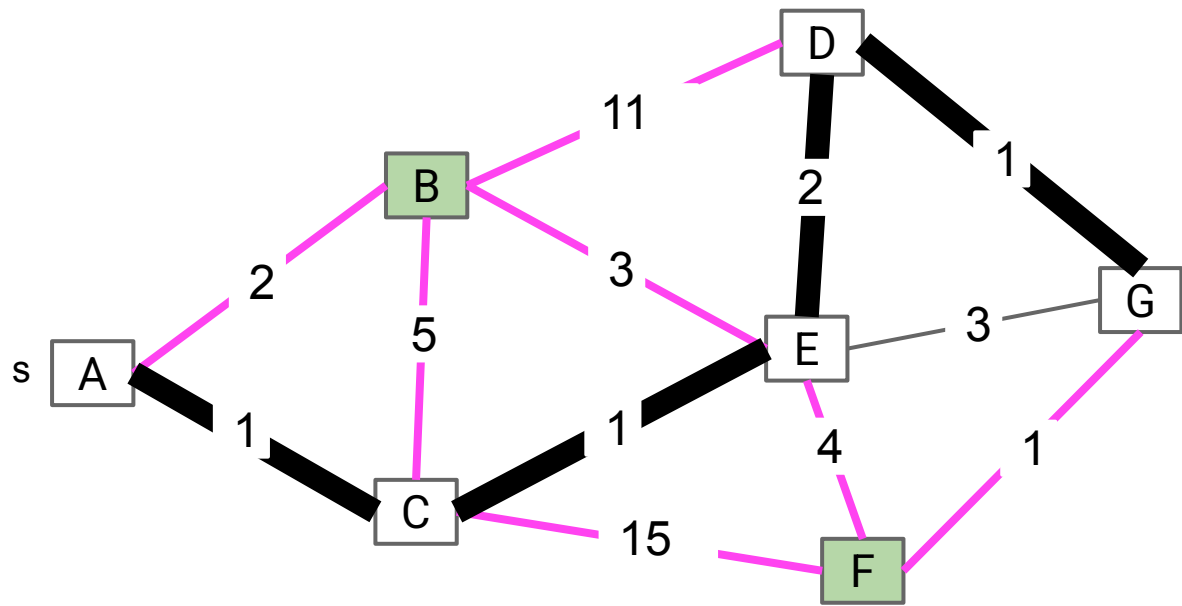


Prim's Demo (Conceptual)

Start from some arbitrary start node.

- Add shortest edge that has one node inside the MST under construction. Repeat until $V-1$ edges.

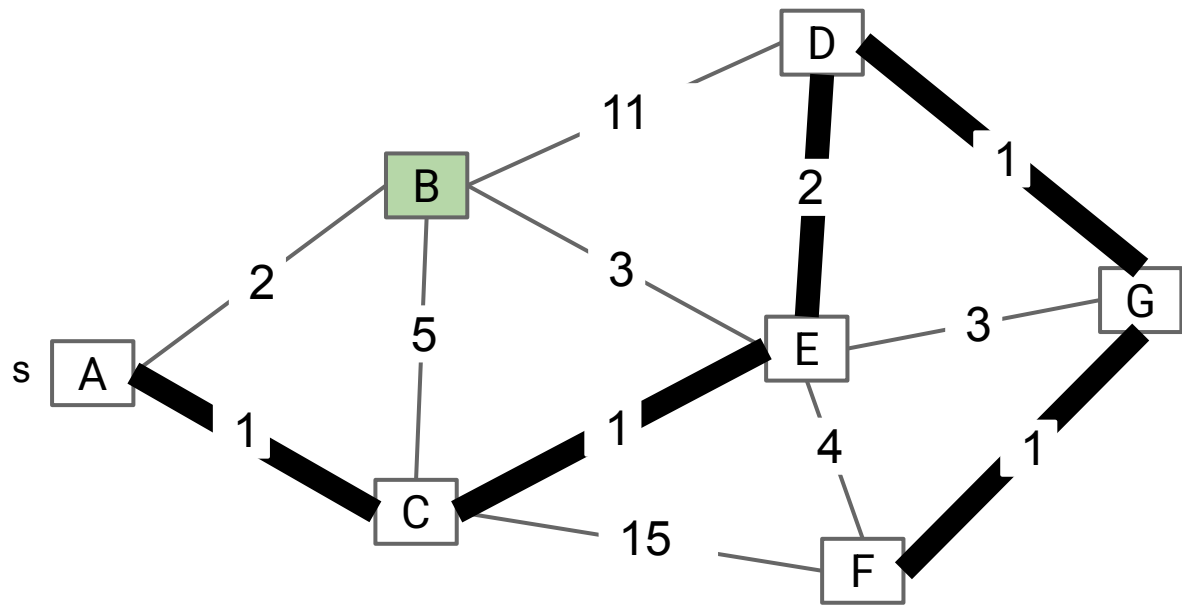
Node	edgeTo
A	-
B	-
C	A
D	E
E	C
F	-
G	D



Start from some arbitrary start node.

- Add shortest edge (mark black) that has one node inside the MST under construction. Repeat until $V-1$ edges.

Node	edgeTo
A	-
B	-
C	A
D	E
E	C
F	G
G	D

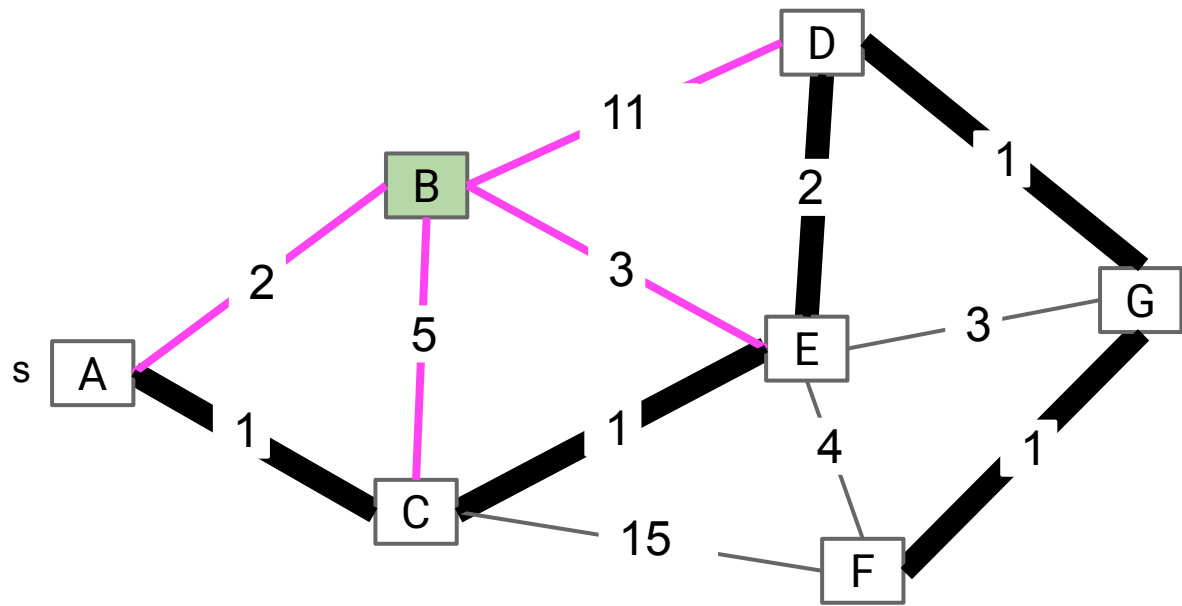


Prim's Demo (Conceptual)

Start from some arbitrary start node.

- Add shortest edge (mark black) that has one node inside the MST under construction. Repeat until $V-1$ edges.

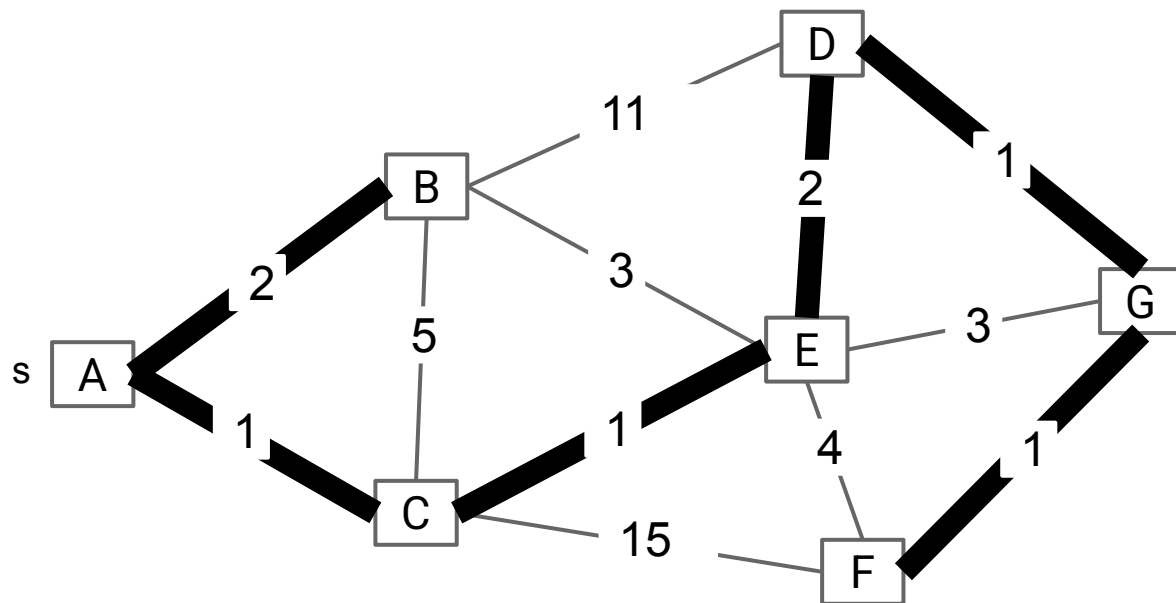
Node	edgeTo
A	-
B	-
C	A
D	E
E	C
F	G
G	D



Start from some arbitrary start node.

- Add shortest edge that has one node inside the MST under construction. Repeat until $V-1$ edges.

Node	edgeTo
A	-
B	A
C	A
D	E
E	C
F	G
G	D

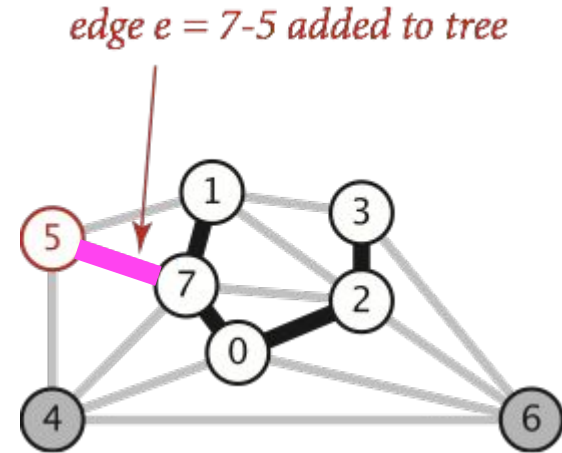


Start from some arbitrary start node.

- Repeatedly add shortest edge that has one node inside the MST under construction.
- Repeat until $V-1$ edges.

Why does Prim's work? Special case of generic algorithm.

- Suppose we add edge $e = v \rightarrow w$.
- Side 1 of cut is all vertices connected to start, side 2 is all the others.
- No crossing edge are in the MST
- No crossing edge has lower weight (considered the edges in increasing weight order).



Optimized Prim's (Demo)

Lecture 24, CS61B, Spring 2026

Minimum Spanning Trees

- Intro
- The Cut Property

Prim's Algorithm

- Basic Prim's (Demo)
- **Optimized Prim's (Demo)**
- Prim's Algorithm Code and Runtime

Kruskal's Algorithm:

- Basic Kruskal's (Demo)
- Optimized Kruskal's (Demo)
- Kruskal's vs. Prim's
- Kruskal's Algorithm Code and Runtime

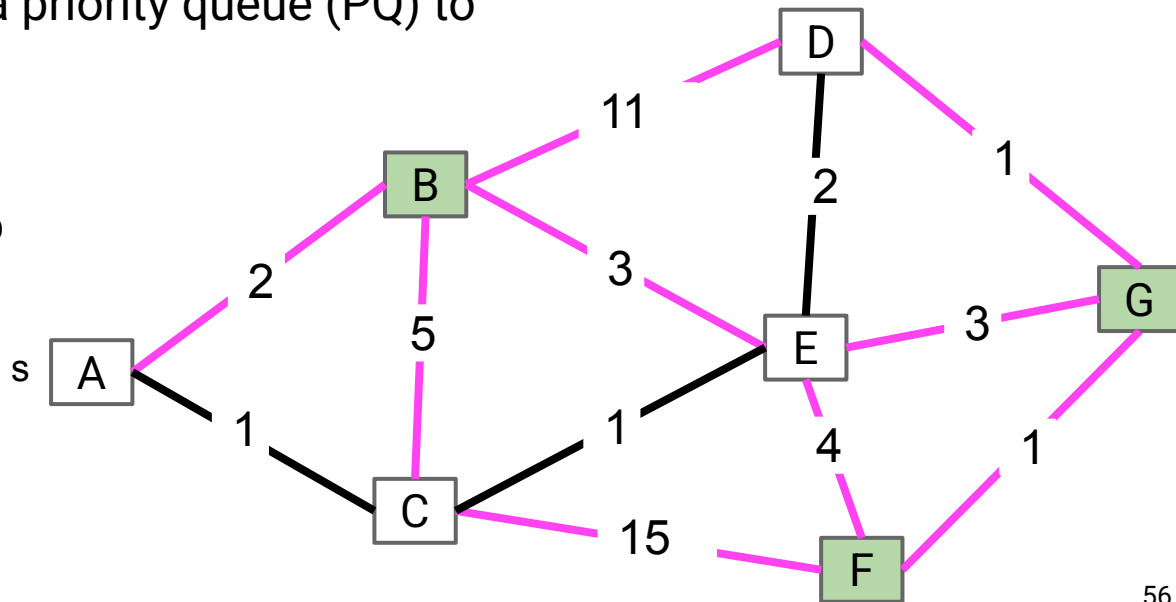
The natural implementation of the conceptual version of Prim's algorithm is highly inefficient.

- Example: Iterating over all purple edges shown is unnecessary and slow.

Can use some cleverness and a priority queue (PQ) to speed things up.

Realistic Implementation Demo

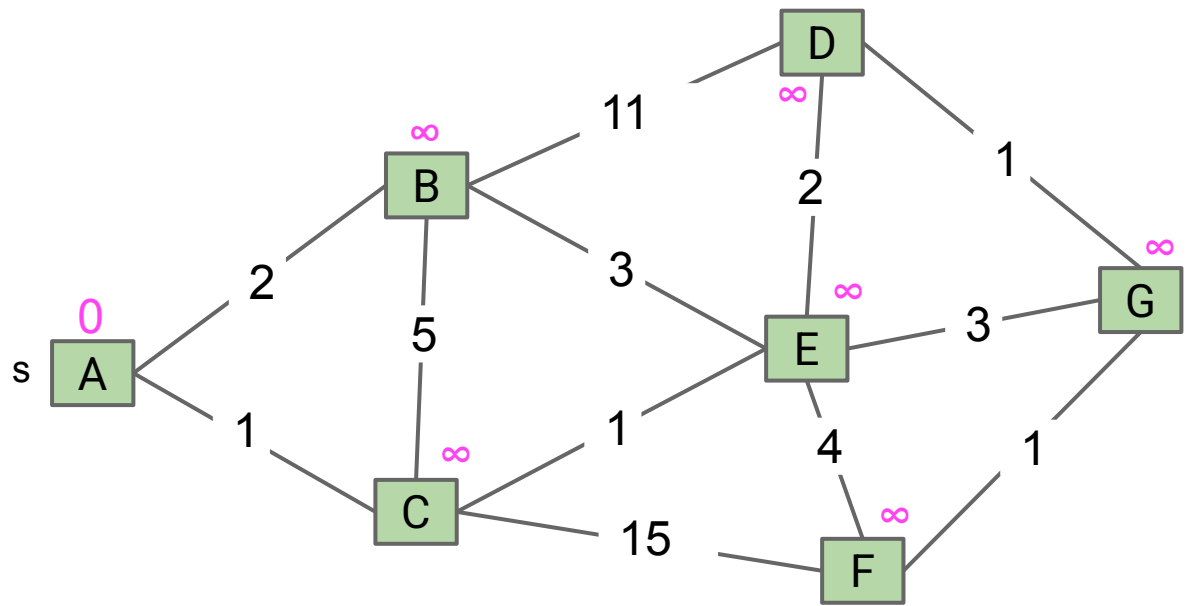
- Very similar to Dijkstra's!



Prim's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from tree.
Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v .

Node	distTo	edgeTo
A	0	-
B	∞	-
C	∞	-
D	∞	-
E	∞	-
F	∞	-
G	∞	-

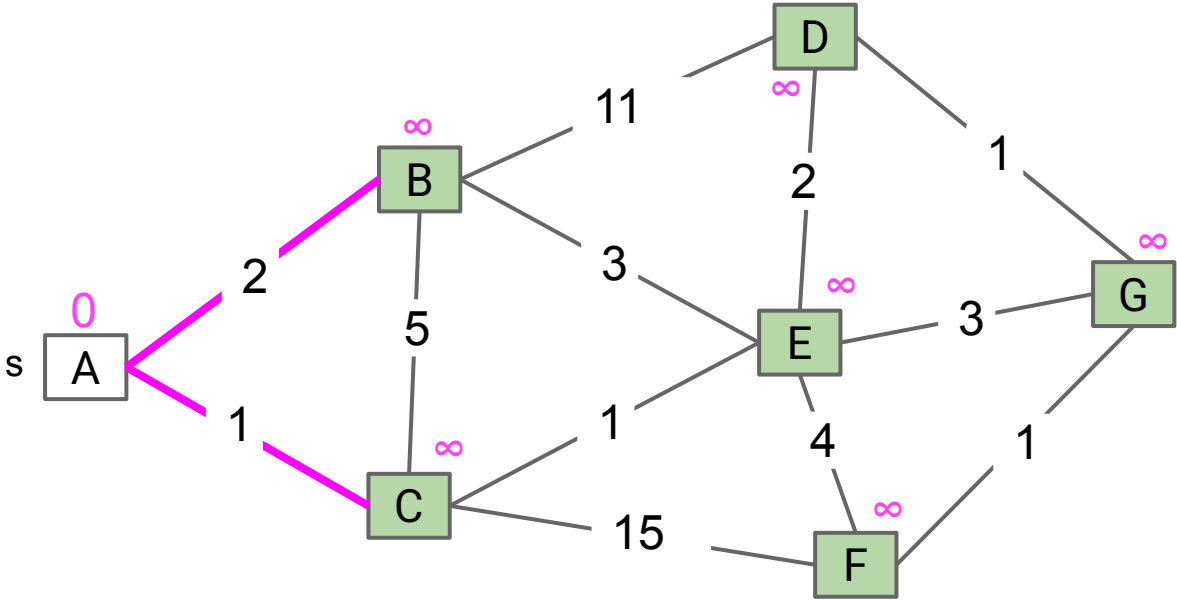


Fringe: [(A: 0), (B: ∞), (C: ∞), (D: ∞), (E: ∞), (F: ∞), (G: ∞)]
***priority queue ordered by weight

Prim's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from tree.
Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v .

Node	distTo	edgeTo
A	0	-
B	∞	-
C	∞	-
D	∞	-
E	∞	-
F	∞	-
G	∞	-

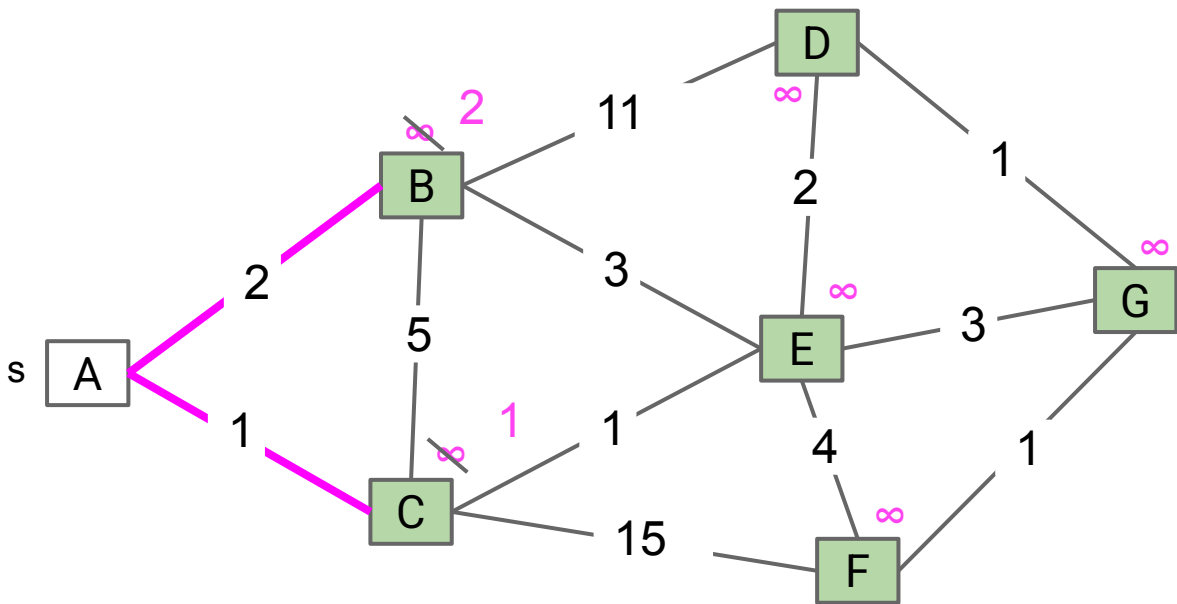


Fringe: [(B: ∞), (C: ∞), (D: ∞), (E: ∞), (F: ∞), (G: ∞)]
***priority queue ordered by weight

Prim's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from tree.
Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v .

Node	distTo	edgeTo
A	-	-
B	2	A
C	1	A
D	∞	-
E	∞	-
F	∞	-
G	∞	-



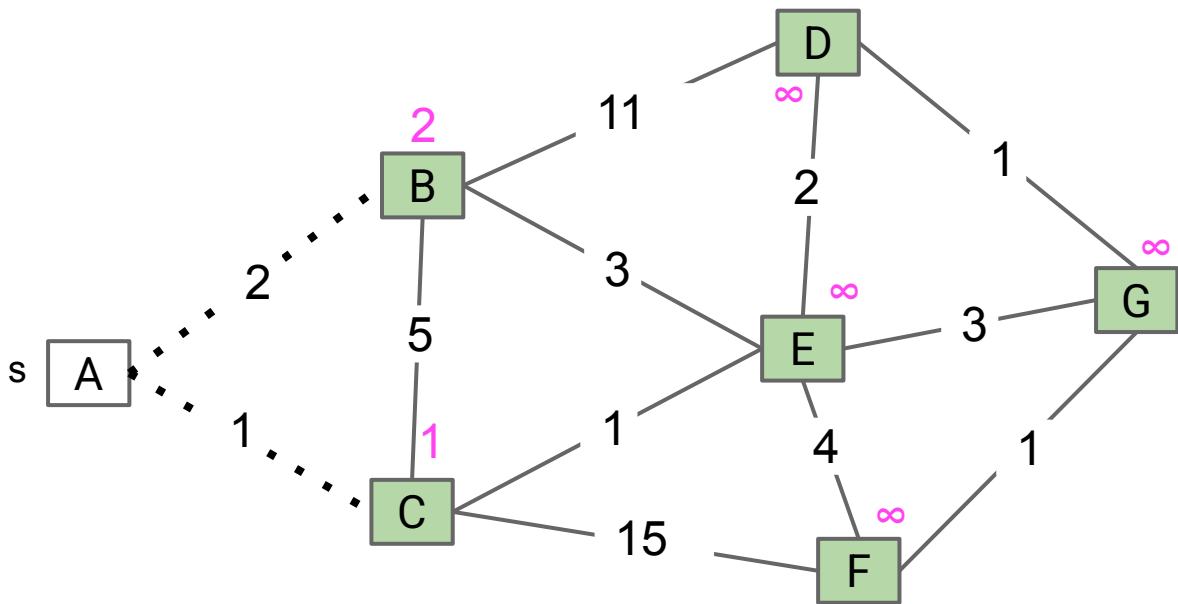
Fringe: [(C: 1), (B: 2), (D: ∞), (E: ∞), (F: ∞), (G: ∞)]

***priority queue ordered by weight

Prim's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from tree.
Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v .

Node	distTo	edgeTo
A	0	-
B	2	A
C	1	A
D	∞	-
E	∞	-
F	∞	-
G	∞	-



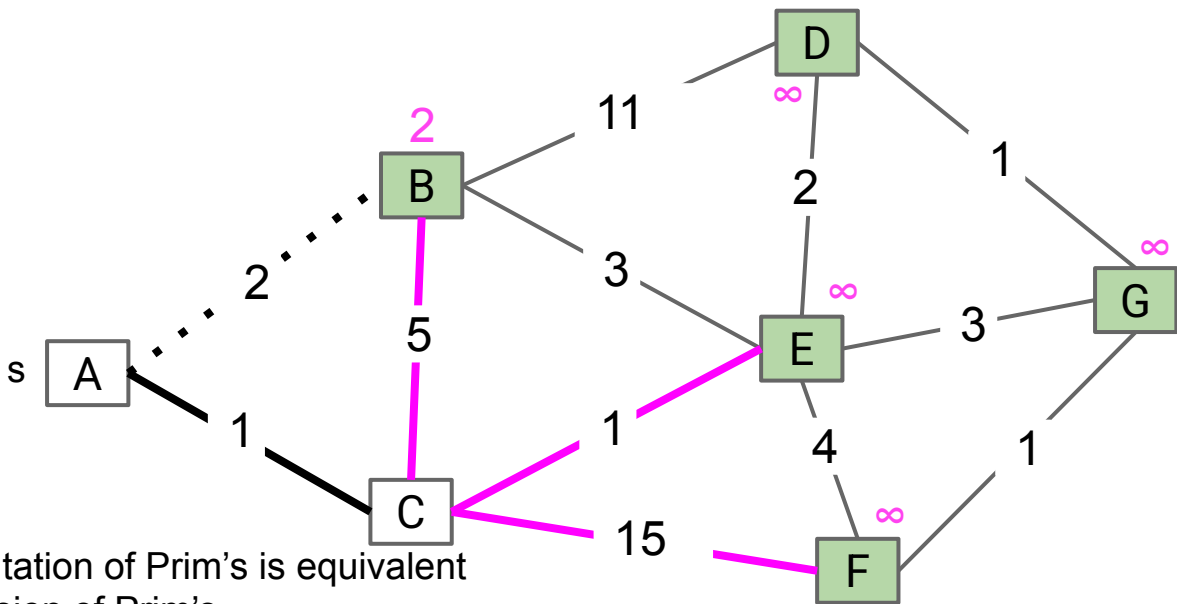
Fringe: [(C: 1), (B: 2), (D: ∞), (E: ∞), (F: ∞), (G: ∞)]

***priority queue ordered by weight

Prim's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from tree.
Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v .

Node	distTo	edgeTo
A		-
B	2	A
C		A
D	∞	-
E	∞	-
F	∞	-
G	∞	-



Note: Vertex removal in this implementation of Prim's is equivalent to edge addition in the conceptual version of Prim's.

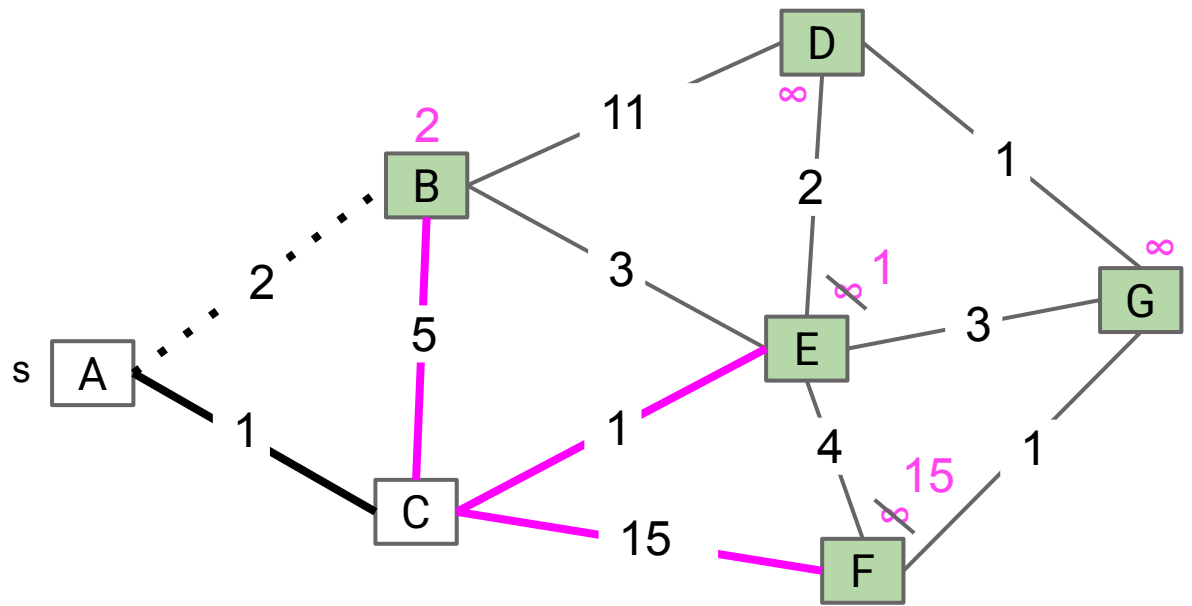
Fringe: [(B: 2), (D: ∞), (E: ∞), (F: ∞), (G: ∞)]

***priority queue ordered by weight

Prim's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from tree.
Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo
A		-
B	2	A
C		A
D	∞	-
E	1	C
F	15	C
G	∞	-



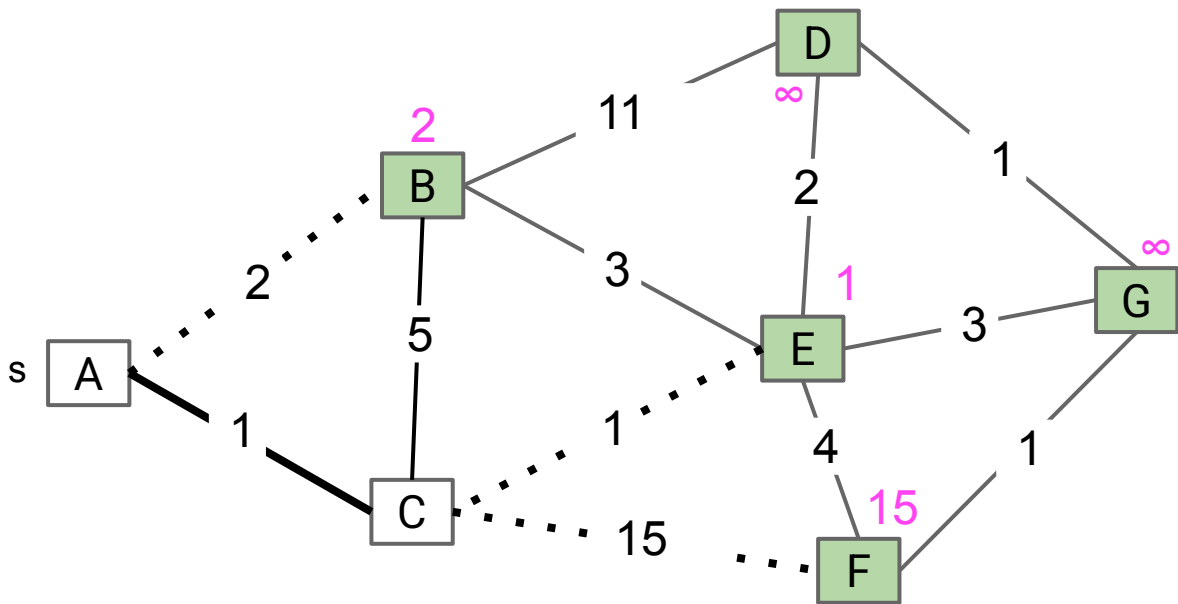
Fringe: [(E: 1), (B: 2), (F: 15), (D: ∞), (G: ∞)]

***priority queue ordered by weight

Prim's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from tree.
Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v .

Node	distTo	edgeTo
A		-
B	2	A
C		A
D	∞	-
E	1	C
F	15	C
G	∞	-



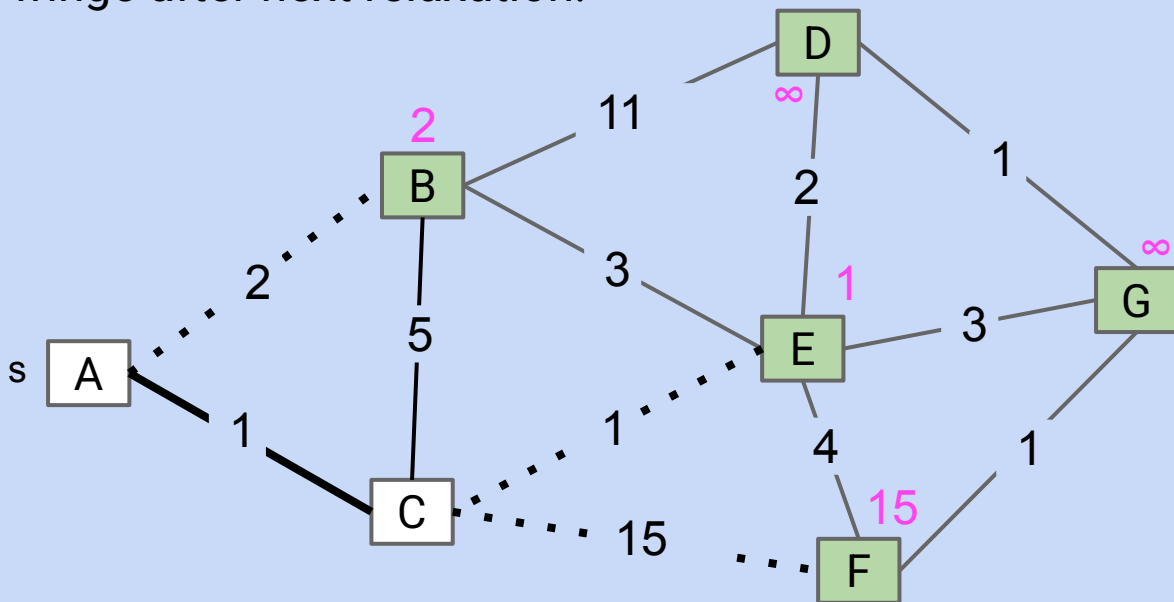
Fringe: [(E: 1), (B: 2), (F: 15), (D: ∞), (G: ∞)]

***priority queue ordered by weight

- Insert all vertices into fringe PQ, storing vertices in order of distance from tree.
- Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v .
- Show distTo , edgeTo , and fringe after next relaxation.

Node	distTo	edgeTo
A		-
B	2	A
C		A
D	∞	-
E	1	C
F	15	C
G	∞	-

***Reminder: green nodes = in fringe



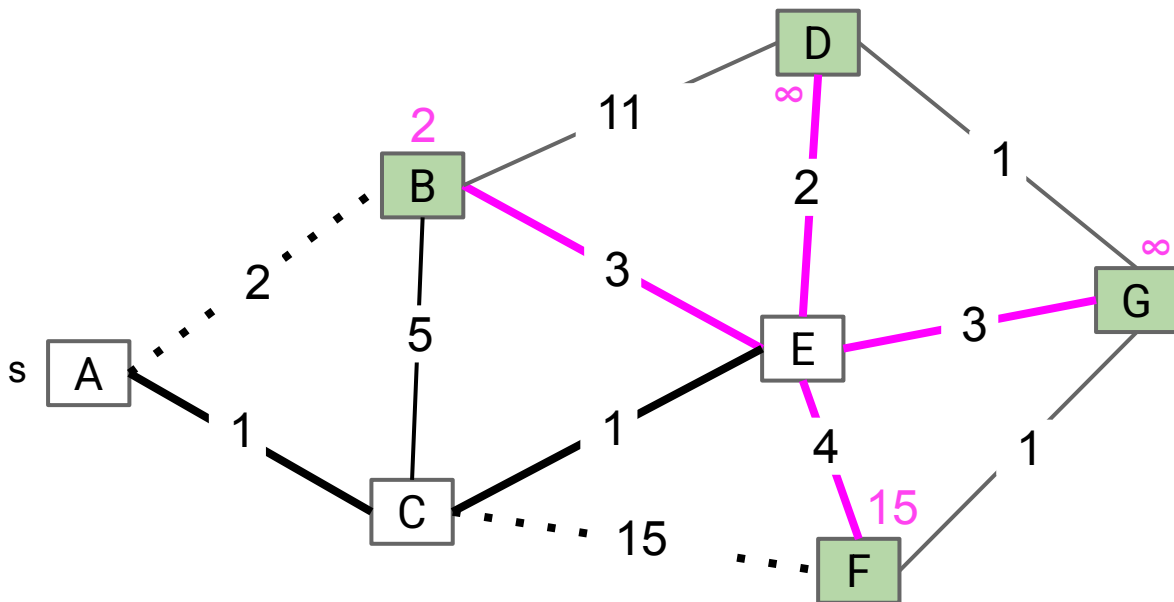
Fringe: [(E: 1), (B: 2), (F: 15), (D: ∞), (G: ∞)]

***priority queue ordered by weight

Prim's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from tree.
Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v .

Node	distTo	edgeTo
A		-
B	2	A
C		A
D	∞	-
E		C
F	15	C
G	∞	-



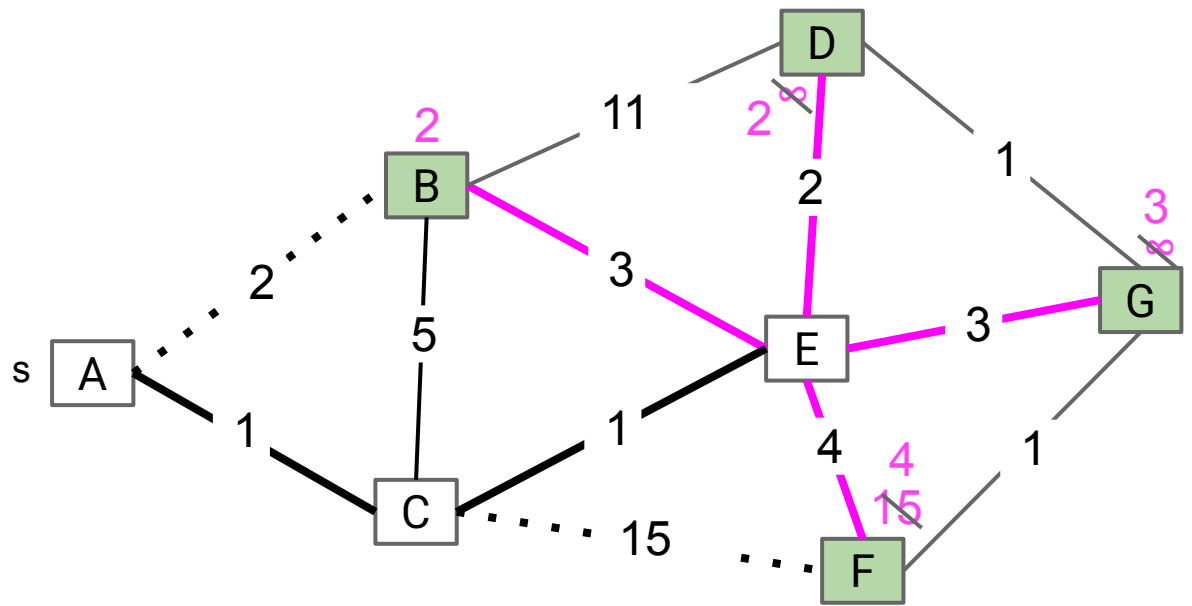
Fringe: [(B: 2), (F: 15), (D: ∞), (G: ∞)]

***priority queue ordered by weight

Prim's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from tree.
Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo
A		-
B	2	A
C		A
D	2	E
E		C
F	4	E
G	3	E



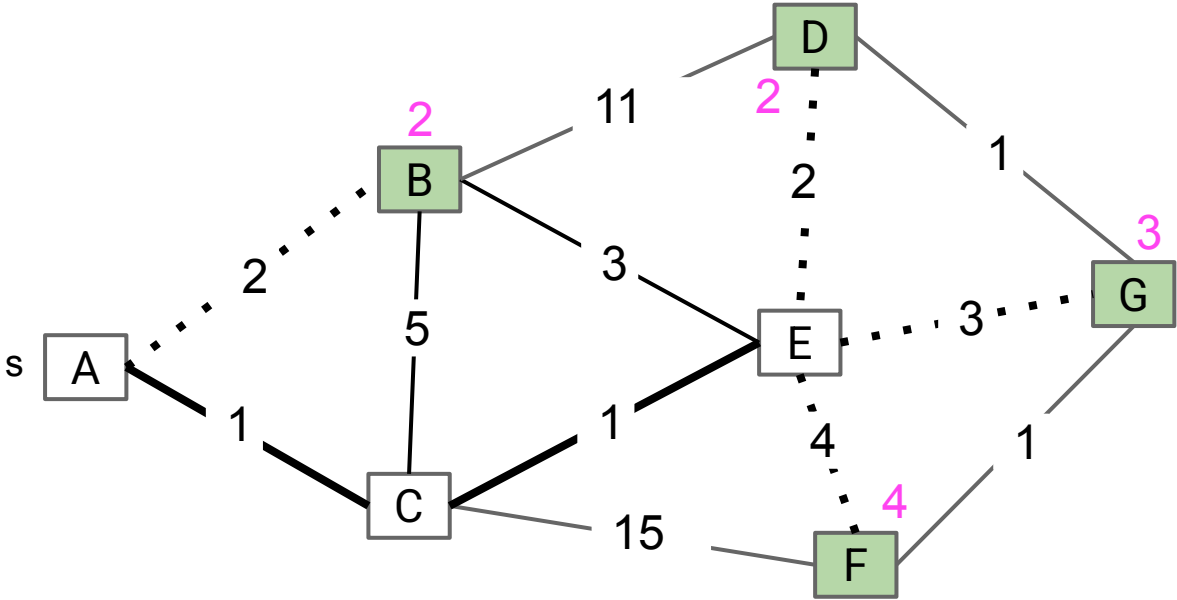
Fringe: [(B: 2), (D: 2), (G: 3), (F: 4)]

***priority queue ordered by weight

Prim's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from tree.
Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo
A		-
B	2	A
C		A
D	2	E
E		C
F	4	E
G	3	E

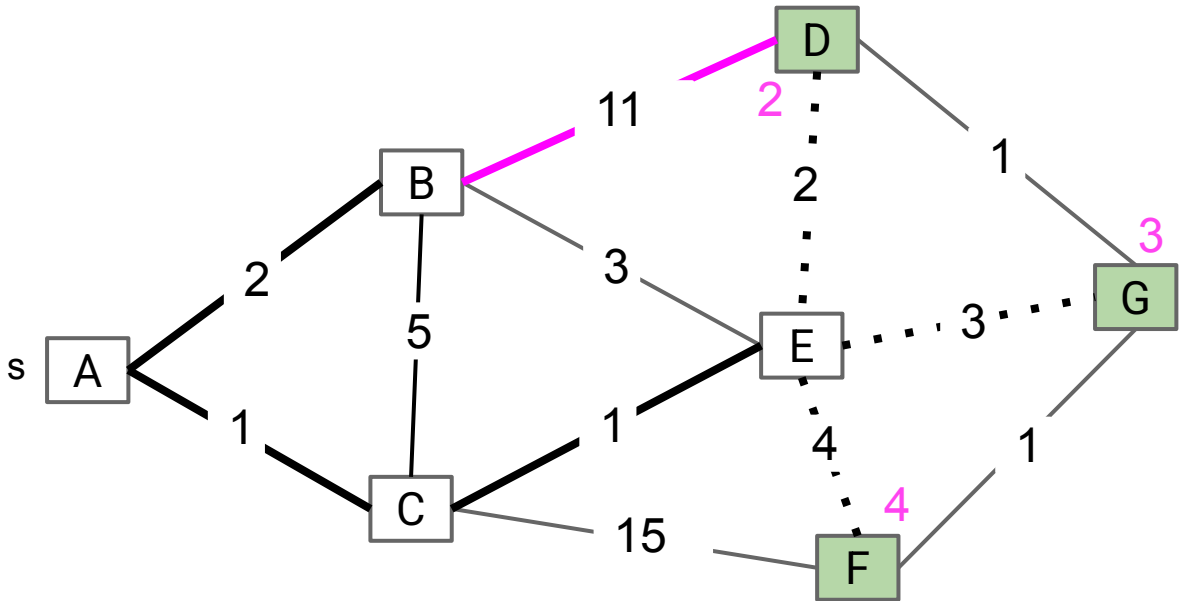


Fringe: [(B: 2), (D: 2), (G: 3), (F: 4)]
***priority queue ordered by weight

Prim's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from tree.
Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo
A		-
B		A
C		A
D	2	E
E		C
F	4	E
G	3	E



Fringe: [(D: 2), (G: 3), (F: 4)]

***priority queue ordered by weight

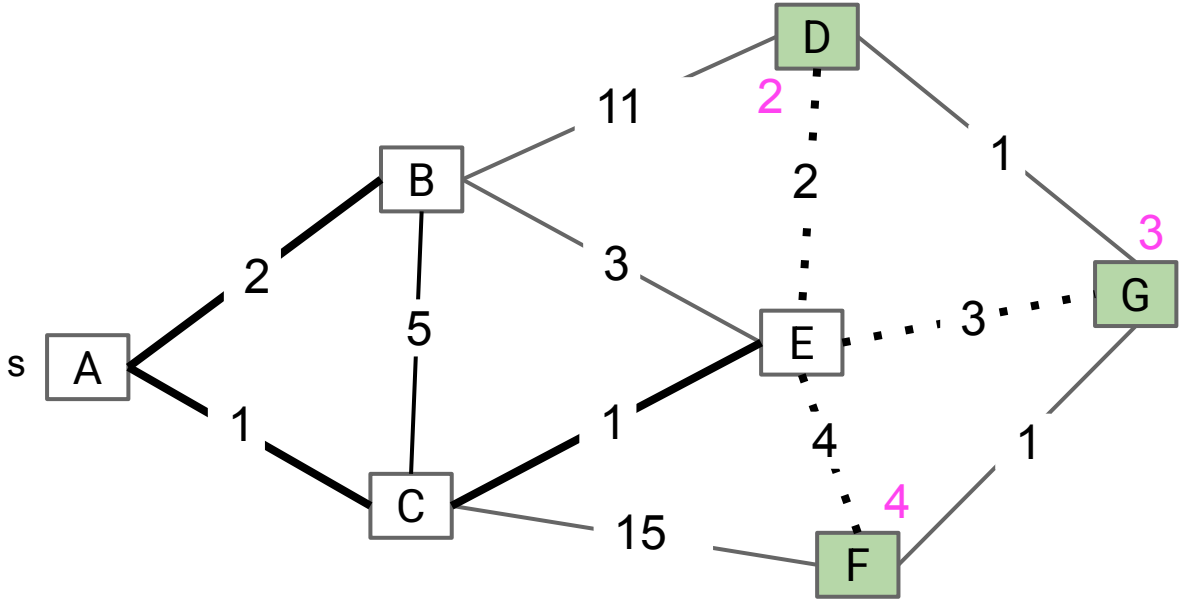
No need to consider edges with weight 5 and 3 since other side is already marked!



Prim's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from tree.
Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo
A		-
B		A
C		A
D	2	E
E		C
F	4	E
G	3	E

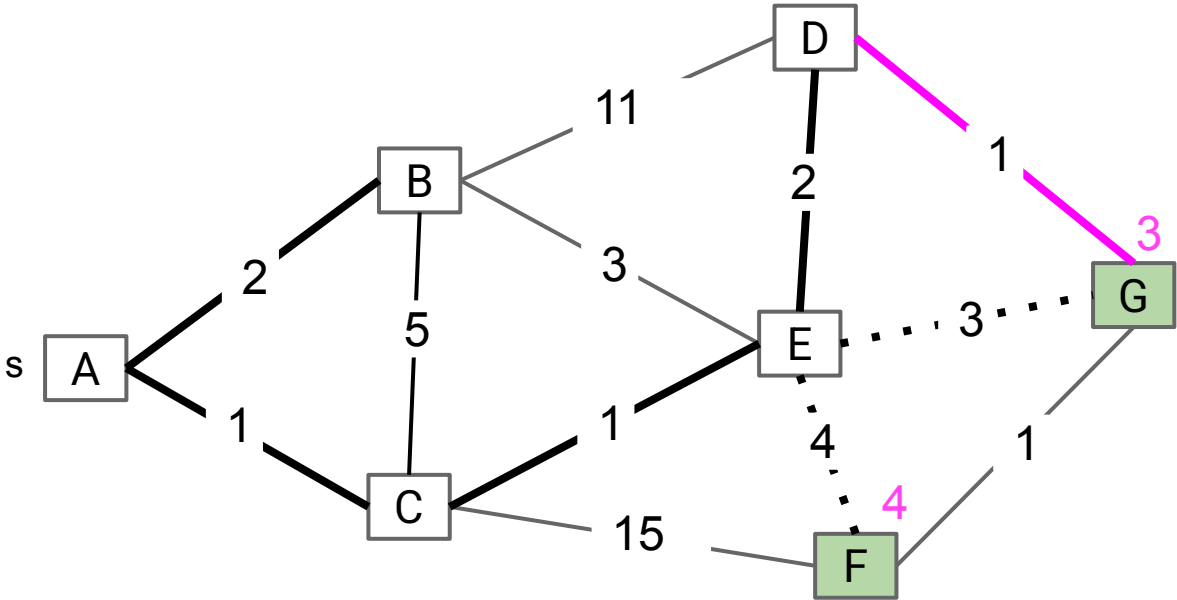


Fringe: [(D: 2), (G: 3), (F: 4)]
***priority queue ordered by weight

Prim's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from tree.
Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo
A		-
B		A
C		A
D		E
E		C
F	4	E
G	3	E

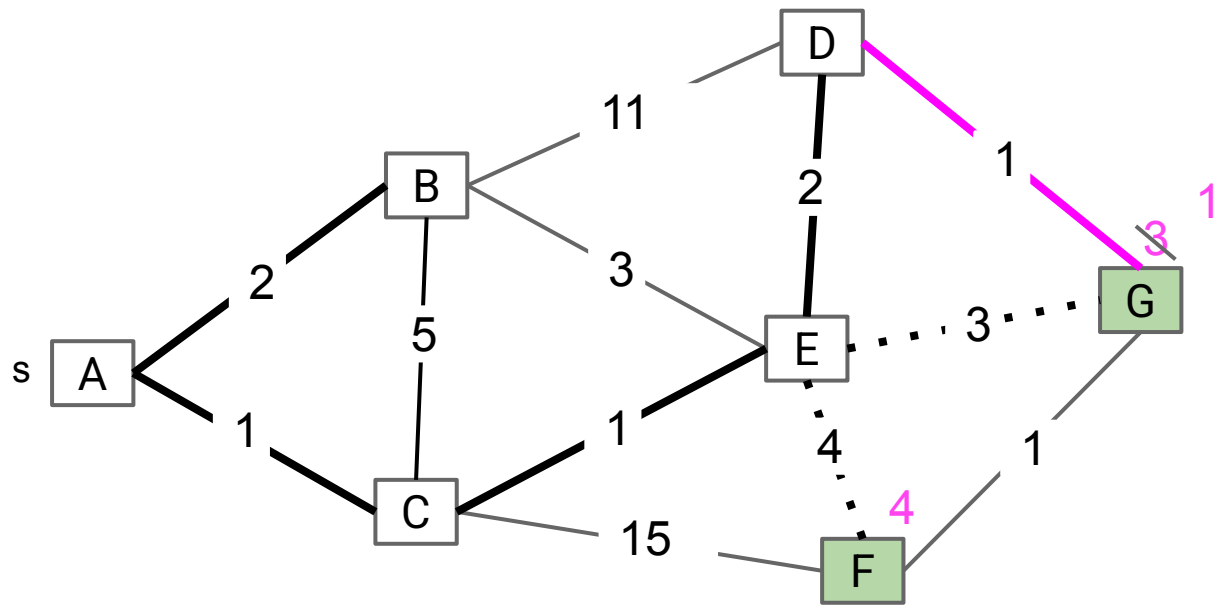


Fringe: [(G: 3), (F: 4)]
***priority queue ordered by weight

Prim's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from tree.
Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo
A		-
B		A
C		A
D		E
E		C
F	4	E
G	1	D



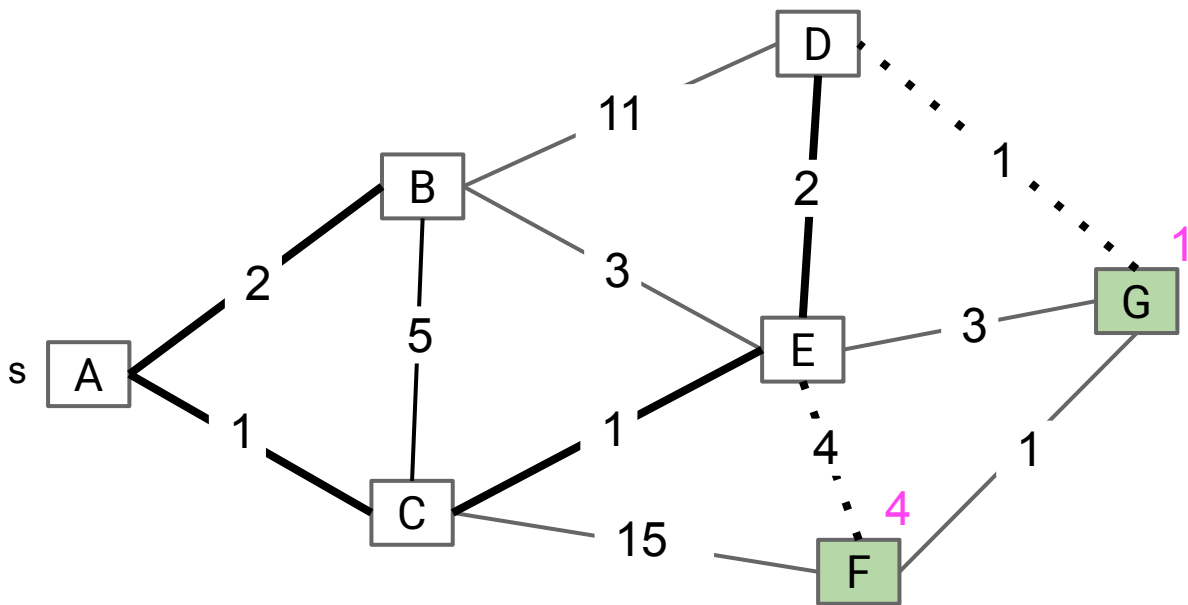
Fringe: [(G: 1), (F: 4)]
***priority queue ordered by weight



Prim's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from tree.
Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo
A		-
B		A
C		A
D		E
E		C
F	4	E
G	1	D

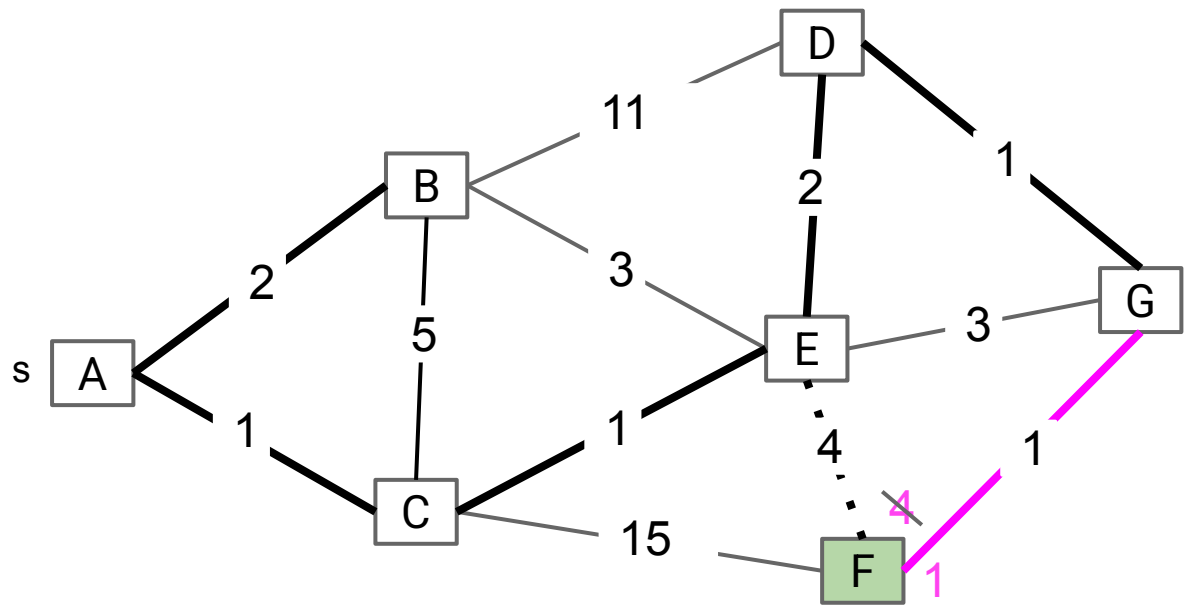


Fringe: [(G: 1), (F: 4)]
***priority queue ordered by weight

Prim's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from tree.
Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo
A		-
B		A
C		A
D		E
E		C
F	1	G
G		D

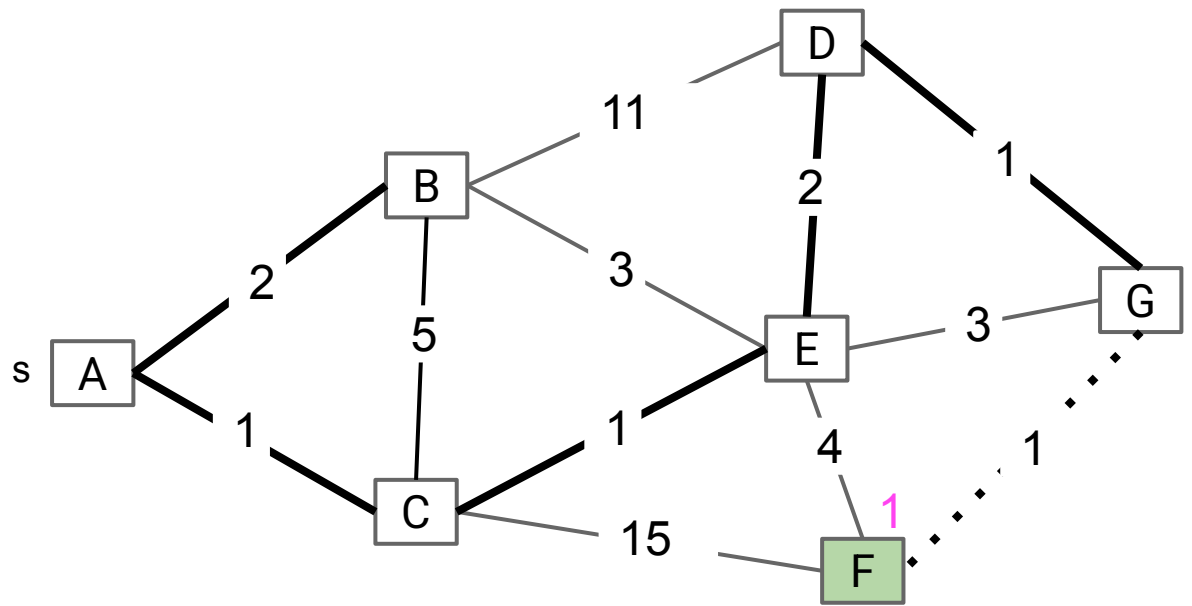


Fringe: [(F: 1)]
***priority queue ordered by weight

Prim's Demo

Insert all vertices into fringe PQ, storing vertices in order of distance from tree.
Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo
A		-
B		A
C		A
D		E
E		C
F	1	G
G		D

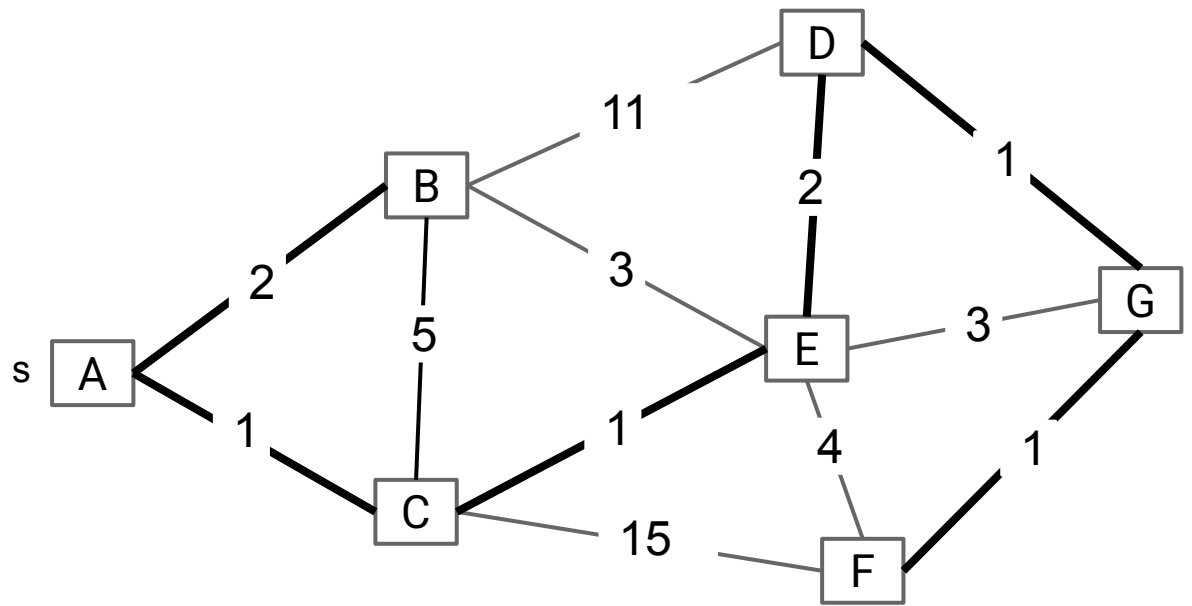


Fringe: [(F: 1)]
***priority queue ordered by weight



Insert all vertices into fringe PQ, storing vertices in order of distance from tree.
Repeat: Remove (closest) vertex v from PQ, and relax all edges pointing from v.

Node	distTo	edgeTo
A		-
B		A
C		A
D		E
E		C
F		G
G		D



Fringe: []



Prim's Algorithm Code and Runtime

Lecture 24, CS61B, Spring 2026

Minimum Spanning Trees

- Intro
- The Cut Property

Prim's Algorithm

- Basic Prim's (Demo)
- Optimized Prim's (Demo)
- **Prim's Algorithm Code and Runtime**

Kruskal's Algorithm:

- Basic Kruskal's (Demo)
- Optimized Kruskal's (Demo)
- Kruskal's vs. Prim's
- Kruskal's Algorithm Code and Runtime

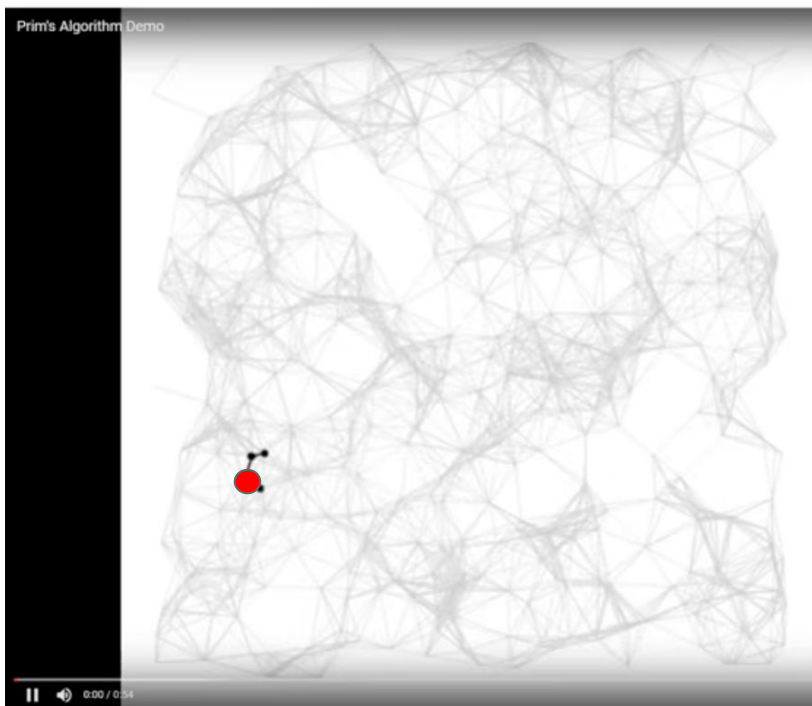
Prim's and Dijkstra's algorithms are exactly the same, except Dijkstra's considers "distance from a single source node", and Prim's considers "distance from a tree."

Visit order:

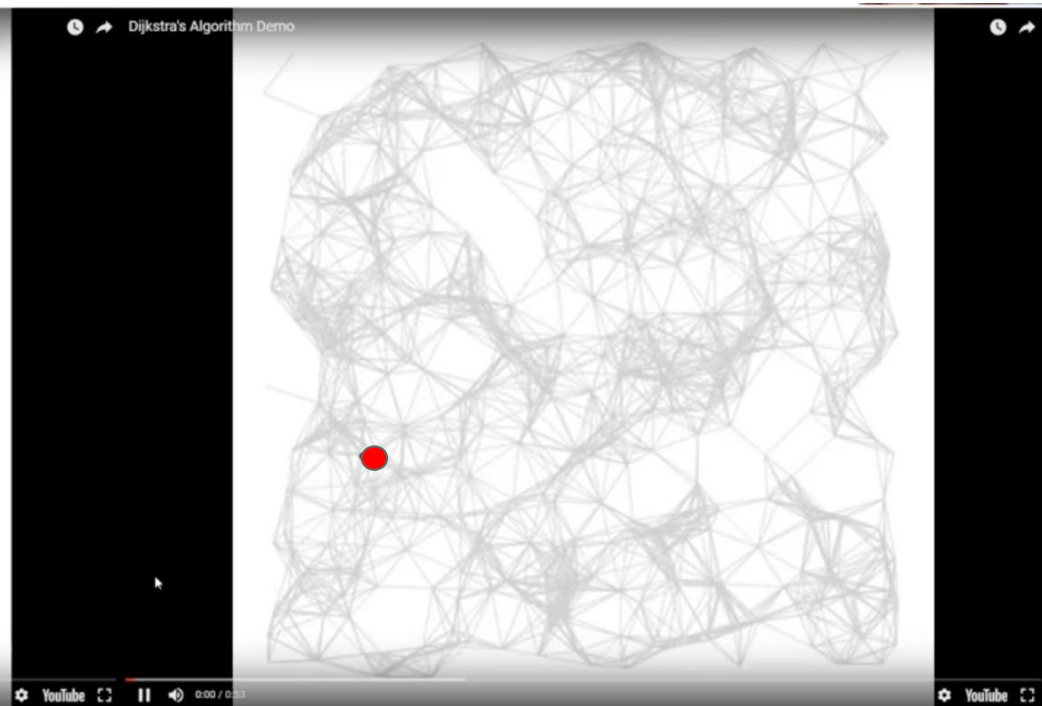
- Dijkstra's algorithm visits vertices in order of distance from the source.
- Prim's algorithm visits vertices in order of distance from the MST under construction.

Relaxation:

- Relaxation in Dijkstra's considers an edge better based on its distance to source.
- Relaxation in Prim's considers an edge better based on its distance to tree.



Prim's Algorithm



Dijkstra's Algorithm

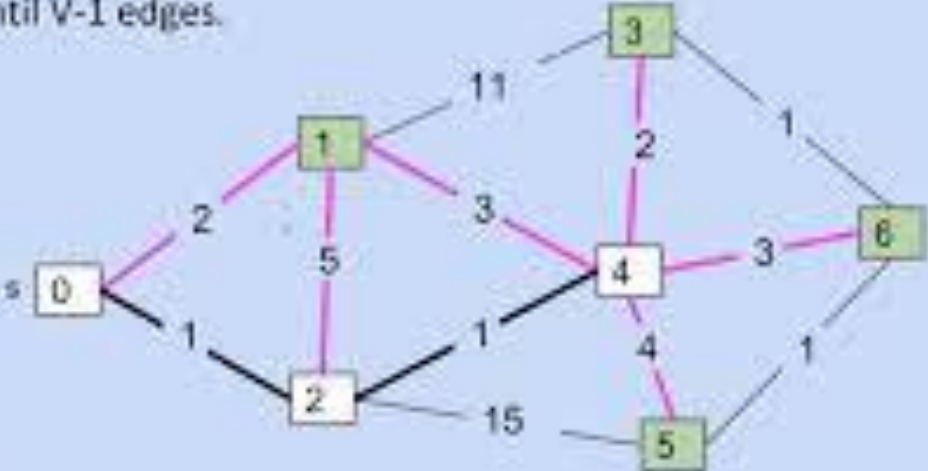
Prim's Demo (Conceptual)



Start from some arbitrary start node.

- Add shortest edge (mark black) that has one node inside the MST under construction. Repeat until $V-1$ edges.

#	edgeTo
0	-
1	-
2	0
3	-
4	2
5	-
6	-



Which edge is added next?

Prim's Implementation (Pseudocode, 1/2)

```
public class PrimMST {  
    public PrimMST(EdgeWeightedGraph G) {  
        edgeTo = new Edge[G.V()];  
        distTo = new double[G.V()];  
        marked = new boolean[G.V()];  
        fringe = new SpecialPQ<Double>(G.V());  
  
        distTo[s] = 0.0;  
        setDistancesToInfinityExceptS(s);  
        insertAllVertices(fringe);  
  
        /* Get vertices in order of distance from tree. */  
        while (!fringe.isEmpty()) {  
            int v = fringe.delMin();  
            scan(G, v);  
        }  
    }  
    ...  
}
```

Fringe is ordered by distTo tree. Must be a specialPQ like Dijkstra's.

Get vertex closest to tree that is unvisited.

Scan means to consider all of a vertex's outgoing edges.

Prim's Implementation (Pseudocode, 2/2)

```
while (!fringe.isEmpty()) {  
    int v = fringe.delMin();  
    scan(G, v);  
}
```

Important invariant, fringe must be ordered by current best known distance from tree.

```
private void scan(EdgeWeightedGraph G, int v) {  
    marked[v] = true;  
    for (Edge e : G.adj(v)) {  
        int w = e.other(v);  
        if (marked[w]) { continue; }  
        if (e.weight() < distTo[w]) {  
            distTo[w] = e.weight();  
            edgeTo[w] = e;  
            pq.decreasePriority(w, distTo[w]);  
        }  
    }  
}
```

Vertex is closest, so add to MST.

Already in MST, so go to next edge.

Better path to a particular vertex found, so update current best known for that vertex.

```
while (!fringe.isEmpty()) {  
    int v = fringe.delMin();  
    scan(G, v);  
}
```

```
private void scan(EdgeWeightedGraph G, int v) {  
    marked[v] = true;  
    for (Edge e : G.adj(v)) {  
        int w = e.other(v);  
        if (marked[w]) { continue; }  
        if (e.weight() < distTo[w]) {  
            distTo[w] = e.weight();  
            edgeTo[w] = e;  
            pq.decreasePriority(w, distTo[w]);  
        }  
    }  
}
```

What is the runtime of Prim's algorithm?

- Assume all PQ operations take $O(\log(V))$ time.
- Give your answer in Big O notation.

Priority Queue operation count, assuming binary heap based PQ:

- Insertion: V , each costing $O(\log V)$ time.
- Delete-min: V , each costing $O(\log V)$ time.
- Decrease priority: $O(E)$, each costing $O(\log V)$ time.

Overall runtime: $O(V \cdot \log(V) + V \cdot \log(V) + E \cdot \log(V))$.

- Assuming $E > V$, this is just $O(E \log V)$ (Same as Dijkstra's).

	# Operations	Cost per operation	Total cost
PQ add	V	$O(\log V)$	$O(V \log V)$
PQ delMin	V	$O(\log V)$	$O(V \log V)$
PQ decreasePriority	$O(E)$	$O(\log V)$	$O(E \log V)$

Basic Kruskal's (Demo)

Lecture 24, CS61B, Spring 2026

Minimum Spanning Trees

- Intro
- The Cut Property

Prim's Algorithm

- Basic Prim's (Demo)
- Optimized Prim's (Demo)
- Prim's Algorithm Code and Runtime

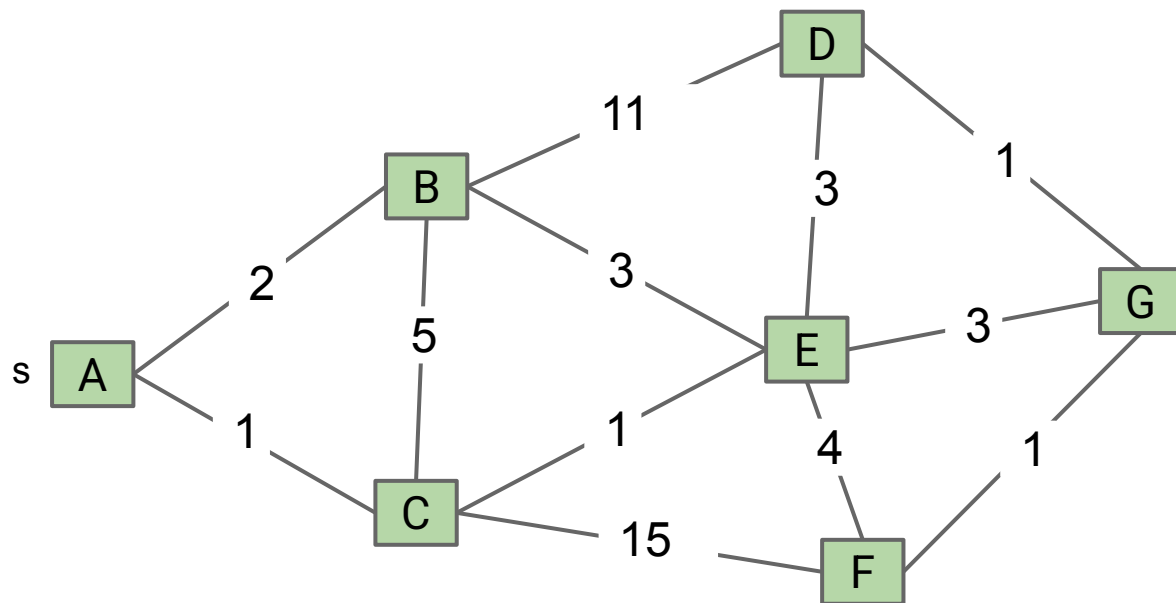
Kruskal's Algorithm:

- **Basic Kruskal's (Demo)**
- Optimized Kruskal's (Demo)
- Kruskal's vs. Prim's
- Kruskal's Algorithm Code and Runtime

Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created. Repeat until $V-1$ edges.

A-C	1
C-E	1
D-G	1
F-G	1
A-B	2
E-B	3
D-E	3
G-E	3
E-F	4
B-C	5
B-D	11
C-F	15



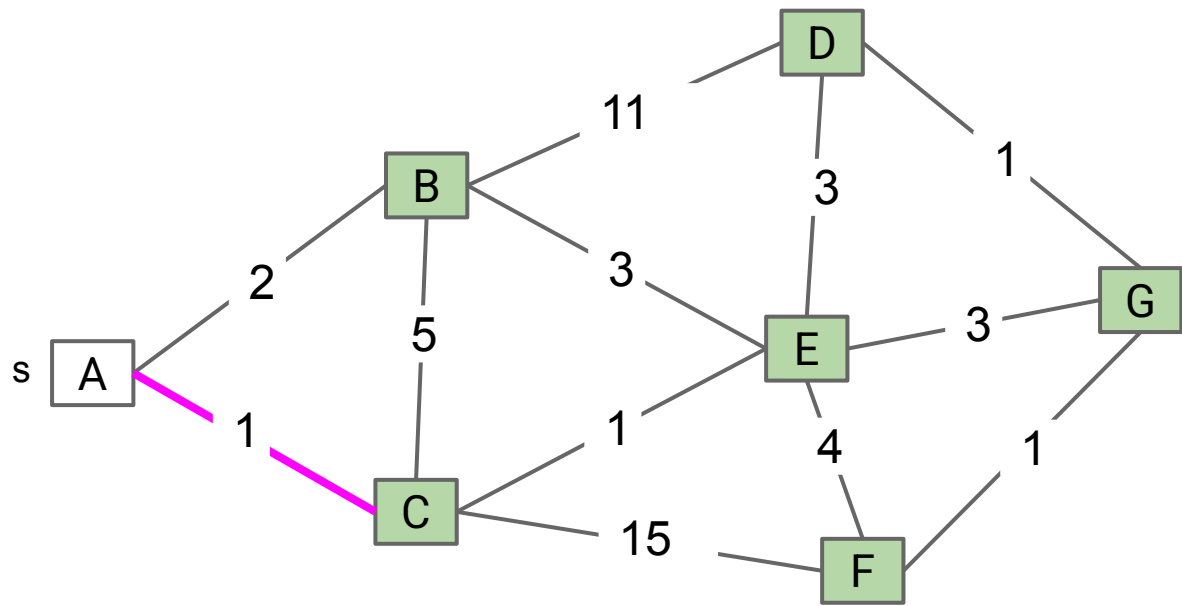
MST: []

Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created. Repeat until V-1 edges.

A-C	1
C-E	1
D-G	1
F-G	1
A-B	2
E-B	3
D-E	3
G-E	3
E-F	4
B-C	5
B-D	11
C-F	15

Cycle?



MST: []

White and green colorings for vertices show cut being implicitly utilized by Kruskal's algorithm.

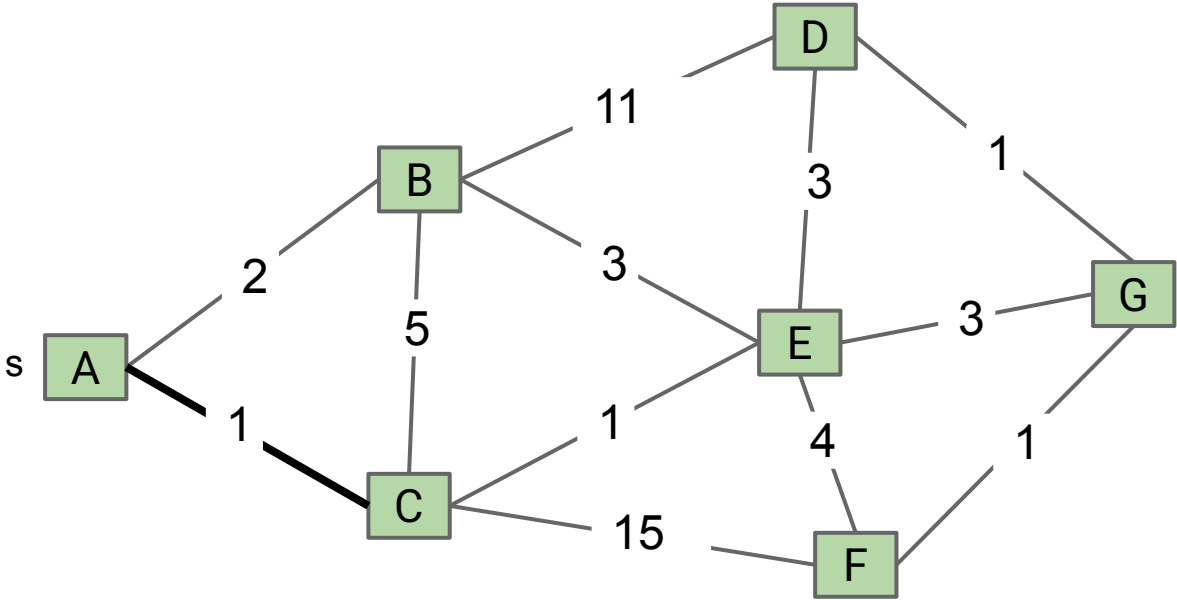


Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created. Repeat until V-1 edges.

A-C	1
C-E	1
D-G	1
F-G	1
A-B	2
E-B	3
D-E	3
G-E	3
E-F	4
B-C	5
B-D	11
C-F	15

Cycle? No.



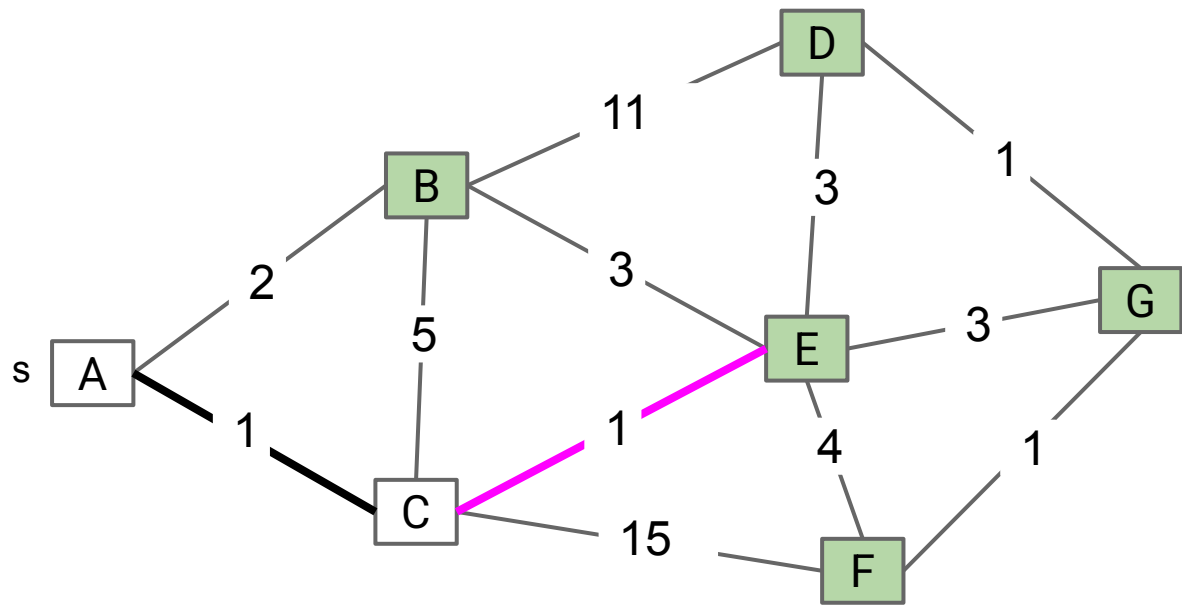
MST: [A-C]

Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created. Repeat until V-1 edges.

A-C	1
C-E	1
D-G	1
F-G	1
A-B	2
E-B	3
D-E	3
G-E	3
E-F	4
B-C	5
B-D	11
C-F	15

Cycle? No.



MST: [A-C]

White and green colorings for vertices show cut being implicitly utilized by Kruskal's algorithm.

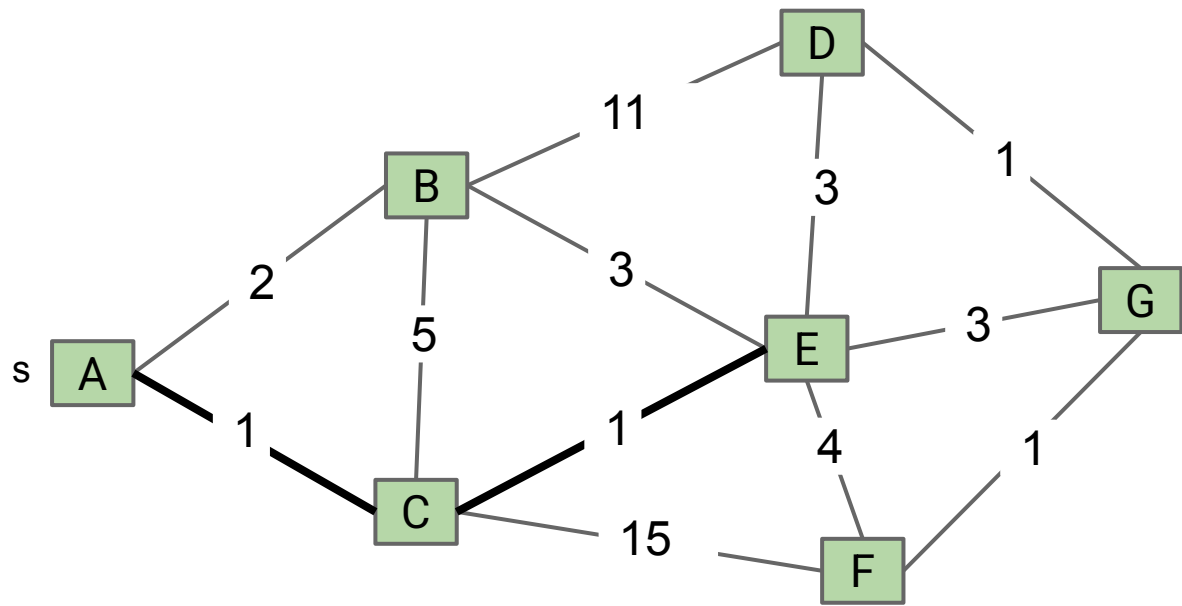


Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created. Repeat until V-1 edges.

A-C	1
C-E	1
D-G	1
F-G	1
A-B	2
E-B	3
D-E	3
G-E	3
E-F	4
B-C	5
B-D	11
C-F	15

Cycle? No.



MST: [A-C, C-E]

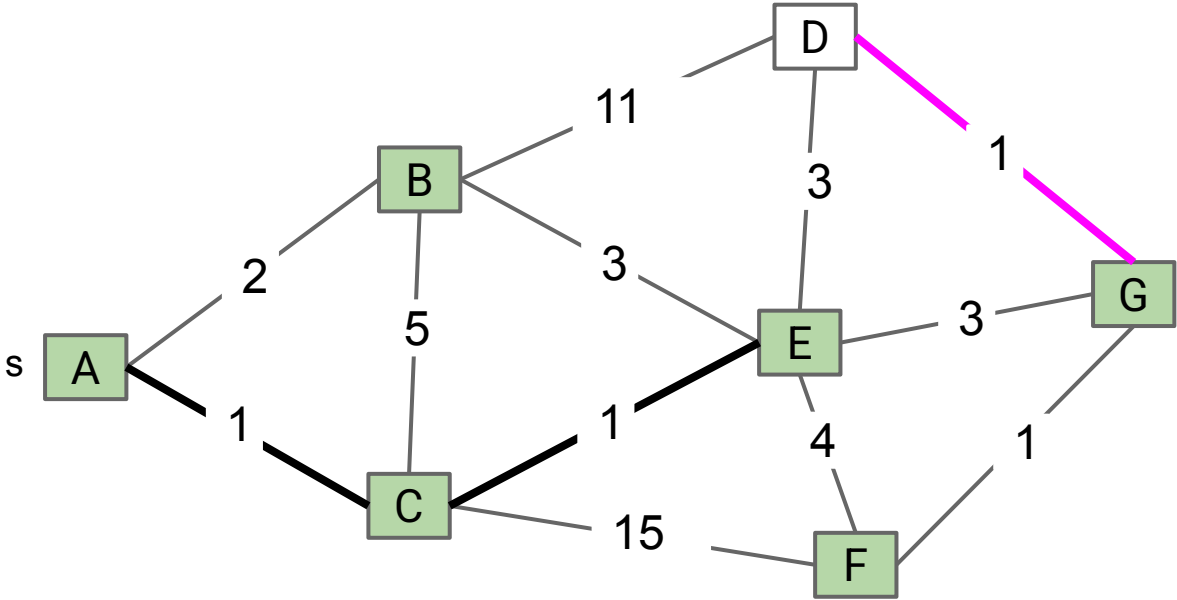


Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created. Repeat until V-1 edges.

A-C	1
C-E	1
D-G	1
F-G	1
A-B	2
E-B	3
D-E	3
G-E	3
E-F	4
B-C	5
B-D	11
C-F	15

Cycle?



MST: [A-C, C-E]

White and green colorings for vertices show cut being implicitly utilized by Kruskal's algorithm.

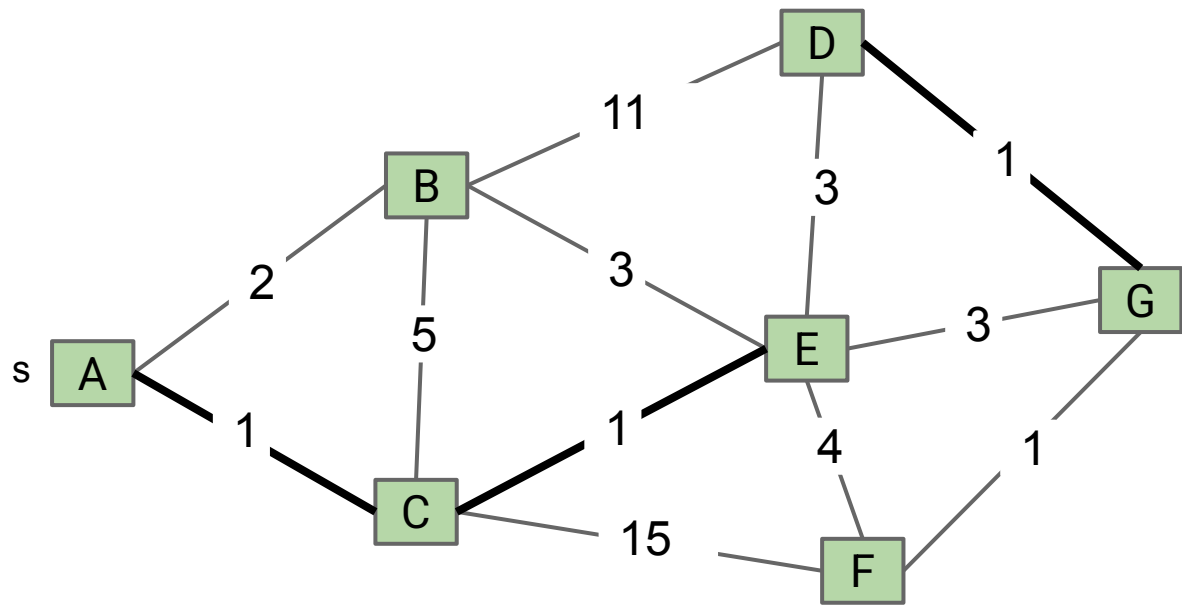


Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created. Repeat until V-1 edges.

A-C	1
C-E	1
D-G	1
F-G	1
A-B	2
E-B	3
D-E	3
G-E	3
E-F	4
B-C	5
B-D	11
C-F	15

Cycle? No.



MST: [A-C, C-E, D-G]

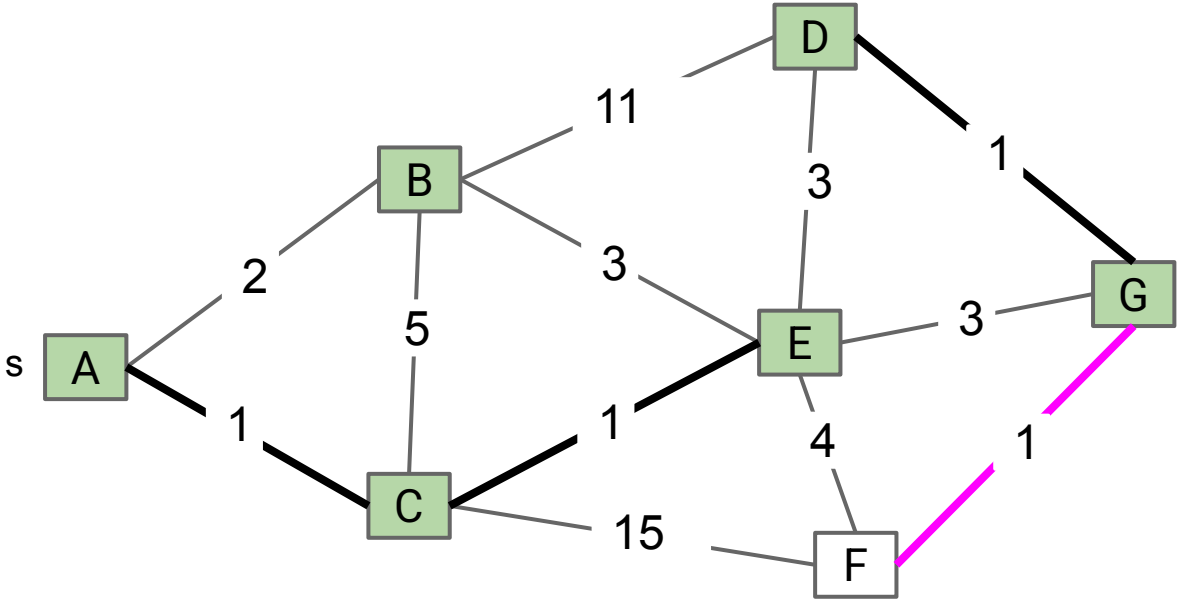


Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created. Repeat until V-1 edges.

A-C	1
C-E	1
D-G	1
F-G	1
A-B	2
E-B	3
D-E	3
G-E	3
E-F	4
B-C	5
B-D	11
C-F	15

Cycle?



MST: [A-C, C-E, D-G]

White and green colorings for vertices show cut being implicitly utilized by Kruskal's algorithm.

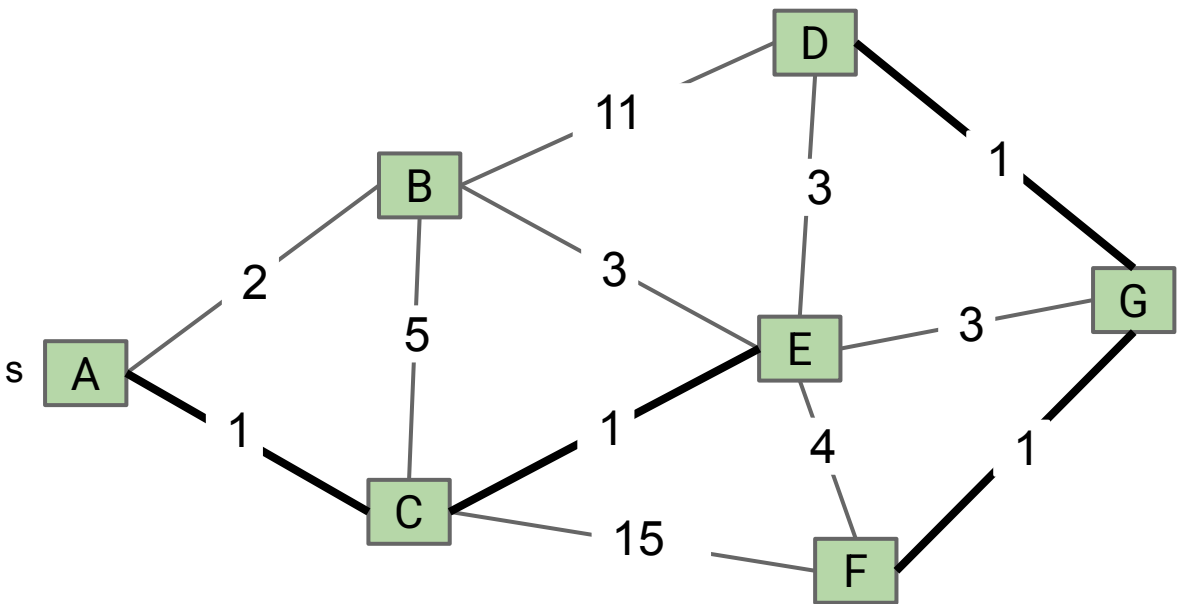


Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created. Repeat until V-1 edges.

A-C	1
C-E	1
D-G	1
F-G	1
A-B	2
E-B	3
D-E	3
G-E	3
E-F	4
B-C	5
B-D	11
C-F	15

Cycle? No.



MST: [A-C, C-E, D-G, F-G]

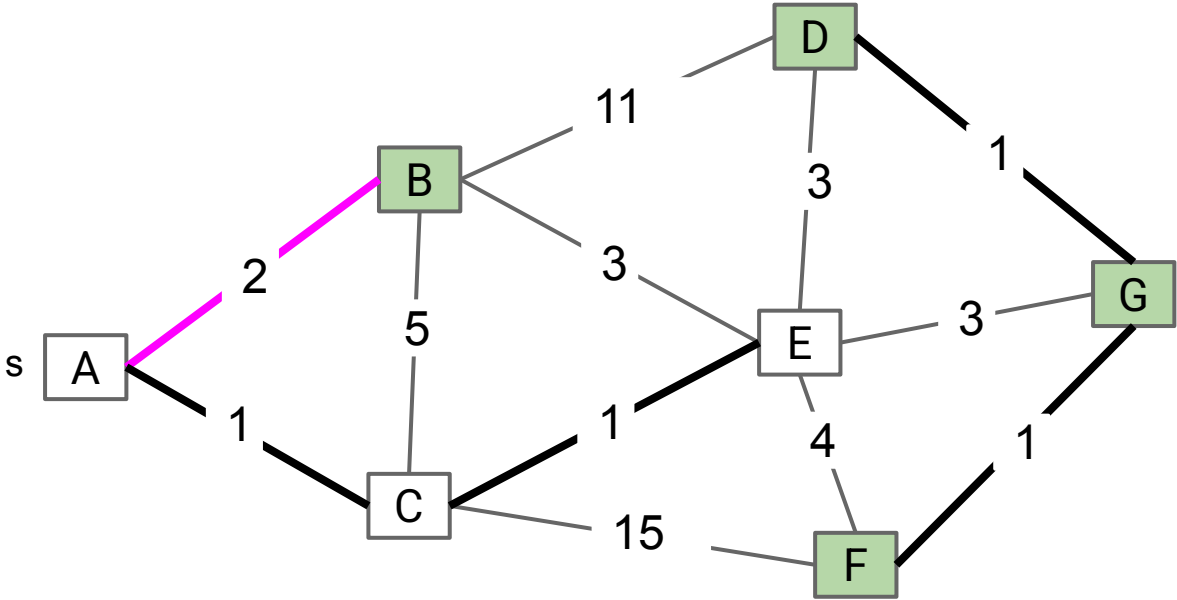


Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created. Repeat until V-1 edges.

A-C	1
C-E	1
D-G	1
F-G	1
A-B	2
E-B	3
D-E	3
G-E	3
E-F	4
B-C	5
B-D	11
C-F	15

Cycle?



MST: [A-C, C-E, D-G, F-G]

White and green colorings for vertices show cut being implicitly utilized by Kruskal's algorithm.

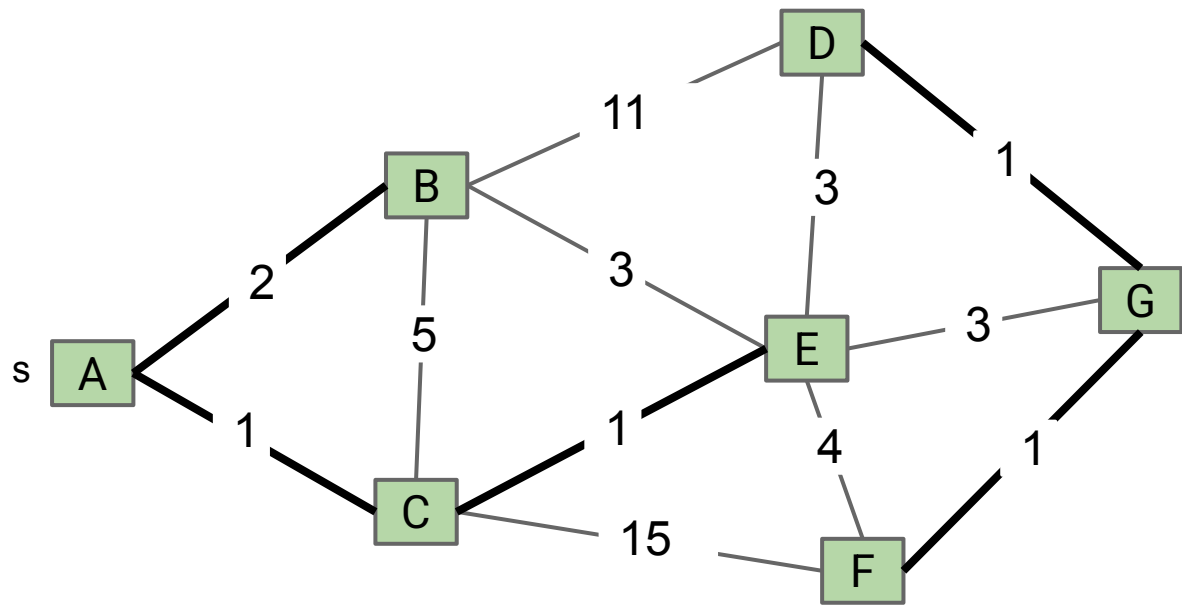


Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created. Repeat until V-1 edges.

A-C	1
C-E	1
D-G	1
F-G	1
A-B	2
E-B	3
D-E	3
G-E	3
E-F	4
B-C	5
B-D	11
C-F	15

Cycle? No.



MST: [A-C, C-E, D-G, F-G, A-B]

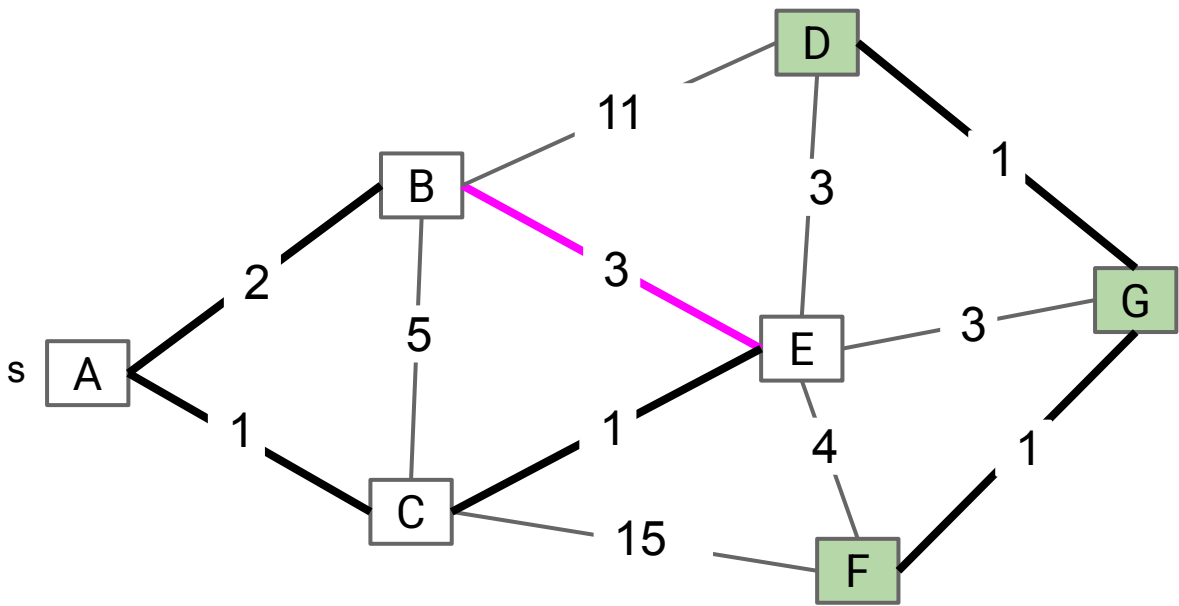


Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created. Repeat until V-1 edges.

A-C	1
C-E	1
D-G	1
F-G	1
A-B	2
E-B	3
D-E	3
G-E	3
E-F	4
B-C	5
B-D	11
C-F	15

Cycle?



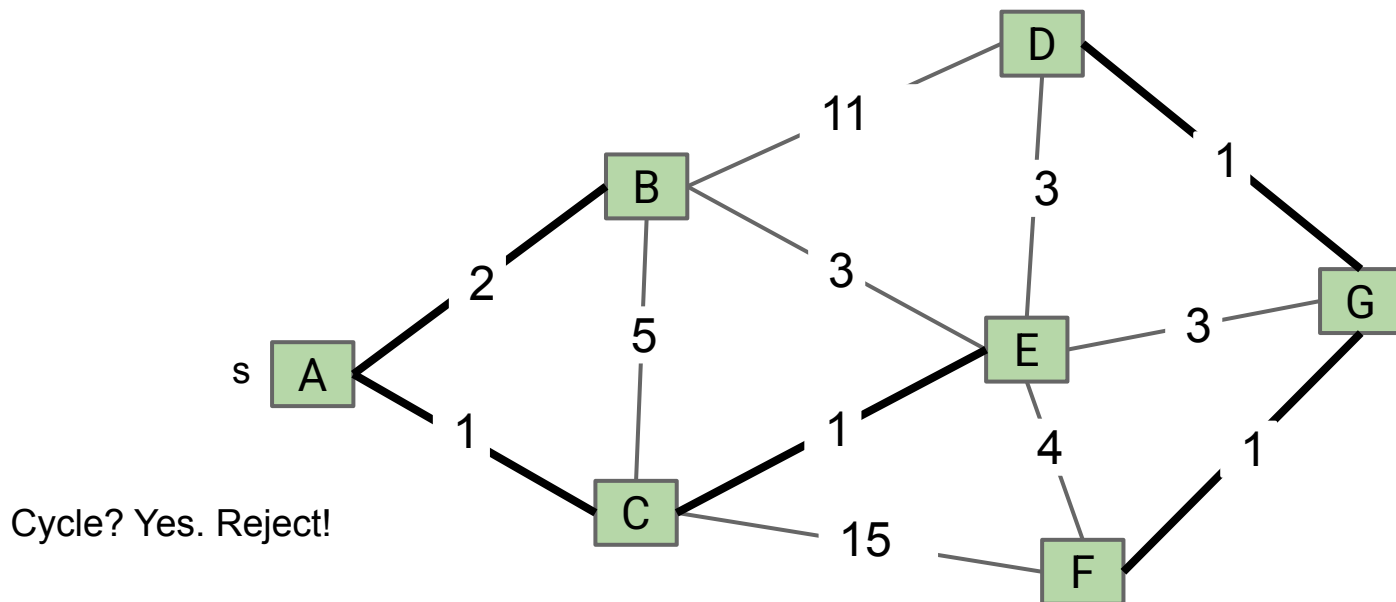
MST: [A-C, C-E, D-G, F-G, A-B]

White and green colorings for vertices show cut being implicitly utilized by Kruskal's algorithm.

Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created. Repeat until $V-1$ edges.

A-C	1
C-E	1
D-G	1
F-G	1
A-B	2
E-B	3
D-E	3
G-E	3
E-F	4
B-C	5
B-D	11
C-F	15



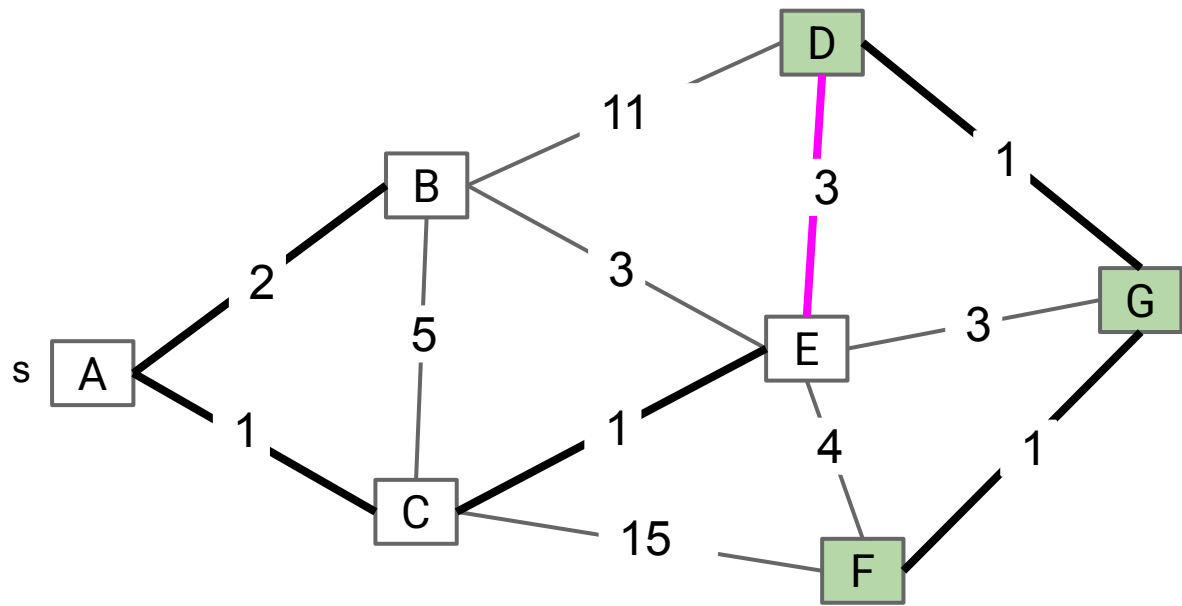
MST: [A-C, C-E, D-G, F-G, A-B]

Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created. Repeat until V-1 edges.

A-C	1
C-E	1
D-G	1
F-G	1
A-B	2
E-B	3
D-E	3
G-E	3
E-F	4
B-C	5
B-D	11
C-F	15

Cycle?



MST: [A-C, C-E, D-G, F-G, A-B]

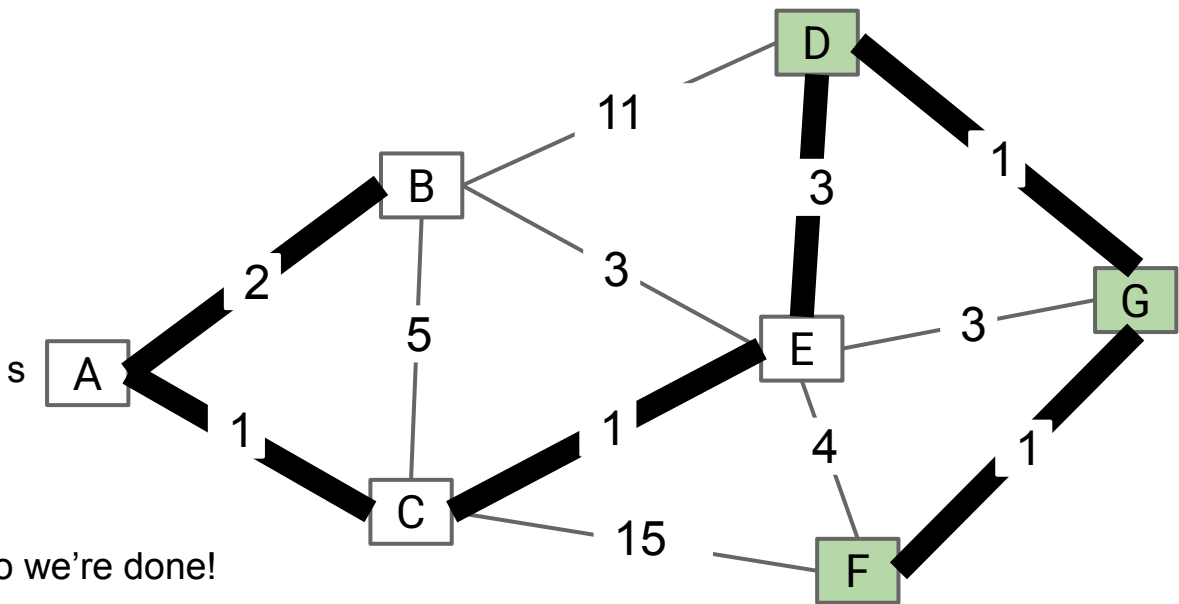
White and green colorings for vertices show cut being implicitly utilized by Kruskal's algorithm.



Kruskal's Demo

Consider edges in order of increasing weight. Add to MST unless a cycle is created. Repeat until V-1 edges.

A-C	1
C-E	1
D-G	1
F-G	1
A-B	2
E-B	3
D-E	3
G-E	3
E-F	4
B-C	5
B-D	11
C-F	15

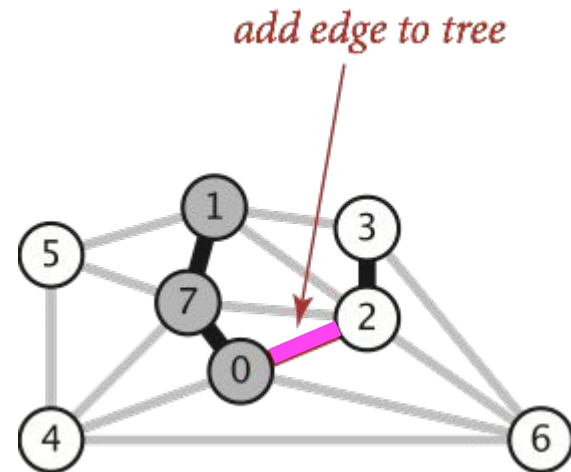


MST: [A-C, C-E, D-G, F-G, A-B, D-E]

Kruskal's Algorithm

Initially mark all edges gray (MST empty).

- Consider edges in increasing order of weight.
- Add edge to MST unless doing so creates a cycle.
- Repeat until $V-1$ edges.



Why does Kruskal's work? Special case of generic MST algorithm.

- Suppose we add edge $e = v \rightarrow w$.
- Side 1 of cut is all vertices connected to v , side 2 is everything else.
- No crossing edge is black (since we don't allow cycles).
- No crossing edge has lower weight (consider in increasing order).

Optimized Kruskal's (Demo)

Lecture 24, CS61B, Spring 2026

Minimum Spanning Trees

- Intro
- The Cut Property

Prim's Algorithm

- Basic Prim's (Demo)
- Optimized Prim's (Demo)
- Prim's Algorithm Code and Runtime

Kruskal's Algorithm:

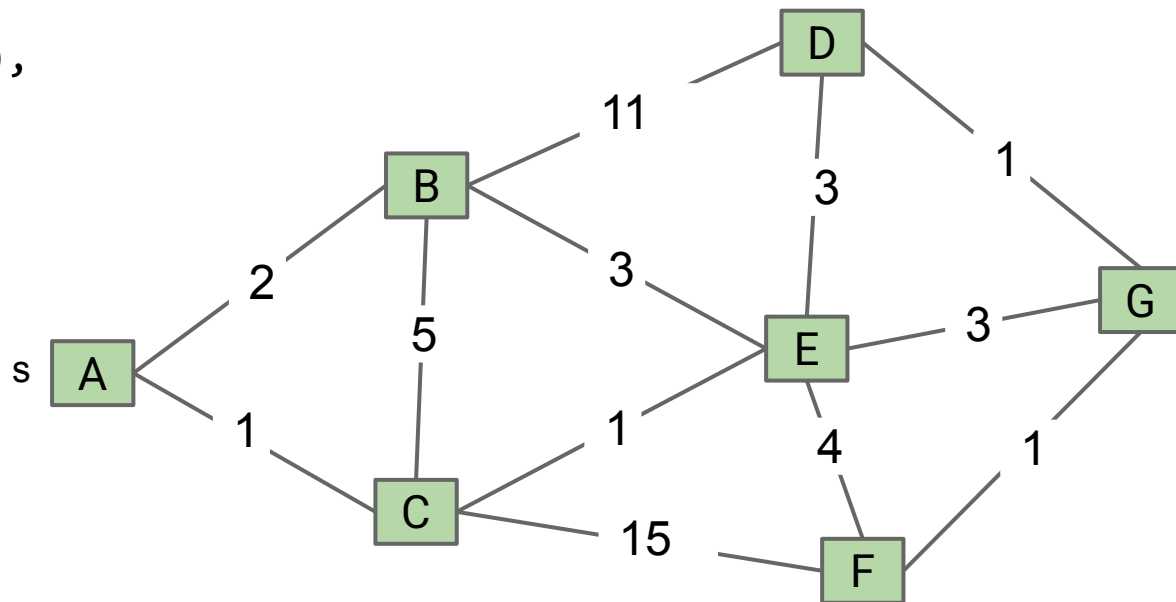
- Basic Kruskal's (Demo)
- **Optimized Kruskal's (Demo)**
- Kruskal's vs. Prim's
- Kruskal's Algorithm Code and Runtime

Kruskal's Demo

Insert all edges into PQ and all vertices into WQU

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

Fringe: (A-C: 1), (C-E: 1),
(D-G: 1), (F-G: 1),
(A-B: 2), (E-B: 3),
(D-E: 3), (G-E: 3),
(E-F: 4), (B-C: 5),
(B-D: 11), (C-F: 15)



WQU: [A, B, C, D, E, F, G]

MST: []

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

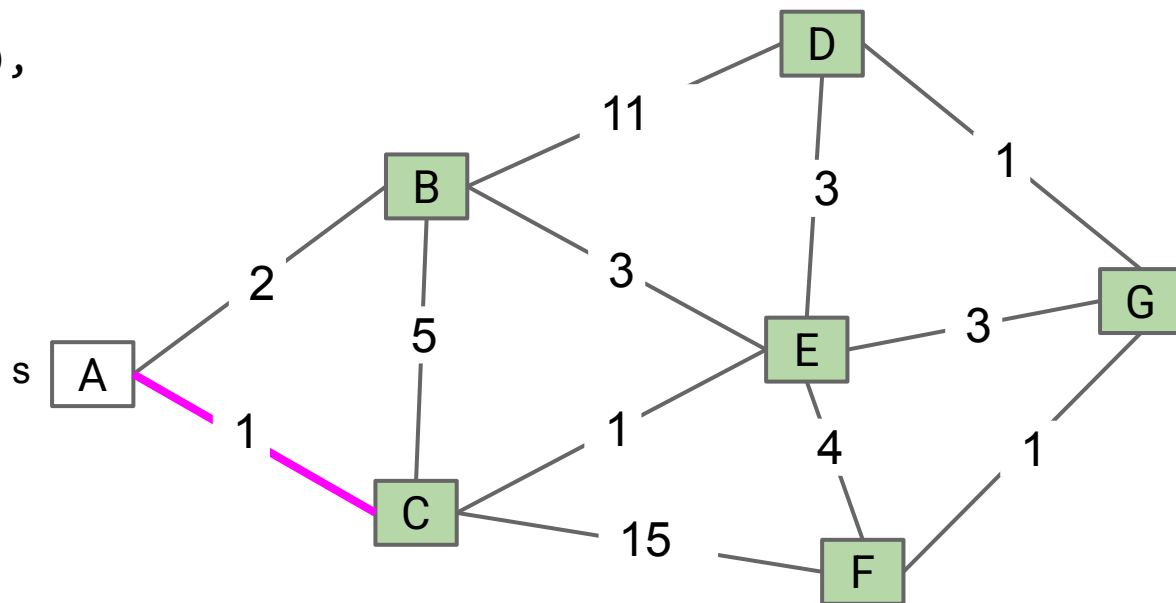
Fringe: ~~(A-C: 1)~~, (C-E: 1),
(D-G: 1), (F-G: 1),
(A-B: 2), (E-B: 3),
(D-E: 3), (G-E: 3),
(E-F: 4), (B-C: 5),
(B-D: 11), (C-F: 15)

Removed edge: A-C

Cycle? isConnected(A, C)

WQU: [A, B, C, D, E, F, G]

MST: []



Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

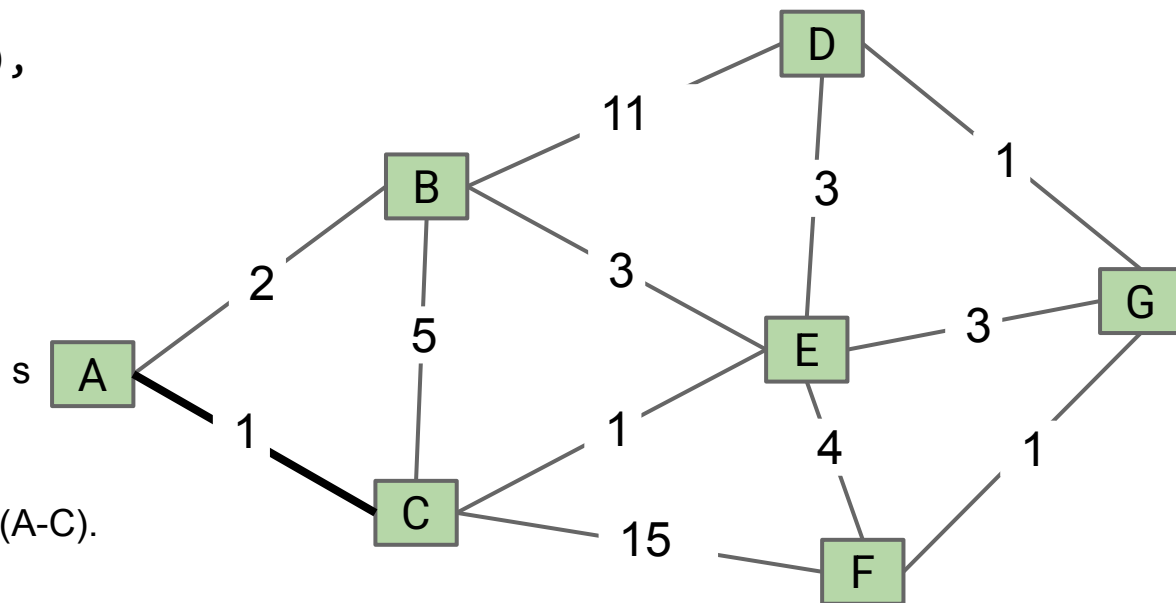
Fringe: ~~(A-C: 1)~~, (C-E: 1),
(D-G: 1), (F-G: 1),
(A-B: 2), (E-B: 3),
(D-E: 3), (G-E: 3),
(E-F: 4), (B-C: 5),
(B-D: 11), (C-F: 15)

Removed edge: A-C

Cycle? No. union(A, C). add(A-C).

WQU: [A-C, B, D, E, F, G]

MST: [A-C]



Kruskal's Demo

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

Fringe: ~~(A-C: 1)~~, ~~(C-E: 1)~~,

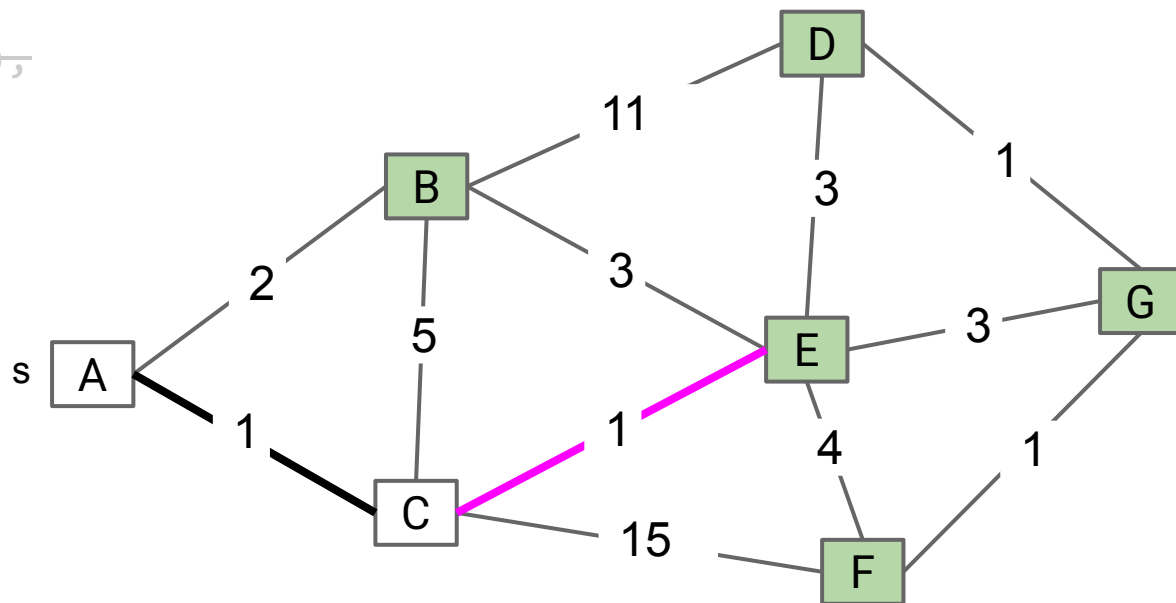
(D-G: 1), (F-G: 1),
(A-B: 2), (E-B: 3),
(D-E: 3), (G-E: 3),
(E-F: 4), (B-C: 5),
(B-D: 11), (C-F: 15)

Removed edge: C-E

Cycle? isConnected(C, E)

WQU: [A-C, B, D, E, F, G]

MST: [A-C]



Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

Fringe: ~~(A-C: 1)~~, ~~(C-E: 1)~~,

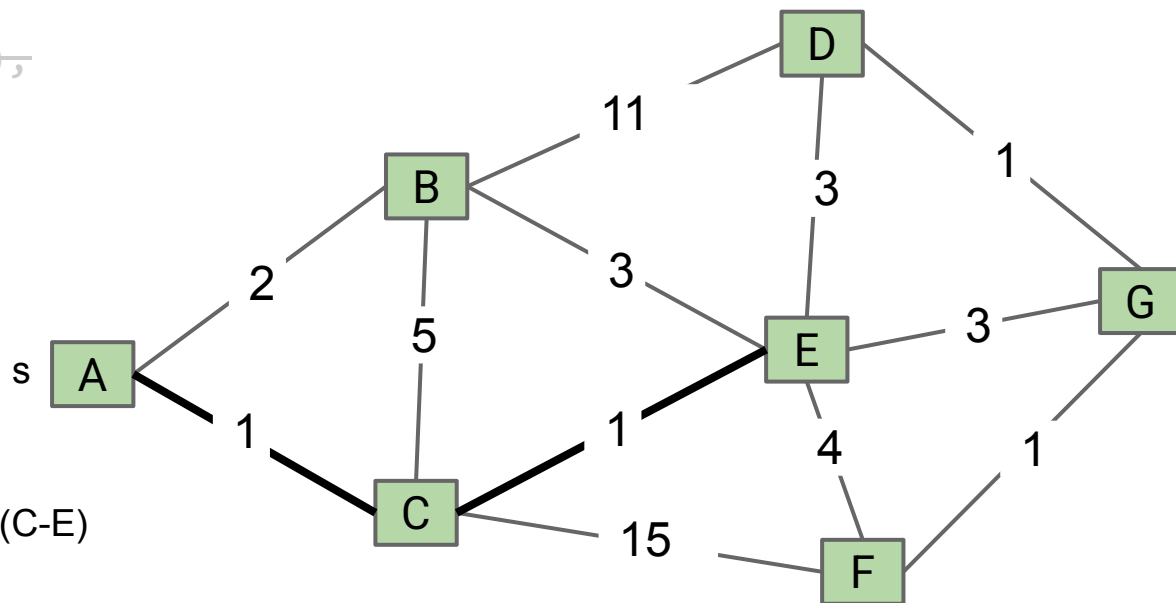
(D-G: 1), (F-G: 1),
(A-B: 2), (E-B: 3),
(D-E: 3), (G-E: 3),
(E-F: 4), (B-C: 5),
(B-D: 11), (C-F: 15)

Removed edge: C-E

Cycle? No. union(C, E). add(C-E)

WQU: [A-C-E, B, D, F, G]

MST: [A-C, C-E]



Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

Fringe: ~~(A-C: 1)~~, ~~(C-E: 1)~~,

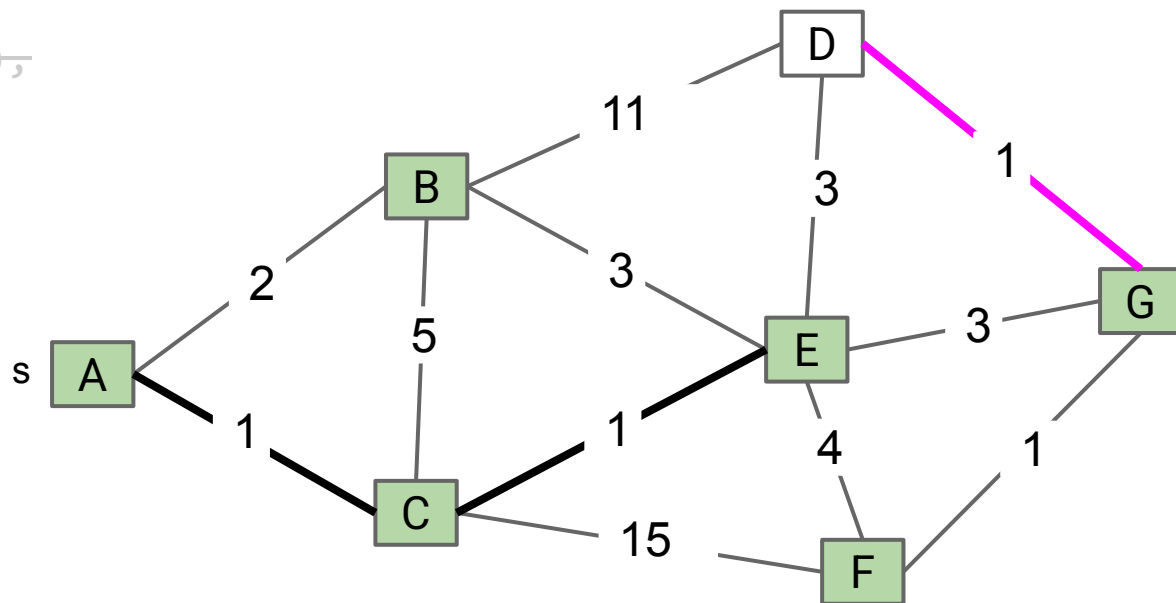
~~(D-G: 1)~~, (F-G: 1),

(A-B: 2), (E-B: 3),

(D-E: 3), (G-E: 3),

(E-F: 4), (B-C: 5),

(B-D: 11), (C-F: 15)



Removed edge: D-G

Cycle? isConnected(D, G)

WQU: [A-C-E, B, D, F, G]

MST: [A-C, C-E]

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

Fringe: ~~(A-C: 1)~~, ~~(C-E: 1)~~,

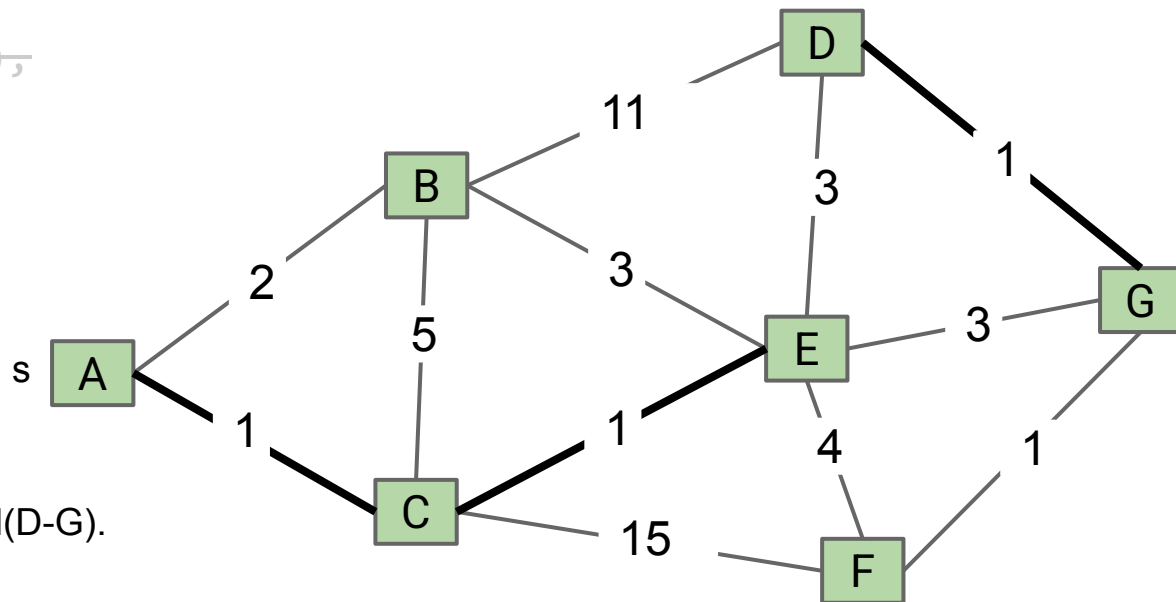
~~(D-G: 1)~~, (F-G: 1),

(A-B: 2), (E-B: 3),

(D-E: 3), (G-E: 3),

(E-F: 4), (B-C: 5),

(B-D: 11), (C-F: 15)



Removed edge: D-G

Cycle? No. union(D, G). add(D-G).

WQU: [A-C-E, **D-G**, B, F]

MST: [A-C, C-E, **D-G**]

Kruskal's Demo

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

Fringe: ~~(A-C: 1)~~, ~~(C-E: 1)~~,

~~(D-G: 1)~~, ~~(F-G: 1)~~,

(A-B: 2), (E-B: 3),

(D-E: 3), (G-E: 3),

(E-F: 4), (B-C: 5),

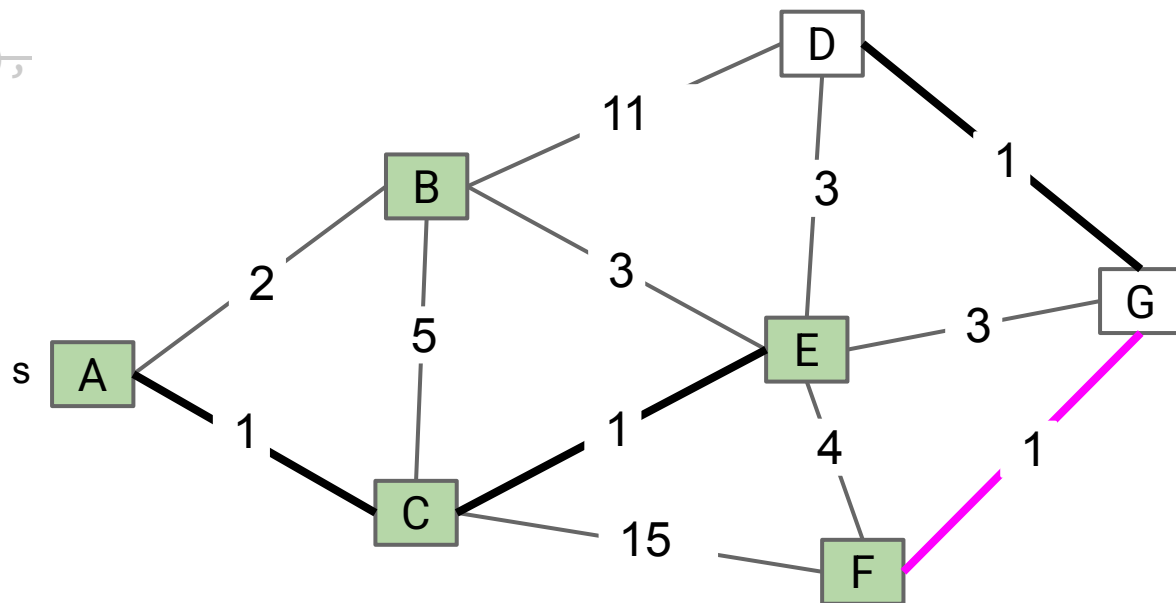
(B-D: 11), (C-F: 15)

Removed edge: F-G

Cycle? isConnected(F, G)

WQU: [A-C-E, D-G, B, F]

MST: [A-C, C-E, D-G]



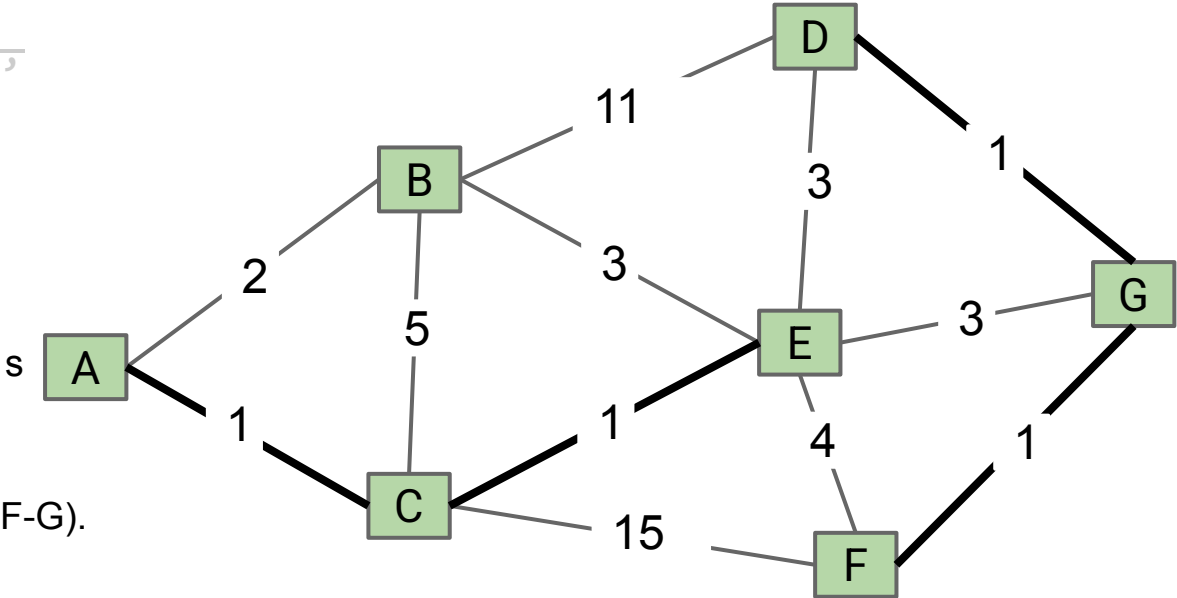
Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

Fringe: ~~(A-C: 1)~~, ~~(C-E: 1)~~,
~~(D-G: 1)~~, ~~(F-G: 1)~~,
(A-B: 2), (E-B: 3),
(D-E: 3), (G-E: 3),
(E-F: 4), (B-C: 5),
(B-D: 11), (C-F: 15)

Removed edge: F-G

Cycle? No. union(F, G). add(F-G).



WQU: [A-C-E, D-G-F, B]

MST: [A-C, C-E, D-G, F-G]



Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

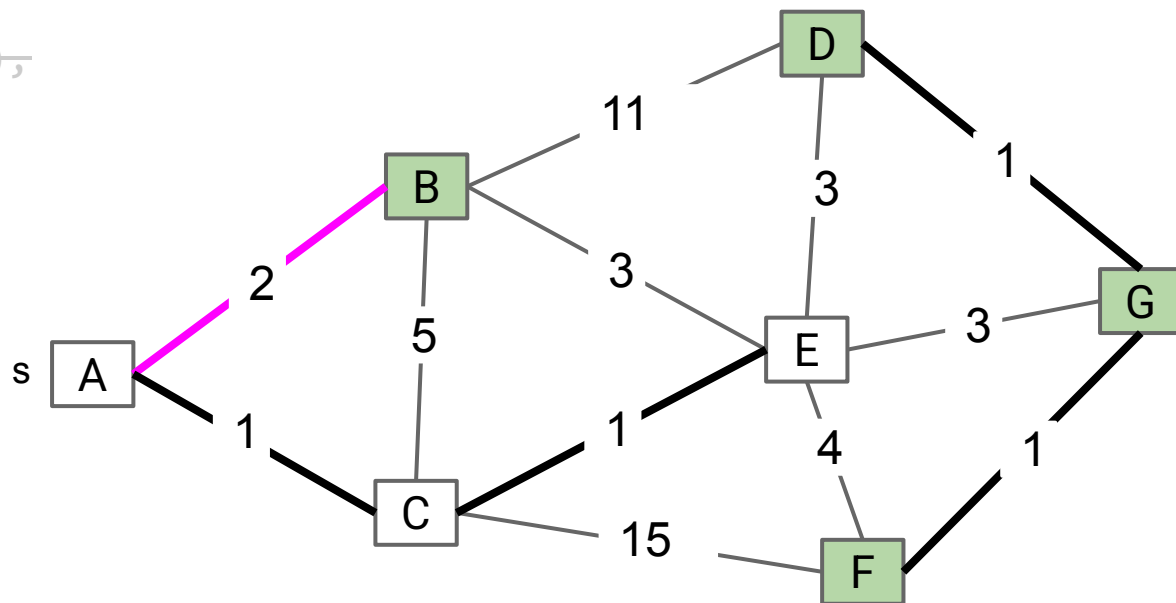
Fringe: ~~(A-C: 1)~~, ~~(C-E: 1)~~,
~~(D-G: 1)~~, ~~(F-G: 1)~~,
~~(A-B: 2)~~, (E-B: 3),
(D-E: 3), (G-E: 3),
(E-F: 4), (B-C: 5),
(B-D: 11), (C-F: 15)

Removed edge: A-B

Cycle? isConnected(A, B)

WQU: [A-C-E, D-G-F, B]

MST: [A-C, C-E, D-G, F-G]



Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

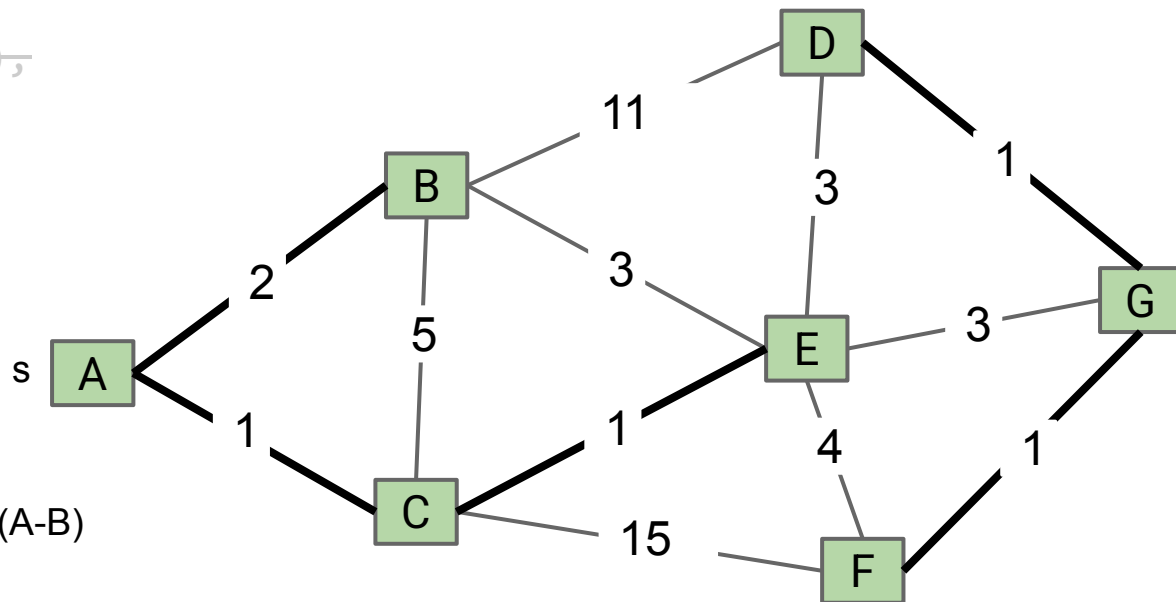
Fringe: ~~(A-C: 1)~~, ~~(C-E: 1)~~,
~~(D-G: 1)~~, ~~(F-G: 1)~~,
~~(A-B: 2)~~, (E-B: 3),
(D-E: 3), (G-E: 3),
(E-F: 4), (B-C: 5),
(B-D: 11), (C-F: 15)

Removed edge: A-B

Cycle? No. union(A, B). add(A-B)

WQU: [A-C-E-B, D-G-F]

MST: [A-C, C-E, D-G, F-G, A-B]



Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

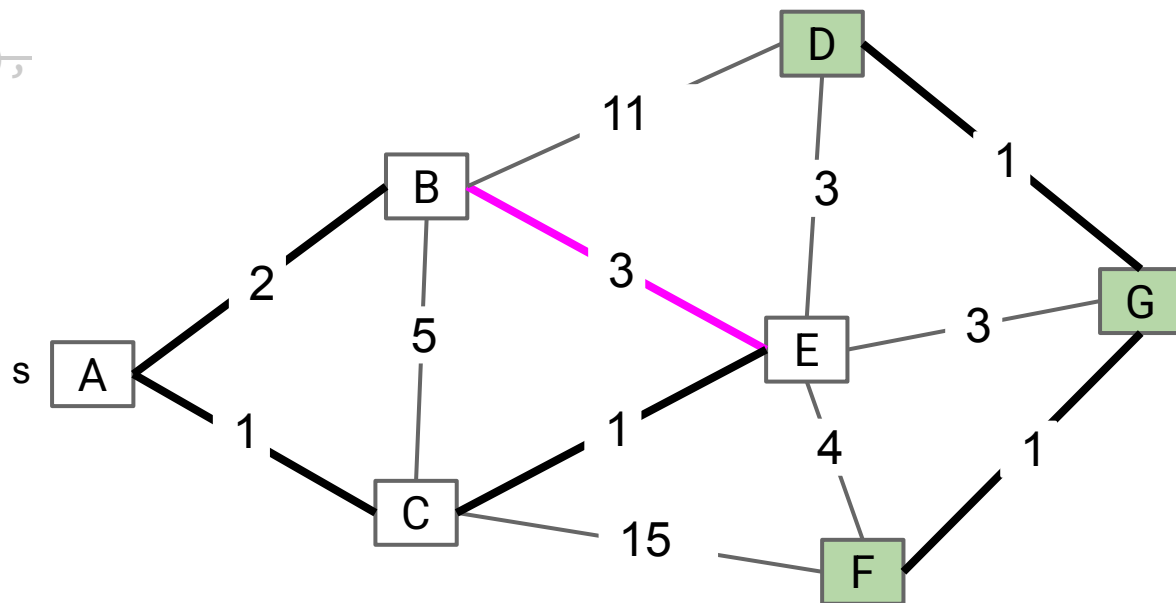
Fringe: ~~(A-C: 1)~~, ~~(C-E: 1)~~,
~~(D-G: 1)~~, ~~(F-G: 1)~~,
~~(A-B: 2)~~, ~~(E-B: 3)~~,
(D-E: 3), (G-E: 3),
(E-F: 4), (B-C: 5),
(B-D: 11), (C-F: 15)

Removed edge: E-B

Cycle? isConnected(E, B)

WQU: [A-C-E-B, D-G-F]

MST: [A-C, C-E, D-G, F-G, A-B]



Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

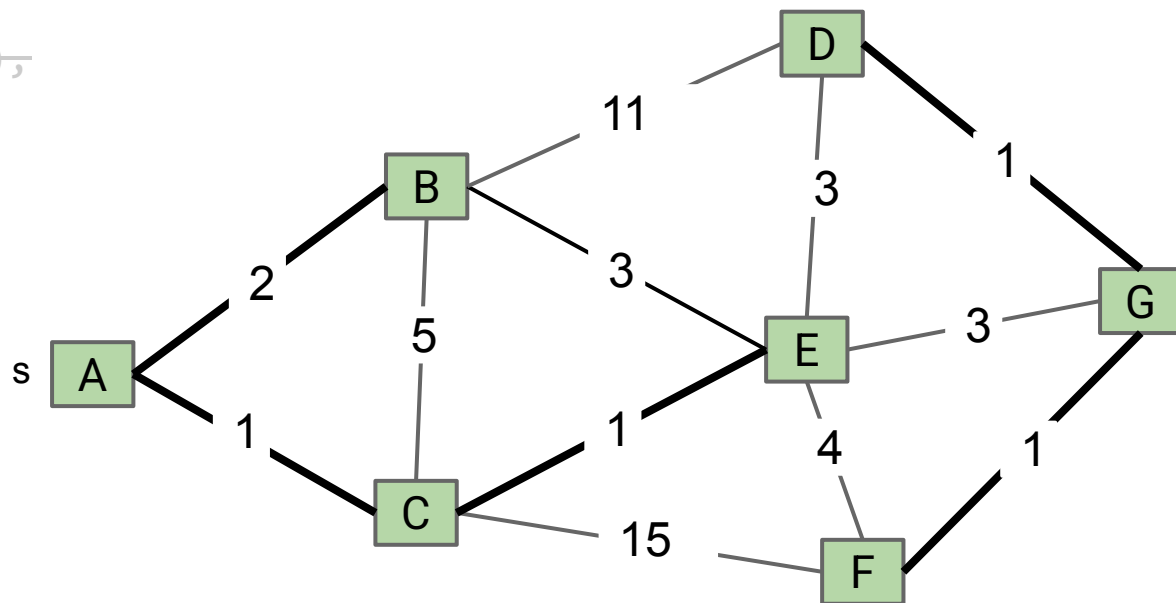
Fringe: ~~(A-C: 1)~~, ~~(C-E: 1)~~,
~~(D-G: 1)~~, ~~(F-G: 1)~~,
~~(A-B: 2)~~, ~~(E-B: 3)~~,
(D-E: 3), (G-E: 3),
(E-F: 4), (B-C: 5),
(B-D: 11), (C-F: 15)

Removed edge: E-B

Cycle? Yes. Do nothing.

WQU: [A-C-E-B, D-G-F]

MST: [A-C, C-E, D-G, F-G, A-B]



Kruskal's Demo

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

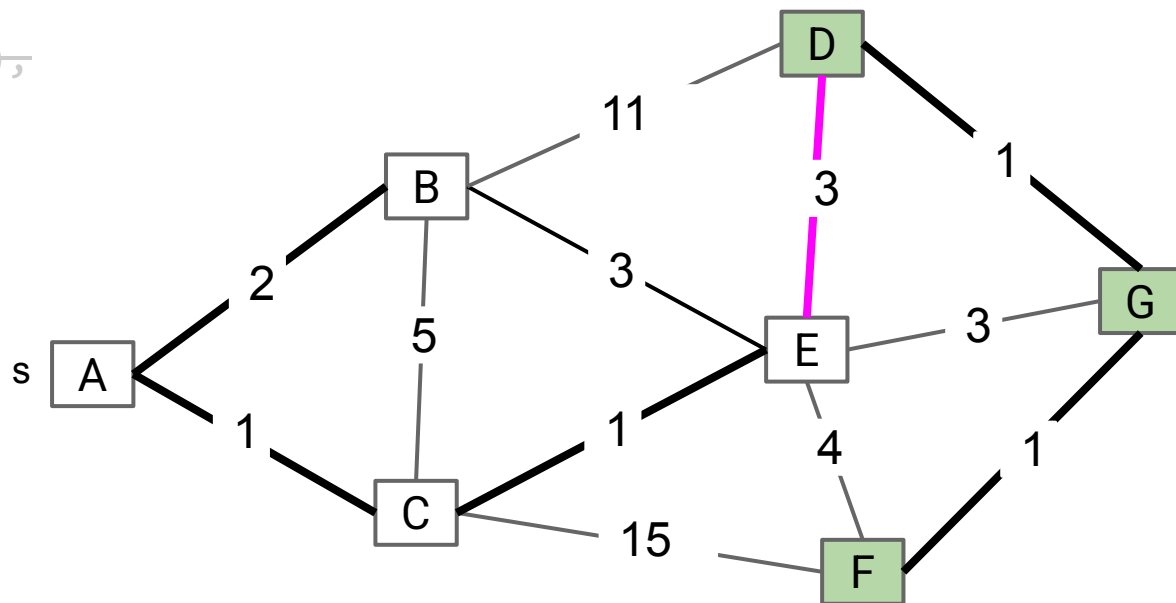
Fringe: ~~(A-C: 1)~~, ~~(C-E: 1)~~,
~~(D-G: 1)~~, ~~(F-G: 1)~~,
~~(A-B: 2)~~, ~~(E-B: 3)~~,
~~(D-E: 3)~~, (G-E: 3),
(E-F: 4), (B-C: 5),
(B-D: 11), (C-F: 15)

Removed edge: D-E

Cycle? isConnected(D, E)

WQU: [A-C-E-B, D-G-F]

MST: [A-C, C-E, D-G, F-G, A-B]



Kruskal's Demo

Insert all edges into PQ.

Repeat: Remove smallest weight edge. Add to MST if no cycle created.

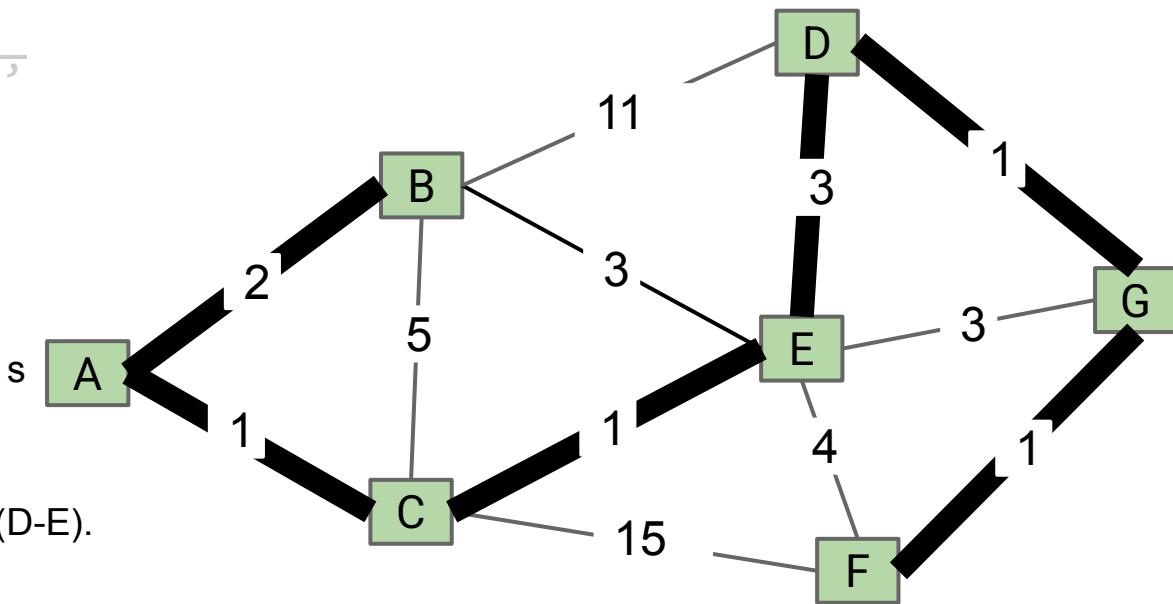
Fringe: ~~(A-C: 1)~~, ~~(C-E: 1)~~,
~~(D-G: 1)~~, ~~(F-G: 1)~~,
~~(A-B: 2)~~, ~~(E-B: 3)~~,
~~(D-E: 3)~~, (G-E: 3),
(E-F: 4), (B-C: 5),
(B-D: 11), (C-F: 15)

Removed edge: D-E

Cycle? No. union(D, E). add(D-E).

WQU: [A-C-E-B-D-G-F]

MST: [A-C, C-E, D-G, F-G, A-B, D-E]



V-1 edges, so done.

Kruskal's vs. Prim's

Lecture 24, CS61B, Spring 2026

Minimum Spanning Trees

- Intro
- The Cut Property

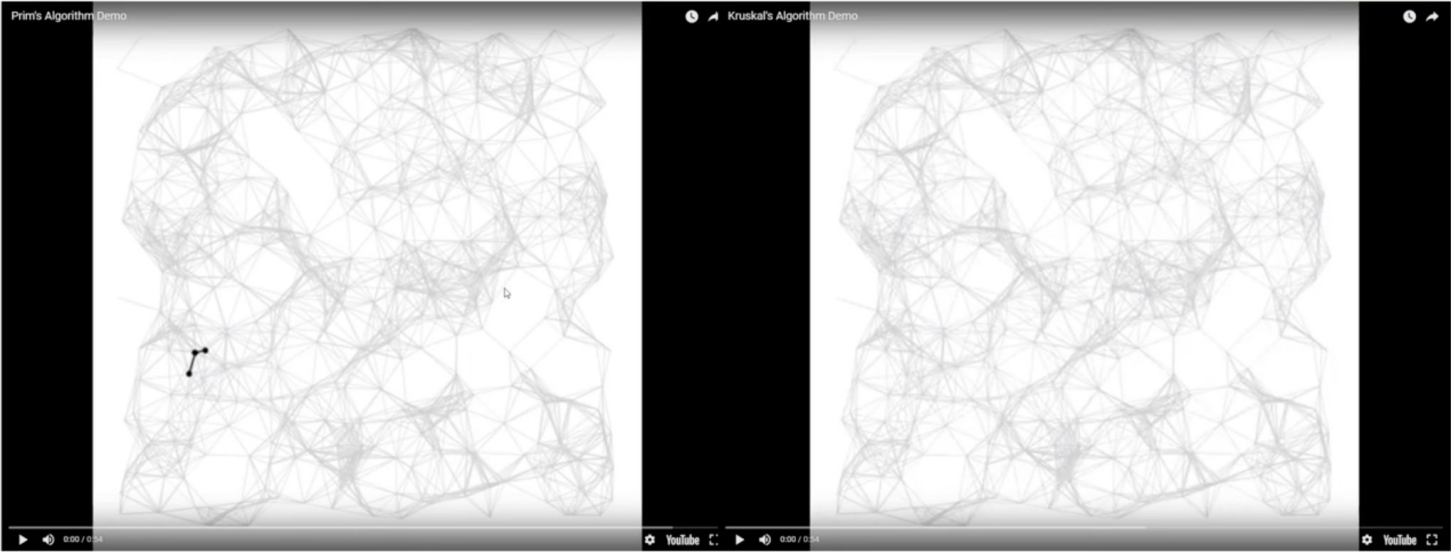
Prim's Algorithm

- Basic Prim's (Demo)
- Optimized Prim's (Demo)
- Prim's Algorithm Code and Runtime

Kruskal's Algorithm:

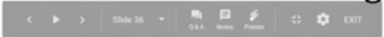
- Basic Kruskal's (Demo)
- Optimized Kruskal's (Demo)
- **Kruskal's vs. Prim's**
- Kruskal's Algorithm Code and Runtime

Prim's vs. Kruskal's



Prim's Algorithm

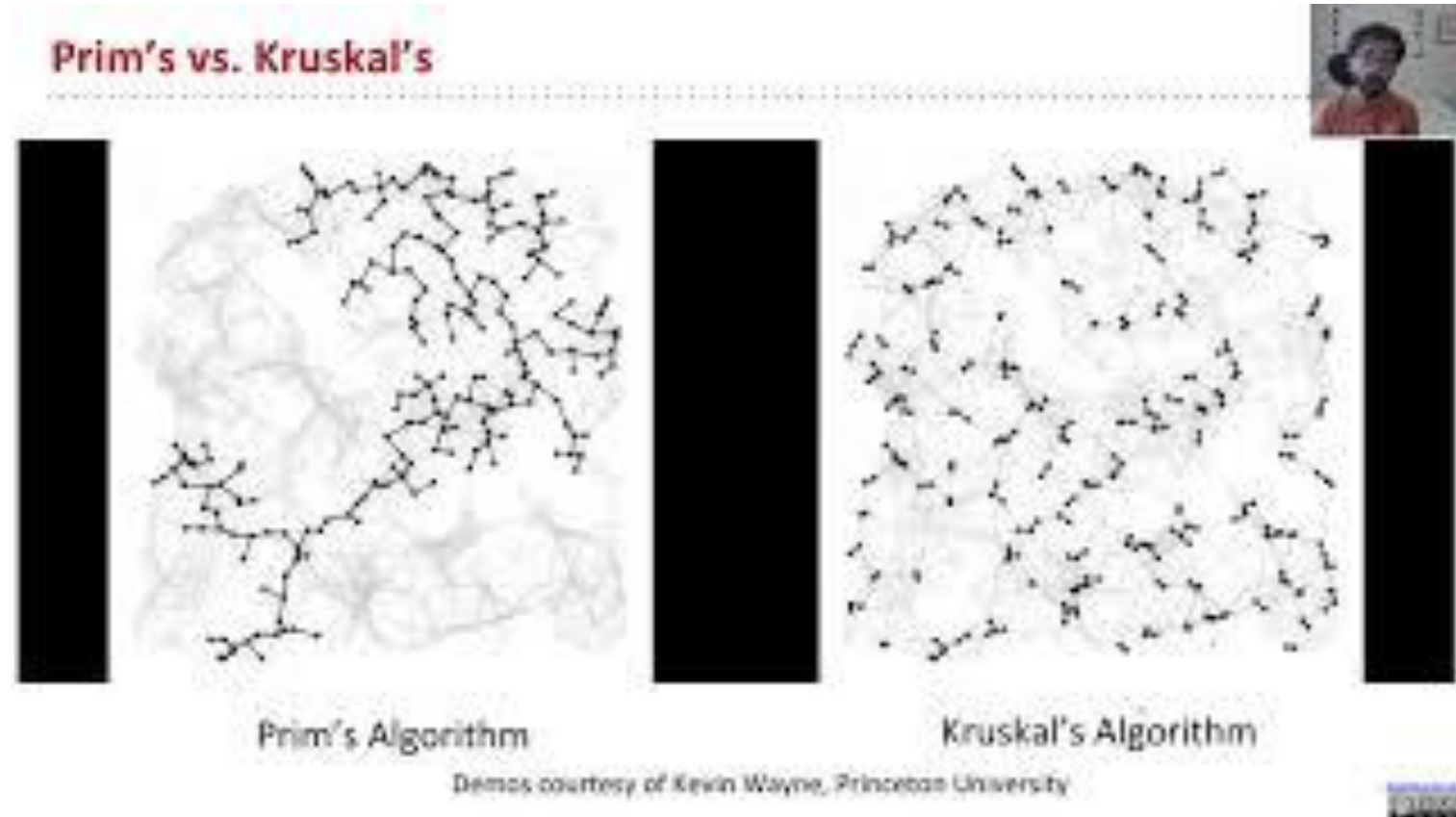
Kruskal's Algorithm



Demos courtesy of Kevin Wayne, Princeton University



Prim's vs. Kruskal's (visual)



Demos courtesy of Kevin Wayne, Princeton University

Kruskal's Algorithm Code and Runtime

Lecture 24, CS61B, Spring 2026

Graph Problem Warmup
Minimum Spanning Trees

- Intro
- The Cut Property

Prim's Algorithm

- Basic Prim's (Demo)
- Optimized Prim's (Demo)
- Prim's Algorithm Code and Runtime

Kruskal's Algorithm:

- Basic Kruskal's (Demo)
- Optimized Kruskal's (Demo)
- Kruskal's vs. Prim's
- **Kruskal's Algorithm Code and Runtime**

Kruskal's Implementation (Pseudocode)

```
public class KruskalMST {  
    private List<Edge> mst = new ArrayList<Edge>();  
  
    public KruskalMST(EdgeWeightedGraph G) {  
        MinPQ<Edge> pq = new MinPQ<Edge>();  
        for (Edge e : G.edges()) {  
            pq.insert(e);  
        }  
        WeightedQuickUnionPC uf =  
            new WeightedQuickUnionPC(G.V());  
        while (!pq.isEmpty() && mst.size() < G.V() - 1) {  
            Edge e = pq.delMin();  
            int v = e.from();  
            int w = e.to();  
            if (!uf.connected(v, w)) {  
                uf.union(v, w);  
                mst.add(e);  
            }  
        }  
    }  
}
```

What is the runtime of Kruskal's algorithm?

- Assume all PQ operations take $O(\log(V))$ time.
- Assume all WQU operations take $O(\log^* V)$ time.
- Give your answer in Big O notation.

Kruskal's algorithm on previous slide is $O(E \log E)$.

Fun fact: In HeapSort lecture, we will discuss how do this step in $O(E)$ time using “bottom-up heapification”.

Operation	Number of Times	Time per Operation	Total Time
Insert	E	$O(\log E)$	$O(E \log E)$
Delete minimum	$O(E)$	$O(\log E)$	$O(E \log E)$
union	$O(V)$	$O(\log^* V)$	$O(V \log^* V)$
isConnected	$O(E)$	$O(\log^* V)$	$O(E \log^* V)$

Note 1: If we use a pre-sorted list of edges (instead of a PQ), then we can simply iterate through the list in $O(E)$ time, so overall runtime is $O(E + V \log^* V + E \log^* V) = O(E \log^* V)$.

Note 2: $E < V^2$, so $\log E < \log V^2 = 2 \log V$, so $O(E \log E) = O(E \log V)$. So while Kruskal's algorithm will be slower than Prim's algorithm for a worst-case unsorted set of edges, it won't be asymptotically slower.

Shortest Paths and MST Algorithms Summary

Problem	Algorithm	Runtime (if $E > V$)	Notes
Shortest Paths	Dijkstra's	$O(E \log V)$	Fails for negative weight edges.
MST	Prim's	$O(E \log V)$	Analogous to Dijkstra's.
MST	Kruskal's	$O(E \log E)$	Uses WQUPC.
MST	Kruskal's with pre-sorted edges	$O(E \log^* V)$	Uses WQUPC.

Question: Can we do better than $O(E \log V)$? See bonus slides.

These slides are covered in the [web videos](#), but we won't cover them live.

Extra: MST Algorithm History

Lecture 24, CS61B, Spring 2026

Minimum Spanning Trees

- Intro
- The Cut Property

Prim's Algorithm

- Basic Prim's (Demo)
- Optimized Prim's (Demo)
- Prim's Algorithm Code and Runtime

Kruskal's Algorithm:

- Basic Kruskal's (Demo)
- Optimized Kruskal's (Demo)
- Kruskal's vs. Prim's
- Kruskal's Algorithm Code and Runtime

170 Spoiler: State of the Art Compare-Based MST Algorithms

year	worst case	discovered by
1975	$E \log \log V$	Yao
1984	$E \log^* V$	Fredman-Tarjan
1986	$E \log (\log^* V)$	Gabow-Galil-Spencer-Tarjan
1997	$E \alpha(V) \log \alpha(V)$	Chazelle
2000	$E \alpha(V)$	Chazelle
2002	optimal (link)	Pettie-Ramachandran
???	$E ???$	???

(Slide Courtesy of Kevin Wayne, Princeton University)